AI-Driven Root Cause Analysis and Automated Triage in Micro-service Environments

Akshay Adik

Doctor Of Business Administration, Golden Gate University

Chair Name: Dr. Anil Kumar Vuppala

September 5, 2026

Table of Contents

1 Introduction **9**

1.1 Background and Context 9

1.2 Problem Motivation 11

1.2.1 Nature of Failures in Micro-service Environment 11

1.3 Significance of Root Cause Analysis 15

1.3.1 Mean-Time-To-Resolution (MTTR) and Economic Loss 15

1.3.2 Explainability and Developer Trust 15

1.3.3 Industrial Adoption 16

1.4 Problem Statement 17

1.4.1 Breakdown of main problem 18

1.5 Research Aim 18

1.6 Research Objective 19

1.7 Research Questions 19

1.8 Testable Hypotheses 20

1.9 Conceptual Framework 20

1.10 Limitation 21

1.11 Study Scope 21

1.12 Assumptions 22

1.13 Definition of Key Terms 22

2 Literature Review **25**

2.1 Historical evolution of Root Cause Analysis (RCA) 29
2.2 Building Blocks of Root Cause Analysis (RCA) 31
2.2.1 Anomaly Detection (AD) 31
2.2.2 Root Cause Localization and Causal Inference (RCL/CI) 32
2.2.3 Interpretation, Explanation and Human Feedback 33
2.2.4 Auto-Remediation (AR) and Mitigation 35
2.2.5 Three Pillar of Observability 37
2.3 Data Preprocessing 40
2.3.1 Data Cleaning and Filtering 41
2.3.2 Data Transformation and Feature Extraction 45
2.3.3 Normalization and Scaling 51
2.3.4 Time Series Alignment 52
2.4 Data Analysis 55
2.5 AI Taxonomy 56
2.5.1 Foundation Capabilities and Constraints of RCA Systems 56
2.5.2 Statistical Modeling Approaches to RCA 57
2.5.3 Classical Machine Learning Approaches 61
2.5.4 Deep Learning for RCA 65
2.5.5 Generative AI and LLM-Driven RCA 71
2.5.6 Multi-modal Data Fusion and Hybrid Intelligence 75
2.5.7 AIOps as an End-to-End RCA Paradigm 77
2.6 Economics 80
2.6.1 ROI Framework 81

2.7 Gaps and Future 92

2.7.1 Technical and Data Challenges 93

2.7.2 **Operational Gap** 96

2.7.3 Economic Gap 97

2.7.4 Research Gap 98

3 Research Method and Design **101**

3.1 Introduction 101

3.2 Research Philosophy 102

3.3 Research Design 104

3.3.1 Research Design Classification 104

3.3.2 Design Science Research (DSR) Framework 104

3.3.3 The Intervention Strategy 106

3.3.4 Unit of Analysis 110

3.3.5 Time Horizon 111

3.3.6 Variables and Constructs 112

3.4 Research Setting and Context 116

3.4.1 The Architectural Setting 116

3.4.2 The Business and Operational Context: High-Stakes Reliability 117

3.4.3 The Data Context: The Three Pillars of Observability 118

3.4.4 The Technological Setting: The Emergence of AIOps 119

3.5 Business Evaluation Layer 121

3.5.1 Managerial application 125

3.5.2 Organizational adoption 126
3.5.3 Governance 128
3.6 Population and Sampling 130
3.7 The AI Artifact Development (The Intervention) 135
3.7.1 Research Artifact Definition 135
3.7.2 Architectural Design Principles 136
3.7.3 System Architecture Specification 138
3.7.4 Data Flow and Control Flow 142
3.7.5 Formal Algorithms 143
3.7.6 Observability Planes and Layers 147
3.7.7 Ablation-Aware Architecture 147
3.7.8 Embedded Novelty 153
3.8 Data Collection Procedures 159
3.8.1 Exploratory Human Evaluation (Research Question 4) 161
3.9 Data Analysis Plan 165
3.9.1 Research Questions and Hypotheses 165
3.9.2 Research Question–Hypothesis–Architecture Mapping 167
3.9.3 Statistical Methods 190
3.9.4 Evaluation Metrics 202
3.10 Validity and Reliability 203
3.10.1 Internal Validity 203
3.10.2 Construct Validity 206
3.10.3 External Validity 207

3.10.4 Reliability 208
3.11 Ethical Considerations 210
3.12 Limitations of the Methodology 212
3.12.1 Core Assumptions and Their Scope Conditions 212
3.12.2 Corpus and Evidential-Status Limitations 213
3.12.3 Technical Limitations 214
3.12.4 Methodological Trade-offs 215
3.12.5 Scalability and External-Replication Boundaries 216
3.13 Summary 216

A Appendix **219**

A.1 Appendix A — Acronyms used in this chapter 219
A.2 Appendix B — Reproducibility and Pre-Warming Artefacts 221
A.2.1 Reproducibility and Pre-Warming Artefacts 221

References **233**

List of Tables **6**

List of Figures **8**

List of Tables

Table 1. Definition of Key Terms 22

Table 2. Anomalies Classification and Symptoms 26

Table 3. Anomaly Origin in the Software Development Life Cycle (SDLC) 28

Table 4. Data Preprocessing Requirements and Techniques Across Observability Data Modalities 41

Table 5. Data Transformation requirement 45

Table 6. Normalization and Scaling Requirement 51

Table 7. Temporal Alignment and Time-Series Processing Techniques Across Observability Data 53

Table 8. Financial and Operational Impact of IT Service Disruptions 82

Table 9. Operational Efficiency and Performance Improvements from AI-Driven Root Cause Analysis 85

Table 10. Annual Inference Cost by Incident Volume 88

Table 11. Cost and Resource Requirements of AI-Driven Root Cause Analysis . . . 88

Table 12. Comparison of Traditional and AI-Driven Root Cause Analysis Approaches 91

Table 13. Constructs, Variables, and Operationalization in HELIOS Evaluation . . . 113

Table 14. Comparative Analysis 120

Table 15. Technical outcomes to business outcomes after deployment 122

Table 16. Decision rights for a HELIOS deployment 128

Table 17. Feature-Flag Matrix for the Eight Confirmatory Variants 148

Table 18. Research Question to Hypothesis to Architecture to Variant to Metric to Statistical-Test Mapping 168

Table 19. Sample-Size Sensitivity for Cohen's h under One-Sided $\alpha = 0.00625$, $\rho = 0.5$, 80% Power 194

Table 20. Panel A — CHASE-to-AIOpsLab Adjustment Components 195

Table 21. Panel B — Operational-Significance Threshold Sensitivity 196

Table 22. Panel C — Per-Hypothesis Power Audit (Ablation Family, $\alpha = 0.00625$, $\rho = 0.5$, $n = 174$) 197

Table 23. Panel D — Per-Hypothesis Power Audit, B-Family under McNemar's Exact ($\alpha = 0.00625$, 80% power) 199

List of Figures

Figure 1. HELIOS Architecture for One Incident 140

1 Introduction

1.1 Background and Context

Micro-service-based distributed system has more than one services which interact with each other to achieve certain business goals. Such system can fail in production due to service fault or defect or runtime performance degradation due to heavy load. These faults can propagate across multiple dependent services. Resolving these defect in running system is often complex due to manual intervention, static diagnostic approaches and dynamic nature of containerized and cloud environment (Fu et al., 2025). Continuous integration and rapid deployment practices makes the environment more dynamic and defect patterns can shift within hours, producing fault signatures that differ from those observed in earlier releases. Identifying the root cause in these dynamic environment has become a moving target.

Software architecture has transitioned from monolithic applications to micro-services architecture. A monolith is a single, unified deployable unit, whereas a micro-service is a collection of small, independent services. Software engineers face scalability and deployment challenges in monolithic applications. Micro-services offer flexibility and modular services that perform specific functions. This architecture has helped reduce the dependency on single services. In addition, it is scalable when coupled with containerized or cloud systems. With multiple services, problems in a part of the service will only impact that functionality without affecting the rest of the application (Nobre et al., 2023b). Advanced deployment tools, such as Docker and Kubernetes, provide flexibility in the deployment of micro-services. Docker is a lightweight runtime container environment, whereas Kubernetes is a container orchestration tool. This has helped major enterprises such as Amazon, Netflix, and Uber to have a decompose architecture with elasticity (Jonnaikode, 2025).

Architectural separation has helped teams choose their technology stack, which creates heterogeneity across products. However, this also leads to a lack of standardization across teams and incurs operational challenges (Nobre et al., 2023b).

Service complexities have increased owing to the autonomous nature and flexibility of choosing technologies. Services use multiple API's and schema structures, resulting in service interconnections and frequent deployments. Every service attempts to implement reliability, which may result in fault-tolerant implementation. This may obscure the issue until it results in significant failure (Z. Chen et al., 2020). Defects in micro-services environment is often different from monolith environment. Micro-services has chained service interaction which is often dynamic. Failure in one service may cascade across API chain or message queue. Origin of the defect may be located several hopes away from point of observable degradation (Noor, 2021). Workload pattern can uncover hidden defects in services. Services that function properly under normal workloads may have several hidden issues under heavy workloads. The monitoring of these services is based on a fixed threshold. Sometimes, it misses the dynamic behavior of the application. Machine Learning models can help predict unusual patterns in applications, but models trained under certain workload data may skip actual issues caused by altogether different application usage patterns (Shivakumar, 2025).

Monitoring should cover all touch points in a multi-cloud environment with a service mesh infrastructure. Meeting the service-level agreement in such a heterogeneous environment requires proactive anomaly detection capability. Generative AI models can assist here. These models work on multi-modal data, such as logs, traces, and metrics, and quantify the distributed system state as a unified context for root cause analysis of defects (T. Wang & Qi, 2024). A huge volume of multi-

modal data is generated in cloud observability. Establishing a causal link with the input request helps identify the actionable insights required for root cause analysis (Gan et al., 2023).

Micro-services provide modular and adaptable architectures. It speeds up the deployment process without affecting the majority of applications. It also offers scalability and helps boost system performance. However, multi-service interactions create a complexity layer. Identifying the root cause of complex service interactions is difficult. The data required to identify the root cause are extensive and cover logs, traces, and metrics. Statistical tools or artificial intelligence can help to harvest data and extract unusual patterns. These patterns can help build cause-and-effect relationships with defects and system environments.

1.2 Problem Motivation

1.2.1 Nature of Failures in Micro-service Environment

Due to inherent complexity, different failures can be observed in micro-services system. This section will discuss different types of failures.

Cascading Failures. Services can be interdependent on each other. For example Service A can be dependent on B and B in turn depend on C. User may be interacting with service A but success of service A depends on success of B and C as well. This propagative characteristics can propagate the local faults through the entire system. Failure can be observed in service A but root cause can be at B or C. The boundary between the root cause and symptoms becomes blurred. Anomalies can quickly propagate across service invocation chain (J. Chen et al., 2024; Jonnaikode, 2025; Sarda et al., 2023; T. Wang & Qi, 2024; S. Xie et al., 2026). This has a cascading effect on

latency, service unavailability, and Quality Of Service (QoS) (Lee et al., 2023a; D. Liu et al., 2021; Sarda et al., 2023; Shetty et al., 2024; Somashekar et al., 2024; S. Zhang et al., 2023).

It creates a back-pressure effect,"where poor performance can spread across the interaction chain (Gan et al., 2022a; Gan et al., 2023). These faults are destructive, create dependency complexity, and propagate quickly (T. Wang et al., 2024; T. Wang & Qi, 2024). Failure in one micro-service delays or fails all other micro-services in the invocation chain (Lee et al., 2023a). For example, Failure in Shipping service can propagate to other services and in turn affect front-end application service. Anomalies can also have an indirect impact, where resource congestion caused by one service located on a certain host can affect all independent services on that host machine (Ma et al., 2020; Wang et al., 2021). Multiple dependent bottlenecks due to the same failure problem can create failures. E.g. Resource contention on shared VM (Somashekar et al., 2024). The propagative pattern is the primary driver of RCA complexity. Engineers must trace back convoluted or misleading failure paths and correlate the data to identify the origin of the failure (T. Wang & Qi, 2024; W. Zhang et al., 2024).

Diversity and Granularity. Failures can be due to kernel faults, software bugs, network failures, resource constraints, over-utilization, performance issues, configuration issues, or non-traditional failures related to a distributed environment (Sarda et al., 2023). These failures cover multiple application layers and are highly diverse in nature. Anomalies are deviations from the expected behavior and are often caused by different types of failures. These failures are systematically categorized at the granular level in the micro-service environment. For example, application, service, instance, pod, container, and node (Fu et al., 2025; Han et al., 2024; Zhang et al., 2025).

These failures are shown in the monitoring data as performance anomalies, such as saturation, I/O bottleneck, high CPU utilization, or explicit status anomalies, such as HTTP 4XX or 5XX errors (Aktaş & Kilinc, 2024; X. Ji et al., 2025; Tang et al., 2025). Common failure type includes CPU exhaustion, memory leak/hog, network delay/jam/loss/partitions, Java Virtual Machine (JVM) related problems or API logic faults (X. Ji et al., 2025; Pei et al., 2025; Pham et al., 2024c; W. Zhang et al., 2024; Y. Zhu et al., 2024). Micro-service-specific failures include configuration faults, such as JVM or Docker resource limits, multi-instant faults due to a lack of coordination or inconsistent state of the same service, and asynchronous execution faults due to the incorrect order of micro-service invocations (Zhou et al., 2019). Additionally, external activities, such as database migration, can also cause failures (S. Zhang et al., 2026). RCA should be capable to handle heterogeneous failure, identify the origin and explain the failure at granular level (Tao et al., 2024; S. Xie et al., 2026; S. Zhang et al., 2026).

Dynamic Systems and Fragmented Data. Micro-services systems are dynamic. Invocation paths or service interaction can change quickly. For example, Service A dependent on Service B can change the dependencies to Service C. Each service has its own monitoring, logging and tracking system in place. Cluster of multiple services generate huge logs, traces and metric data. Cloud infrastructure has made deployment and scalability easier than before. Effective RCA in such environment is influenced by these architectural challenges which can lead to delays in diagnosis, alerts overburden, and low granular results. Due to continuous delivery practices, such as frequent updates, the environment is dynamic and volatile (Aktaş & Kilinc, 2024; J. Chen et al., 2024; Heinrich et al., 2017; Ma et al., 2020; L. Wang et al., 2023; Yu et al., 2021; Zhou et al., 2019). Identifying normal behavior in such environments is difficult and can create false alarms (Z. Chen

et al., 2020; Heinrich et al., 2017; Nobre et al., 2023b; Wang et al., 2021; Xing et al., 2025). Architecture of multi-tier environment is decomposed into multiple layers such as presentation, API, service, messaging and data tier. Fault originated in one tier can propagate via chain and create a flood of homogeneous incidents. Observability of the services becomes fragmented when services are collecting logs, traces and metrics in silos. Relying on single-modal data can create blind spots where some failure patterns can be skipped (Huang et al., 2024; Tao et al., 2024; S. Xie et al., 2026; Yu et al., 2023; S. Zhang et al., 2023). For example, failure may not be shown in the metric data but can be seen in the log data (C. Zhao et al., 2023). Pain points observed in RCA includes,

Dynamic Dependency. The dependency graph changes over time (L. Wang et al., 2023).

Telemetry Noise. Monitoring creates a huge volume of data, and sometimes real failures are buried in this data (Yu et al., 2023).

Coarse-grained result. RCA localization requires a Service Reliability Engineer (SRE) to spend additional time troubleshooting the service to identify the function-level or metric-level root cause (Wang et al., 2024; Yu et al., 2023; Y. Zhu et al., 2024).

Metric data integrity. SRE may collect data selectively, resulting in partial or incomplete diagnosis (X. Jiang et al., 2023).

To overcome these pain points, researchers have used AIOps and machine learning techniques with the use of multi-modal data (logs, traces, metric,s etc.) to identify anomalous patterns (Hu et al., 2025; Lee et al., 2023a; S. Zhang et al., 2023). This has helped in fine-grained root cause

analysis with minimal downtime (Wang et al., 2024; S. Xie et al., 2026; Yu et al., 2023; Y. Zhu et al., 2024).

1.3 Significance of Root Cause Analysis

1.3.1 Mean-Time-To-Resolution (MTTR) and Economic Loss

RCA can helps in reducing mean time to resolution which translate into substantial economic saving and maintaining high Service Level Objections (SLO). Production incidents disrupt the normal operation of the system, which can lead to unsatisfied end users and economic losses. RCA helps minimize the gap between system incidents and recovery. This helps minimize the MTTR. Research shows that IT downtime costs $26.5 billion annually in lost revenue (C. Chen et al., 2023). In addition, it costs reputational damage, and immediate incident resolution can help reduce damage. For example, the RCA system and TraceDiag integrated into the Microsoft M365 Exchange system helped reduce the RCA time by 96.5% (Ding et al., 2023). Another RCA system, KPIRoot+, deployed in a cloud environment, has shown an improvement in root cause localization from 30 min to 5 min (W. Gu et al., 2025). CloudRCA deployed in Alibaba helped engineers reduce the time spent on failure by more than 20% (Y. Zhang et al., 2021). This shows that RCA can help minimize system failure and reduce downtime. This directly contributes to reducing reputation damage and financial losses caused by failure (C. Chen et al., 2023).

1.3.2 Explainability and Developer Trust

RCA should provide interpretation and explainable results for site reliability engineers (SRE) or on-call engineers (OCE) (Y. Chen et al., 2023). The trust of engineers in the RCA system's results will enable them to take the necessary mitigation actions. Machine learning or deep learn-

ing models are important for automated RCA, but the outputs produced by these models should be explainable. Explainability is important for justifying the root cause of failure and identifying scenarios that lead to performance degradation (Harutyunyan et al., 2023). Engineers require clear and transparent reasoning to trust and use the results of automated RCA. The RCA results are not interpretable, and SRE are often skeptical of accepting the RCA output for fear of losing repair time or worsening the fault. The system leverages explainability in different ways. RCACopilot and SynergyRCA use large language models to generate human-readable explanations of root causes with clear action steps (Y. Chen et al., 2023; Shan et al., 2025; Xiang et al., 2025). Groot makes casual relationship understandable. It provide the visual representation that shows how different system component are causally connected with root cause (H. Wang et al., 2021). The SAGE system has gone beyond the diagnosis. It can help in operational decision making. It generates actionable decisions, such as identifying resources with high utilization (Gan et al., 2021, 2022b). Rule induction systems, such as Chain-of-Event provide systems behavior as related events. These system provide parameters that SRE can tune as per their domain knowledge. It provides the likelihood of one event causing another (Yao et al., 2024). This helps in informed judgment rather than binary decisions. Interpretability and explainability are necessary features of an RCA system. This builds trust in the AIOps system. It helps reduce manual diagnosis effort and provides crucial insights that may be left by manual interpretation, and creates knowledge for further system enhancement to avoid future failure (Harutyunyan et al., 2023; Soldani & Brogi, 2023).

1.3.3 Industrial Adoption

The success of RCA systems can be measured in terms of their operational efficiency and adoption in production environments, such as cloud services or Kubernetes clusters. RCA system

must scale effectively with size and complexity of micro-service environment. It should process large volume of logs, metrics and traces (Fu et al., 2025; W. Gu et al., 2025; Pham et al., 2024c). So, much of latest research is focused on demonstrating if approach can work in production environment where traditional or statistical methods fails to perform (T. Wang et al., 2024; T. Wang & Qi, 2024). This shift is reflected in several deployment like Microsoft's RCA copilot and eBay's LatentScope (Y. Chen et al., 2023; Z. Xie et al., 2024). TraceDiag has been integrated into the Microsoft M365 Exchange system (Ding et al., 2023). BALANCE is deployed in production to handle real-world RCA problems (C. Chen et al., 2023). SynergyRCA demonstrated its dependability in two real-world Kubernetes cluster scenarios. It has achieved an average diagnostic time of two minutes (Xiang et al., 2025). KylinRCA is designed for practical deployment in production and is packaged as a containerized application for rapid cloud deployment using Helm charts (Jian, n.d.). This indicates that the adoption of AI tools and technology in root cause analysis is not limited to theoretical studies but is also becoming part of the automated incident management process.

1.4 Problem Statement

The micro-service environment is dynamic, scalable, and heterogeneous. The organization has adopted the architecture and is using it in a production environment. This architecture increases the cost of incident resolutions owing to its complex structure and large volume of data. The root cause analysis of incidents is mainly manual or statistical rule-driven. Site reliability engineers must spend time on manual investigations. Micro-service environments can create cascading failures, their service dependency graphs can change quickly, and observability solutions are mostly siloed or service-specific. These conditions increase the mean time to detect (MTTD) and mean time to resolve (MTTR).

While recent developments in AI and machine learning have helped in building better solutions, they suffer from explainability, poor scalability, and weak multi-modal integration. Consequently, SRE or operational teams lack trust in these solutions. This study addresses the problems of AI-driven RCA approaches by studying Gen AI-based solutions. The main purpose of this study is to reduce the cost of defect management and improve operational efficiency by providing better explainability, scalability, and trust.

1.4.1 Breakdown of main problem

Data fragmentation and fusion. Observability data are specific to services or individual applications. Its volume and velocity are high. Combining and referring to this data for the entire application requires high cost and computation.

Causal Inference Problem. Dues to dynamic dependencies, identifying true root cause is challenging.

Explainability. AI-based RCA are black boxes by nature, and operations teams are often reluctant to trust their outputs.

When the volume of the data is high, the scaling RCA solution is often cost-sensitive and can create huge costs and processing power.

1.5 Research Aim

The primary aim of this research is to evaluate AI-driven approaches for root cause analysis in a production environment. These approaches are evaluated based on cost, applicability,

efficiency, and adoptability in a micro-service environment. The solution should be able to scale large-scale micro-services.

1.6 Research Objective

To define an advanced scalable approach for root cause analysis that will help reduce the mean time to detect and mean time to resolve the defect. This approach aims to integrate multi-modal data, such as traces, logs, and metrics generated by micro-services, and establish a causal link between the cause and effect. Secondary objectives are,

- To investigate how multi-modal data help in improving the accuracy of root cause analysis in micro-service environment
- Establish dynamic causal discovery and build the propagation path of fault diagnosis.
- Analyze the LLM models for root cause explanations and long-term fix.

1.7 Research Questions

- **RQ 1:** How can generative AI support root cause analysis, and what impact does it have on cost, mean time to resolve (MTTR) and effectiveness of fix?
- **RQ 2:** What are the operational challenges in adopting GenAI based RCA solutions?
- **RQ 3:** How can the effectiveness of the GenAI solution be assessed in comparison with the statistical and traditional RCA approaches?
- **RQ 4:** How does the explainability of GenAI-based solutions influence the interpretation of the root cause output in the RCA workflow?

1.8 Testable Hypotheses

- **H1:** If a hybrid AI RCA system integrating statistical, graph-based, and generative reasoning is used in place of single-modality AI or statistical baselines, then Mean-Time-To-Resolution (MTTR) will be significantly reduced.
- **H2:** If a hybrid multi-pipeline architecture (statistical, graph, and LLM reasoning operating as peers) is used in place of any single reasoning pipeline, then root-cause diagnostic accuracy (HR@3) will improve.
- **H3a:** If the RCA system produces explainable output (e.g., Chain-of-Evidence narratives) in place of opaque/black-box output, then operations-team trust in the system's diagnoses will increase.
- **H3b:** If operations-team trust in the RCA system's output increases, then stated intent to accept that output, inside the exploratory user study, will increase. Whether a firm adopts the system is specified as a staged path in Chapter 3 and is not estimated by this test.
- **H4a:** If a hybrid AI RCA approach is used in place of single-modality alternatives, then Cost per Resolution (CpR) will be lower (Confirmatory — B-H7).
- **H4b:** If estate size increases from 100 to 5,000 services in the preregistered scaling sub-study, then Plane-2 latency and CpR will remain within the design-committed SLO bounds (Separate preregistered experiment, not part of the main 16k-run ablation).

1.9 Conceptual Framework

Framework consist of below building blocks,

- **Multi-modal observability data:** Metrics, logs, and trace data, along with other data generated by micro-services, help identify the state of the application.
- **Causal Inference:** Transform data helps in identifying the actionable insight using AI based model
- **Generative AI:** Generative AI models can help improve the explainability of solutions with clear action steps.

Together, these help build a better root cause analysis system in micro-services.

1.10 Limitation

The main limitation of this study is the publicly available dataset used to test the GenAI solution and compare it with traditional approaches.

- The performance of AI models may degrade owing to concept drift.
- Organization culture, and management may influence adoption outcome.
- Cost of training model could be high at initial stages.

1.11 Study Scope

This study focuses on a production-grade micro-service environment. Deployment is preferably performed using a containerized environment, such as Kubernetes. Scope will be as below,

- GenAI assisted approach
- Operational metric such as MTTR, MTTD and Cost
- Defect management and incidence response processes

1.12 Assumptions

- Incident data is available
- Practitioner posses baseline operational knowledge

1.13 Definition of Key Terms

Table 1
Definition of Key Terms

Concepts	Definition
Root Cause Analysis (RCA)	Method of identifying the underlying origin of a fault or defect.
Mean-Time-To-Resolution (MTTR)	Average time taken to resolve production issues.
Mean-Time-To-Detect (MTTD)	Average time taken to identify a production issue.
Cascading Failure / Chain Failure	A typical pattern in microservices where failures in one service propagate through dependency chains, potentially obscuring the original source of the fault.
Metrics	Structured quantitative time-series data related to CPU usage, resource utilization, latency, and other system performance indicators.

Concepts	Definition
Logs	Semi-structured or unstructured data generated by services, including events, runtime information, and error data.
Traces	Hierarchical data that tracks the flow of a user request across services.
Anomaly Detection (AD)	The first stage of the RCA lifecycle, involving the identification of deviations from expected patterns or behavior.
Root Cause Localization (RCL)	The process of identifying the location or source of a defect or failure.
Causal Inference / Causal Link	A statistical approach used to identify cause-and-effect relationships.
Generative AI (GenAI) / Large Language Models (LLMs)	Advanced artificial intelligence models used to generate human-readable explanations, reasoning, and remediation recommendations.
Hallucination	A situation in which a model generates factually incorrect information or a reasoning path, potentially resulting in incorrect explanations or remediation steps.

Concepts	Definition
Concept Drift	The degradation of model accuracy over time due to changes in system behavior or underlying data patterns.
Alert Fatigue	A condition caused by a high volume of alerts, particularly false positives, which can reduce the effectiveness of operators.

2 Literature Review

When production system deviates from its intended usage patterns or some unusual activity is detected during monitoring then system reliability, stability and security is under threat. These deviations in system behavior is called anomalies (Ahmad et al., 2025; Hagemann & Katsarou, 2020a; Sarda et al., 2023). These anomalies could be benign or malignant and details analysis has to be performed to identify and take precautionary measures. So, production support team which consist of Service Reliability Engineer (SRE) or On Call Engineers (OCE) conduct details root cause analysis of these anomalies. Typically, every anomaly is cause-and-effect relationship and can be grouped in different categories. These categorizations help engineers to perform the remediation steps as per the pattern. Anomalies can be grouped based on symptoms such as Performance, reliability or traffic anomalies. Another way to classify these anomalies are based on level of details or in which part of application layer does it exist. For example, Application level, service level or operational anomalies. Another way to classify is based on root cause of anomalies like resource system failures, network communication failure, application faults or security breach. Below table enlist anomalies and its corresponding symptoms.

Table 2
Anomalies Classification and Symptoms

Anomalies	Symptoms
Performance	Increased response time, latency, CPU leaks, and I/O bottlenecks (Fernando et al., 2025; D. Liu et al., 2021; Noor, n.d.; Sarda et al., 2023; Tien et al., 2019; Xing et al., 2025; Yu et al., 2021).
Reliability	Failure to perform a particular service operation (D. Liu et al., 2021; Sarda et al., 2023).
Traffic	An increase or decrease in service requests based on geography, time of day, or other contextual factors (D. Liu et al., 2021).
Application	Failure symptoms observed across the application as a whole (Soldani & Brogi, 2023). These can be further classified into performance, reliability, security, or other categories.
Service	Failure observed in a specific service or part of the services within the overall application (Soldani & Brogi, 2023).

Anomalies	Symptoms
Operational	Slow response time, increased error rates, and other operational symptoms (Sarda et al., 2023).
Resource/System Failure	CPU, memory, disk I/O, container, or node-related failures (Pei et al., 2025; Pham et al., 2024a; Rouf et al., 2024; Sarda et al., 2023; Zhou et al., 2019; Z. Zhu et al., 2024).
Network Communication	Network latency, delays, packet-related issues, connectivity problems, and similar failures (Fu et al., 2025; H. Huang et al., 2024; Lee et al., 2023a; Pei et al., 2025; Pham et al., 2024a; Z. Zhu et al., 2024).
Application Fault	Faults caused by complex microservice interactions, configuration issues, or software defects (C. Zhang, Peng, Sha, et al., 2022; Zhou et al., 2019).
Security	Security threats such as Denial-of-Service (DoS) or Distributed Denial-of-Service (DDoS) attacks (Fernando et al., 2025; Nobre et al., 2023b).

Anomalies can be introduced throughout the software development life cycle (SDLC). Some of the anomalies can be introduced in early stages of SDLC but can be found in later stages which highlights importance of testing and monitoring.

Table 3
Anomaly Origin in the Software Development Life Cycle (SDLC)

Fault Origin in SDLC	Anomalies
Design and Development	Runtime issues found in an application are often caused by incorrect implementation or flaws in the system design (Gulenko, 2020).
Deployment and Configuration	Incorrect configuration or deployment setups can degrade application performance, create security vulnerabilities, and cause service outages (Z. Chen et al., 2020; Sarda et al., 2023; Shetty, Bansal, Kumar, Rao, Nagappan, & Zimmermann, 2021).
Testing	Anomalies are typically injected during this stage to verify the stability and reliability of an application. Effective testing strategies can also uncover runtime issues introduced during the design and development phases (Sarda et al., 2023; Shivakumar, n.d.).

Fault Origin in SDLC	Anomalies
Operation	Unplanned downtime, performance issues, and operational failures can negatively affect application stability, reliability, and security (Z. Chen et al., 2020; Gulenko, 2020; Sun, Zheng, et al., 2025).

2.1 Historical evolution of Root Cause Analysis (RCA)

When a failure occurs in a production system, the team performs a structured review of the system to identify the cause of the failure. The goal is to perform the root cause analysis of the issue so that failures can be avoided in the future. To do this, SRE manually gathers information from logs, traces, or metrics and tries to understand how the incident started and its impact on the system over the period. They then apply structured analysis and determine the actual root cause of the issue (Y. Chen et al., 2023). Common techniques used in this process include “Five Why,” “Fishbone” or barrier analysis method. These methods are cost-effective and widely used; however, they require more time to identify the issue, need expert human input, and are error-prone in complex micro-service environments (S. Jin & Bei, n.d.).

Every production environment has a monitoring system in place that uses different rules to trigger alarms. For example, raise a alert if heap memory usage or CPU usage cross certain boundary These rules helps in identifying the problem which occurs repeatedly (Ding et al., 2023; Gan et al., 2023; Panwar et al., 2025; T. Wang & Qi, 2024). For example, MonitorRank uses a

heuristic algorithm that depends on a predefined view of the system's network topology to identify problems (Ma et al., 2020). This approach works well for simple issues; however, in modern micro-service environments, maintaining a large number of rules becomes impractical. Correlation-based methods also struggle with cause-and-effect relationships, which may lead to incorrect sources of failure. As a result, these approaches can be rigid and lack robustness, which may produce inaccurate results.

Anomaly Detection (AD) has evolved from statistical rule based system to advance machine learning or deep learning models. These models learn normal patterns and deviations. AD is the first step in the RCA. This is performed when the system encounters unusual patterns or behaviors (Jha et al., 2025). AD has evolved from using statistical checks, such as N-sigma (Pham et al., 2024a; C. Zhang, Peng, Zhou, et al., 2022) or clustering techniques such as BIRCH (Pham et al., 2024a) to modern deep learning models such as variational autoencoders (VAE) (Boesel et al., 2023; Z. He et al., 2023; Jha et al., 2025; Z. Zhu et al., 2024), LSTM or Transformers (Vervaet, 2023; S. Zhang et al., 2026). These models are trained offline on fault-free data to learn the normal patterns of the system (Z. Zhu et al., 2024). These models must refer to high-dimensional and nonlinear data and adapt to dynamic changes in micro-service environments. Variational autoencoders are used to construct anomalous patterns that reveal deviations; however, these techniques lack precision and accuracy in detecting anomalies.

Single model data approaches can be biased because they capture only part of the system state and shows the system behavior from one perspective. Multi-modal data gives complete system view and root cause analysis performed using this data has better fault detection rate compare to single modal data (Hwang et al., 2021; Yu et al., 2023). Frameworks such as CloudRCA, Eadro,

and MRCA integrate logs, traces, and metric data. This helps them to address broader range of faults (Lee et al., 2023a; Wang et al., 2024; Y. Zhang et al., 2021).

Hybrid RCA combines multiple data types and analytical techniques to identify root causes. It achieves this by building cross-layer reasoning (Hwang et al., 2021). For example, a CPU spike in the infrastructure layer increases the latency of the upstream system. Each data type brings in different perspective which helps in reducing missed anomalies and improve accuracy (Z. Xie et al., 2024).

2.2 Building Blocks of Root Cause Analysis (RCA)

Traditional root cause analysis involves anomaly detection, identifying the exact root cause, and fixing the root cause. These steps were conducted manually by an expert engineer. The latest developments in AI and DevOps have helped the RCA system analyze heterogeneous data and provide more plausible solutions using an automated pipeline (Cheng et al., 2023; Sarda et al., 2023; T. Wang & Qi, 2024). The main goal of a modern RCA system is to reduce the mean time to resolve (MTTR) (Sarda et al., 2023). The subsequent sections discuss the different stages of these pipelines.

2.2.1 Anomaly Detection (AD)

First, collect the data from from different sources, such as metrics, logs, or traces. Data can be structured, such as metrics, or semi-structured or unstructured, such as traces or logs. It perform normalization and aggregation steps to clean the data. The main challenges in this step are handling the huge volume of data along with its high velocity (Han et al., 2024; H. He et al., 2024; Wang et al., 2024; Zheng et al., 2024).

Preprocessed data are used by different machine learning or deep learning models to identify anomalous patterns or deviations (Pham et al., 2024a; Y. Zhang et al., 2021). Different sampling techniques are often used by these models during the training phase to identify deviations effectively (Ding et al., 2023; H. Huang et al., 2024). Models such as LSTM, VAE, and CNN have been used to detect anomalies (Jonnaikode, 2025; Pham et al., 2024b; Wang et al., 2024; Z. Zhu et al., 2024).

2.2.2 Root Cause Localization and Causal Inference (RCL/CI)

This is the core brain of the RCA system. The output of the AD phase is fed into these steps, which further preprocess and filter the data. The filtered data were used to detect cause-and-effect relationships, which helped to identify the root cause of the issue. Let us look at the process described below.

Data Preparation and Filtering. The system attempts to match alerts from the monitoring system with existing knowledge. System component matching alert are called Incident handlers. These handlers collect data specific to incidents from multiple sources. This information can be extensive, and different strategies, such as graph pruning and reinforcement learning (RL), can be used to isolate unwanted information from relevant information (Y. Chen et al., 2023). TraceDiag uses RL to reduce redundant components from the service dependency graph (Ding et al., 2023).

Causal Modeling and Root Cause Inference. Causal Modeling and Root Cause Inference. Data from the previous steps were used to create a graph of service interaction, which can form the basis for a cause-and-effect chain. Graphs are scanned to identify unusual patterns in different variables that cannot be explained by parents. This process is known as intervention recognition

(Hardt et al., 2024; Pham et al., 2024b; Z. Xie et al., 2024; S. Zhang et al., 2026). It uses different causal discovery algorithms, such as PC, Granger causality, and DAG-GNN (Pham et al., 2024d; T. Wang & Qi, 2024; Wang et al., 2024). KyLinRCA uses a cross-level modeling technique that connects the resource, container, and business layers (Jian, n.d.).

In the RCA using a graph-based approach, the services are treated as nodes and the communication link between the two services as edges. The system graph is modeled using this knowledge. Then, algorithms such as PageRank, Random Walk and Graph Neural Network traverse this system graph and rank the components that are most likely to cause failure based on dependencies (Pham et al., 2024b; Wang et al., 2024; Y. Zhu et al., 2024). Although they can identify the root cause, these approaches can be slow because of the unclear stopping point. The reinforcement learning method applied in TraceDiag and MRCA handles this by gathering enough evidence before stopping the analysis (Ding et al., 2023; Wang et al., 2024).

2.2.3 Interpretation, Explanation and Human Feedback

Result of root cause analysis should be easy to understand and engineer should be able to perform clear actionable steps. For example RCA Copilot and LasRCA group the incident into categories and provide clear explanation of those categories (Y. Chen et al., 2023; Shan et al., 2025). Large Language Model (LLM) can integrate the troubleshooting document and suggest actionable steps to resolve the issue (Han et al., 2024). In addition, engineer can interact with LLM models to understand the problem in much better way and improve accuracy (Cheng et al., 2023; L. Wang et al., 2023; Y. Zhang et al., 2021). Methods such as the Five Why can be used by SRE while interacting with LLM to extract objective evidence over subjective judgment. It helps in fixing common attribution errors and provides more accurate postmortem analysis (S. Jin & Bei, n.d.).

Modern micro-service-based systems generate huge amounts of logs, traces, and metrics per service. These data are unstructured, and the software does not understand these raw data. Deep learning can help in understanding raw data by converting unstructured data into a structured format. This is a type of teaching that helps computers read incidents. SoftNER can scan the entire logs and identify which service failed, what type of error occurred and shows the path of method execution. It uses an advanced neural network algorithm and identifies patterns from temporal data. This information is stored in a knowledge graph (T. Wang & Qi, 2024). DeepLog analyzes the log sequence to learn the normal patterns. When unusual activity occurs, DeepLog detects it and highlights which system has caused the issue (Girish & Rao, 2023; X. Ji et al., 2025; C. Wang et al., 2024; C. Zhang, Peng, Sha, et al., 2022). Deep learning methods are effective for identifying temporal patterns. They can detect certain signals that may be missed by simple heuristic rules. These models can produce clean and organized diagnostic evidence that can be used in postmortem reports and future learning (Z. Chen et al., 2022; Girish & Rao, 2023).

Once the model has identified the root cause of the defect, the next logical step is to use this information for future learning. Machine learning models such as KNN, Random Forest, Neural Network, and logistic regression can learn the classified patterns and predict the next classification based on the input. It can categorize the most affected services and show metrics that matter most (Fu et al., 2025; Nobre et al., 2023a; Zhou et al., 2019). For example, the BARO framework uses multivariate Bayesian online change point detection (BOCPD) and a robust scorer to analyze metric data to identify the main issues in micro-services (Pham et al., 2024b).

2.2.4 Auto-Remediation (AR) and Mitigation

An LLM can help generate remediation steps based on the root cause identified in the previous step (Shan et al., 2025). The LLM to historical incident data and the entire system context as an input while suggesting remediation steps. Deep reinforcement learning (DRL) and meta-learning can independently correct errors that occur during the learning process by their own (C. Ji & Luo, 2025a; H. Qiu et al., 2020). This helps in recovering and adapting quickly to changing conditions. These models continuously interact with the system and attempt to identify the best methods. This helps build a self-healing system that can detect issues, identify the root cause, generate a fix, and apply the fix without human intervention. Kepler and Amazon DevGuru use an RL-based approach to predict and apply corrective actions (C. Ji & Luo, 2025a). Reinforcement learning (RL) agents can restart pods, scale services, and change settings. They get rewards from metric services to learn the best configuration (H. Qiu et al., 2020). The FIRM framework uses a deep deterministic policy gradient algorithm to manage cloud resources better than traditional methods (H. Qiu et al., 2020). Meta-learning helps quickly adjust to unexpected failures (Chang et al., 2024).

ML Based resource management. Machine Learning models can predict resource adjustments rather than reacting to failure. ML models can predict system load changes and scale resources accordingly. The Narya system uses ML to decide between VM migration or reboot (Cheng et al., 2023), learning workload patterns instead of fixed thresholds.

Self Healing Systems. These systems are built using agentic AI. Multiple agents work independently and collaboratively to solve complex operational problems (Esposito et al., 2025). This

helps automate diagnosis, planning, and execution and reduces human burden (An et al., 2024; Pei et al., 2025; Shi et al., 2025) . For example, Nissist uses multiple LLM agents that understand the intent, retrieve the actions from the knowledge base, and generate a semi-automated remediation plan (An et al., 2024). FlowXpert enhances LLM agents with reinforcement learning to automate the troubleshooting workflow (Shi et al., 2025). These systems follow the reasoning and action (ReAct) pattern. This renders the troubleshooting process transparent (Cornacchia et al., 2025; Pei et al., 2025).

Automated Response. Once the problem and root cause are identified, the system can produce the output with the required code changes and configuration suggestions to optimize the resources or initiate recovery workflows. LLM help generate code and recover scripts or configuration files from the RCA output (Sarda et al., 2023). Prompt engineering can help generate a complete system code without human intervention. SynergyRCA generates a remediation command based on the identified root cause (Xiang et al., 2025). This helps in faster resolution of issues and reduces downtime and operational burden (Ding et al., 2023; Sarda et al., 2023).

Remediation Effectiveness. In this approach, the LLM is combined with a knowledge graph to extract the root cause of the failure. Learning happens from past events, current condition and operational data. To fix problems, a GenAI agent uses the Five Why method to ask questions about failures. Another agent examines reasons by comparing them to historical incidents (S. Jin & Bei, n.d.). This approach uses code and LLMs to identify underlying issues. It integrates cognitive processes with actions, resulting in reduced incidents over time (S. Jin & Bei, n.d.).

Knowledge Base (KB). Continuous improvement happens when the system learns from new data. It updates its knowledge and models. These models use the MAPE-K cycle: Monitor, Analyze, Plan, Execute, and Knowledge. The system watches, studies data, makes decisions, and saves information (Z. Chen et al., 2022; Esposito et al., 2025; Hrusto et al., 2022; Islam et al., 2025). The AIOPSLAB framework has validated agents by using fault injection and chaos engineering to simulate real-world failures (Y. Chen et al., 2025; Sarda et al., 2023). This helps identify whether agents can diagnose the problem with appropriate remediation strategies (Y. Chen et al., 2025).

Meta-Learning and Adaptation. Retraining improves performance, but it is often too slow when the system dynamically changes and evolves over time. Meta-learning helps model to adapt quickly to new failure patterns. Meta-learning techniques, such as Model-Agnostic Meta-Learning (MAML), allow models to generalize better (Duan, Bao, et al., 2024). It performs few-shot or zero-shot learning. This helps them learn new issues with little data. The performance of meta-learning is far better in environments where data are sparse or evolving. It helps in fast adaptation, sustains the diagnostic accuracy despite data drift, and eliminates issues in the long term (Duan, Bao, et al., 2024; Wang et al., 2025).

2.2.5 Three Pillar of Observability

It is clear from RCA literature that no single telemetry stream is enough to reconstruct the state of distributed system during the faults. This is frequently highlighted in literature but its reason is not reviewed to often. Metric, logs and traces capture partial view of the runtime state and success of the RCA system depends on how views of the system can be reconciled from observability into coherent picture (T. Wang & Qi, 2024; S. Zhang et al., 2023).

Treating these three pillars complementary is easy to justify since all of them shows different perspective of the same system. Metrics carry continuous numerical things like CPU usage, RAM usage, throughput and error rate. It helps to answer if something is happening and how badly it is impacting. Logs carry semi-structured text at runtime. It helps to answer and what and why of the system failures. Specifically when stack-traces or exception are emitted at runtime. Traces is only modality which encode the topology and temporal ordering across service boundaries. They help to answer where exact failure is occurring. System diagnosis in practice involves triangulating all three (Aktaş & Kilinc, 2024; Y. Chen et al., 2023; Cheng et al., 2023).

Suppose high traffic e-commerce deployment start returning HTTP 500s, The metric stream shows that latency and error rate is increasing. Log highlight the ConnectionPoolTimeoutException at persistence boundary. None of the signal is diagnostic in isolation since pool timeout can occur anywhere. But when trace data shows the order service stalled the connections for thirty seconds while inventory service respond normally. This give engineer some information to act upon. So, if any pillar is removed then diagnosis can collapse back into ambiguity. This is, I would argue, the structural reasons that empirical work keeps finding multi-modal pipelines outperforming single modality baseline on accuracy, robustness and false alarm rate (Lee et al., 2023a; Wang et al., 2024; Yu et al., 2023).

More recent development is that the tree canonical no longer enough to identify the failures within system. Code repository state, deployment events, configuration history, incident records, and even operator chat are increasingly being treated as legitimate inputs to RCA rather than as out-of-band metadata. The shift is conceptual as much as practical.

Why multi-modal data is necessary?. Three properties of multi-modal data occurs repeatedly across the literature. Multi-modal data improves the coverage because failure mechanism rarely manifest uniformly across modalities. Like slow memory leak is almost invisible in trace data. It improves the fidelity of the results by reducing the false positive rate which is high in single modality detectors (Huang et al., 2023; Wang et al., 2024). In addition it helps in improving the interpretability of the results. When LLM based layer is sitting on top of fused evidences, engineer can read it any time (Sarda et al., 2023; Sun, Luo, et al., 2025).

None of this is free and cost gets less attention that it deserves. Processing multi-modal data have trade-offs which matters materially. The first, multi-modal data fusion is complex. These data differ in structure, in temporal granularity and in cardinality. Representing them into single representation is modeling problem and this can create failure modes which did not exists in any of the input streams (Tao et al., 2024; Zhang et al., 2025).

The second issue is related to economics of velocity and volume. Micro-services produces huge telemetry data at rates that strain ingestion before performing analysis. Cost processing and storing the data outpace the marginal RCA value in return. This crossover is unclear and majority of time deployment specific (Y. Chen et al., 2023; Cheng et al., 2023).

The third is noise and incompleteness. Multi-modal pipeline is as reliable as its weakest input. Missing log levels, sampled-out traces or pre-aggregated metrics can quietly hide the anomalies the pipeline was built to surface (Fu et al., 2025; T. Wang & Qi, 2024).

The fourth and most underrated in the modeling literature is representational bias during training. Models exposed to multi-modal data tend to lean on whichever modality is most discriminative on the training distribution, and that becomes a liability the moment that modality is

degraded or absent in production. Feature-selection discipline and explicit modality balancing are therefore not stylistic refinements but design prerequisites (Tao et al., 2024).

A fifth constraint is labeling cost. Building labeled multi-modal data requires expert effort across all three pillars at once. In fast moving production environment, labeling pipeline rarely keeps up with the system it is meant to characterize (H. Huang et al., 2024; Lee et al., 2023a). This single constraint quietly shapes much of the methodological terrain covered in sections 2.4 and 2.6. It is the reason unsupervised and self-supervised approaches dominate the field. Same reason few-shot and in-context-learning methods have attracted the attention in the field.

So, multi-modal necessary but it does not justify itself. The diagnostic gains are real, and the empirical evidence supports them, but those gains are conditional on solving the fusion, cost, and labeling problems that single-modality RCA simply does not have to face. The rest of this chapter looks at how the field has tried to resolve those tensions, starting with the preprocessing pipelines that prepare the raw signals (section 2.4) and moving on to the modeling families that operate on the representations those pipelines produce (section 2.6).

2.3 Data Preprocessing

Data preprocessing is critical multi-stage requirement for effectively utilizing heterogeneous data. Raw data is noisy, voluminous, unstructured or misleading to get accurate diagnostic results. Data preprocessing often involves data cleaning and filtering, data transformation or feature extraction, normalization and time series alignment.

2.3.1 Data Cleaning and Filtering

Tables shows the raw data to be analyzed or transformed to improve the efficiency and accuracy of downstream models.

Table 4
Data Preprocessing Requirements and Techniques Across Observability Data Modalities

Data Modality	Requirement/Technique	Details and Purpose
General/Logs	**Filtering Irrelevant Data**	Irrelevant information should be removed to improve computational efficiency and reduce storage costs (Tang et al., 2025). This is particularly important in high-concurrency, high-volume production environments. Strategies may include filtering alerts raised fewer than a specified number of times per day and filtering logs based on time windows, location, or other relevant criteria (Chakraborty et al., 2023; Tang et al., 2025).

Data Modality	Requirement/Technique	Details and Purpose
Logs (Text)	**Noise Reduction**	Human-readable elements such as punctuation, newline characters, and unique identifiers including timestamps and IP addresses may be removed. Logs can also be converted to lowercase to reduce ambiguity (Sarika et al., 2023). Irrelevant patterns identified through regular expressions may also be removed (Cheng et al., 2023).
Metrics (KPI)	**Variance Filtering**	Since anomalies are typically reflected through fluctuations in metric values, metrics with low variance may be removed to reduce irrelevant features (Huang et al., 2023).

Data Modality	Requirement/Technique	Details and Purpose
Metrics (KPI)	**Handling Missing Values**	For categorical attributes, missing values can be replaced with a placeholder value (Bansal et al., 2020). For continuous attributes, missing values may be replaced using median values (Bansal et al., 2020) . Other approaches include imputing missing values based on patterns identified within the dataset (Almajed et al., 2025).

Data Modality	Requirement/Technique	Details and Purpose
Traces	**Data Cleaning**	Irrelevant or noisy data may be removed to support valid temporal analysis (Khodabandeh et al., 2025). Fields containing redundant information, such as `traceId`, may also be removed during transformation into tabular formats (Poghosyan et al., 2023).
KPI (Time Series)	**Edge Transition Removal**	Samples captured during the initial onset or transition phase of anomalies may be excluded because they can reflect unstable behavior and introduce noise into the analysis (Jonnaikode, n.d.).

2.3.2 Data Transformation and Feature Extraction

Transformation convert heterogeneous data into structured format. This include feature engineering to replaces important data with more valuable derived data. Below table describe transformation requirement and its purpose.

Table 5
Data Transformation requirement

Data Modality	Requirement/Technique	Details and Purpose
Logs (Unstructured)	**Template Extraction**	Unstructured log messages are converted into structured templates (Duan et al., n.d.; Duan, Bao, et al., 2024; Zheng et al., 2025). Algorithms such as Drain, IPLoM, and Spell are used for this purpose (Cheng et al., 2023; Duan et al., n.d.; Tang et al., 2025). If weak methods are employed, they can negatively affect subsequent anomaly detection (Duan et al., n.d.; Vervaet, 2023).

Data Modality	Requirement/Technique	Details and Purpose
Logs (Structured)	**Feature Extraction**	Structured logs are converted into numerical representations or vectors (Araujo et al., 2023; Sarika et al., 2023).
Logs (Structured)	**TF-IDF Encoding**	TF-IDF encoding generates log-frequency features to highlight distinctive words associated with errors for classification (Han et al., 2024; Sarika et al., 2023; T. Wang & Qi, 2024; Zhang et al., 2025; Zheng et al., 2025).

Data Modality	Requirement/Technique	Details and Purpose
Logs (Structured)	**Word/Sentence Embedding**	Methods such as GloVe, RoBERTa, and Semantic-BERT are used to generate feature vectors that capture the semantic meaning of log messages (Chakraborty et al., 2023; Sarika et al., 2023; C. Zhang, Peng, Sha, et al., 2022; S. Zhang et al., 2024). Log-tailored language models are designed to handle the characteristics of unstructured logs (H. He et al., 2024; Zheng et al., 2024).

Data Modality	Requirement/Technique	Details and Purpose
Logs/Metrics (Joint)	**Event Transformation**	Multimodal data is transformed into unified system events (Aktaş & Kilinc, 2024; T. Wang et al., 2024; Yao et al., 2024; Yu et al., 2023). Metrics can be used to generate events based on threshold rules or anomaly detection algorithms (Yao et al., 2024).
Metrics (Time Series)	**Feature Engineering (Time Domain)**	Statistical count-based, trend-based, and temporal features are extracted from time-series data (T. Huang et al., 2022). For instance, 12 types of time-series features can be derived to improve fault representation capabilities (Jian, n.d.).

Data Modality	Requirement/Technique	Details and Purpose
Metrics (Time Series)	**Feature Extraction (Frequency Domain)**	Frequency-domain characteristics are extracted using the Fast Fourier Transform (FFT) (T. Huang et al., 2022; Jonnaikode, n.d.).
Traces	**Feature Extraction (Trace Level)**	Traces are encoded into structural and status information (H. Huang et al., 2024). Key features include the latency of each service call, invocation relationships, and derived features such as the number of calls over a time interval and average duration (Han et al., 2024; Wang et al., 2025; S. Zhang et al., 2026).
Traces	**Feature Vector Construction**	Raw trace data is processed to obtain span vectors (Z. Zhu et al., 2024).

Data Modality	Requirement/Technique	Details and Purpose
General (Heterogeneous)	**Dimensionality Reduction**	Dimensionality is reduced to improve model performance (Xin et al., 2024). Techniques include Principal Component Analysis (PCA), Independent Component Analysis (ICA), and autoencoders (AE) (Xin et al., 2024) (ganSageUsingUn-supervised2021].
KPI (Time Series)	**Symbolic Aggregate Approximation (SAX)**	Time-series data is downsampled to create a more compact representation (W. Gu et al., 2025). Improved SAX representation (iSAX) incorporates trend variations to improve accuracy (W. Gu et al., 2025).

2.3.3 Normalization and Scaling

Normalization helps to analyze the data from different sources and scales without any bias toward features.

Table 6
Normalization and Scaling Requirement

Data Modality	Requirement/Technique	Details and Purpose
General	**Normalization**	Normalization filters and standardizes data to reduce bias (Gulenko, 2020). It also helps standardize data into a consistent format (Hwang et al., 2021).
Metrics (KPI)	**Min-Max Normalization**	Applied to standardize values into a unified range (Sarda et al., 2023). This technique is commonly used for numerical metrics (J. Chen et al., 2024; Jian, n.d.; Zhou et al., 2019).

Data Modality	Requirement/Technique	Details and Purpose
Metrics (KPI)	**Z-Score Normalization**	Standardizes data to have a mean of zero and a standard deviation of one. It converts metrics with different ranges into a unified scale (Jian, n.d.; Xin et al., 2024) .
Metrics (Raw Data)	**Unit Norm Scaling**	Raw metric data can be standardized by dividing each value by its range.
Metrics (High-Dimensional)	**Scaling for LLMs**	Time-series inputs are scaled to a limited number of significant figures. This reduces the amount of data required within the limited context window of an LLM (Y. Gu et al., 2025).

2.3.4 Time Series Alignment

Temporal alignment is crucial when high frequency metric logs are combined with low frequency traces. It helps in correlating cause-and-effect relationship.

Table 7
Temporal Alignment and Time-Series Processing Techniques Across Observability Data

Data Modality	Requirement/Technique	Details and Purpose
Metrics/Logs/Traces (Cross-Modal)	**Timestamp Unification/Synchronization**	Establishes a consistent time base across different data modalities (Tang et al., 2025).
Time Series (General)	**Temporal Alignment (Bucketing)**	Multimodal data is aggregated into a specified time granularity (Han et al., 2024). The time interval is partitioned into fixed-size buckets (Han et al., 2024; Hu et al., 2025; Tien et al., 2019).
Logs to Time Series	**Frequency Aggregation**	Log templates are converted into time series by counting the frequency of occurrences within defined time windows (Pham et al., 2024b; Wang et al., 2024; Zheng et al., 2024).

Data Modality	Requirement/Technique	Details and Purpose
Time Series (Anomaly Specific)	**Anomaly Alignment**	Time-series signals from the faulty node and candidate root-cause nodes are trimmed to their overlapping durations around the onset of the anomaly (Y. Lin, n.d.).
Metrics	**Sequence Decomposition**	To make time-series data more stable, it can be decomposed into trend and seasonal components using methods such as moving averages or Robust STL (Ge et al., 2024; W. Gu et al., 2025; Y. Zhang et al., 2021). This decomposition helps identify normal patterns and changes over time.

Essentially, the data preprocessing stage in contemporary RCA functions like a refining plant. Initially, the raw and diverse data (ore) must undergo cleaning (removal of impurities), followed by smelting (feature extraction and encoding) to form uniform components, and finally, it must be standardized (normalization and alignment) to ensure that the advanced machinery (ML/DL

and causal models) can accurately piece together the diagnosis (Araujo et al., 2023; Fu et al., 2025). It is very important to understand that mistakes in data preparation can harm the performance of best AI models. So, proper preparation is crucial to fully benefit from advanced AI systems (Duan et al., n.d.; Pham et al., 2024b).

2.4 Data Analysis

The analytical building block uses a machine learning-based approach. Instead of correlation or heuristic-based rules, machine learning along with graph-based or dynamic causal inference is used. Traditional RCA uses statistical-based analysis, such as Pearson correlation, rule-based approaches, or relies on simple heuristics, such as determining the root cause based on the earliest anomaly event.

Modern Root Cause Analysis (RCA) methods often use Causal Discovery. This involves creating graphs with techniques like the PC algorithm, Granger causality, and gradient-based methods, such as DAG-GNN in CausalRCA. More advanced methods include Graph Neural Networks (GNNs), Graph Attention Networks (GAT), and Hierarchical Reinforcement Learning (HRLHF) for complex reasoning and modeling dependencies (Gulenko, 2020; Jian, n.d.; Pham et al., 2024b; L. Wang et al., 2023; T. Wang & Qi, 2024; Wang et al., 2024; Z. Zhao et al., 2025).

Relying only on correlation is not enough because “correlation does not ensure causation.” Simple graph methods often struggle with changes and confounding factors. Advanced causal methods solve these problems by looking at relationships over time or treating faults as interventions (Sometimes called Intervention Recognition). This helps find the root cause by identifying changes in the mechanism. Dynamic causal modeling is critical for modern micro-services because

system dependency graphs and causal mechanisms often change during an anomaly (e.g., due to congestion or load balancing).

2.5 AI Taxonomy

2.5.1 Foundation Capabilities and Constraints of RCA Systems

RCA system in a micro-services environment requires balance between cost, speed, scale, and explainability. Four key evaluation criteria are used, which reflect the main trade-offs engineers face while building an RCA system.

Efficiency and Deployability. The RCA system must operate quickly and inexpensively. They should not consume excessive computing resources or require long training times. Deployability is not only about speed or cost, but also includes (C. Chen et al., 2023). * How easy to integrate RCA systems * How difficult it is to maintain * How much operational efforts it requires once deployed

Expressiveness and Modeling Power. Micro-service systems produce complex and non-linear behaviors. The RCA must be sufficiently powerful to model these patterns. The trade-off is that higher modeling power comes with a higher computational cost and operational complexity.

Scalability and Robustness. In a real environment, data are noisy, incomplete, and continuously evolving. Service dependencies and patterns change over time, which is known as concept drift (Jian, n.d.; Panwar et al., 2025). RCA system must scale to * hundred or thousands of metrics * Large number of interconnected services * constant change in system behavior

Causality, Interpretability and Actionability. Engineers must understand and trust the results.- Causality: Understanding the actual cause of failures, not just what was correlated (Z. Chen et al., 2022; Harutyunyan et al., 2023). * Interpretability: Understand why the model flagged certain metrics or services as problematic. * Actionability: Detailed guidance on how to fix the issue and not just highlighting impacted services (Y. Lin, n.d.; C. Zhang et al., 2024).

2.5.2 Statistical Modeling Approaches to RCA

Statistical modeling has been used in the RCA for a long time. These models are efficient, require less computation, and are easy to deploy.

Strengths. Efficiency, Simplicity, Low Overhead. Traditional statistics use rule-based structures to highlight anomalies in production systems. These can operate with very low computational resources. Techniques such as the 3σ rule, ARIMA, vector auto-regression (VAR), Principal Component Analysis (PCA), and basic clustering require minimal training data (F. Chen, Zhang, et al., 2024; Cheng et al., 2023; Hagemann & Katsarou, 2020a). These approaches are extensively used by practitioners and are easy to implement. Owing to their widespread usage, these are easy to explain and maintain.

Bayesian methods, particularly Multivariate Bayesian Online Change Point Detection (BOCPD), represent an important advancement in statistical modeling. BOCPD model detects the changes in time-series data using probablistic dependency. BARO framework uses this technique and its performance is better than baseline statistical models. This shows that anomaly detection can use the statistical model with improved accuracy and computatinal efficiency (Pham et al., 2024a; Pham et al., 2024d). Due to its computational efficiency, these methods scale well with

large dataset. It also exposes the relationship among different parameters. This transparancy makes esier auditing and compliance for engineer and it improves the trust (Shan et al., 2025). As a result statistical analysis has become default choice in preliminary RCA (Fernando et al., 2025; J. Qiu et al., 2019).

Limitations. Expressiveness, Assumptions, Robustness. The statistical method assumes data follows common patterns over time (Almajed et al., 2025; H. He et al., 2024). In complex micro-service environments, nonlinear patterns violate this assumption (H. He et al., 2024). These methods show poor accuracy, with VAR achieving F1 score of 0.50 and AUC of 0.74, while PCA reaches F1 score of 0.56 for anomaly detection (Z. Chen et al., 2022; Fernando et al., 2025). They struggle with high-dimensional data and dynamic changes (Almajed et al., 2025; F. Chen, zhang, et al., 2024; H. He et al., 2024).

In micro-service environments, unrelated components may seem interconnected due to concurrent interactions, causing erroneous conclusions (Ding et al., 2023; Pham et al., 2024a; L. Wang et al., 2023). Simple rules like the 3-Sigma method often trigger false alarms, requiring manual adjustments (Harutyunyan et al., 2023). The Bayesian Online Change Point Detection (BOCPD) method aims to detect changes intelligently but is limited by data-based assumptions in dynamic environments (N. Chen et al., 2023). These methods can model basic patterns but may misidentify normal behaviors as anomalies. Modern cloud monitoring systems need precision, adaptability, and minimal manual intervention (Qi et al., 2022; Xing et al., 2025). Statistical causal discovery uses algorithms like PC, FCI, and Granger causality to create causal graphs (Cheng et al., 2023; Pham et al., 2024d; Pham et al., 2024a). Systems like CloudRanger and Microscope use these

algorithms and scoring methods for root cause analysis, but evaluations show performance similar to random selection (Pham et al., 2024a).

Statistical Causality and Graph-Based RCA. Engineers use correlation-based methods to check if Key Performance Indicators (KPIs) meet requirements (T. Wang & Qi, 2024). However, these methods often miss complex interrelationships between services, leading to false alarms (Ding et al., 2023). Statistical methods analyze metrics to establish cause-and-effect relationships by creating causal graphs based on timing and dependency patterns (Cheng et al., 2023; Pham et al., 2024d; Pham et al., 2024a). The monitoring system uses these techniques to identify the most likely root-cause service (Cheng et al., 2023; Pham et al., 2024a; Pham et al., 2024d). These systems show limited effectiveness, with accuracy similar to random selection (Pham et al., 2024d). Determining cause-and-effect from observational data requires substantial resources (C. Chen et al., 2023; Hardt et al., 2024), while micro-service systems' dynamic nature makes this process unreliable in complex environments (Gan et al., 2023).

Some statistical techniques, such as MonitorRank, use a graph-based approach to determine the root cause of a problem (Ma et al., 2020). In this approach, a service is considered a node, and a service call from one service to another is considered an edge. Then, they applied Random Walk or PageRank to simulate how the problem might spread through the system (Fu et al., 2025; Pham et al., 2024d; T. Wang & Qi, 2024). Some tools rank services based on how likely they are to be the source of an issue. These approaches do not require labeled dataset and are often fast and lightweight. They can be run in a production system (Fu et al., 2025). However, they struggle in micro-service environments, where service dependencies frequently change,and a simple graph traversal method does not capture the full complex dependencies.

In the process of constructing pathways, these algorithms frequently assess the current steps and determine the subsequent edge within the system. However, they can be misled by patterns that appear correlated but lack any causal relationship (Soldani & Brogi, 2023).

Bayesian Inference and Probabilistic Improvements. Bayesian methods use probabilities which helps them to manage uncertainty, such as weather spikes in response time. The Bayesian online change point detection (BOCPD) uses this approach and monitors time-series data to detect system behavior changes. This typically indicates the start of failure. It considers multiple metric changes over time to detect anomalies. It can estimate the exact time when a failure occurs (Pham et al., 2024a). If the anomaly detection is incorrect, the subsequent steps conducted to determine the root cause will also fail (Pham et al., 2024a).

Some RCA systems use clustering techniques with a simple machine learning approach to group similar metric patterns. Tools such as MicroRCA and MicroDiag use this technique. They group normal and abnormal behavior patterns together in different groups (D'Angelo & d'Aloisio, 2024; L. Wu et al., 2020; Pham et al., 2024a). These methods are easier to implement, cheaper, and beneficial for early stage RCA. However, their reliability in micro-service environments is still a concern compared to other modern approaches (Heeb et al., 2023).

The key performance indicator-based RCA system refers to many KPI's such as latency, error rate, and CPU usage, simultaneously. Adtributor and Hotspot detect failures more reliably when problems are not shown in a single KPI (C. Chen et al., 2023; Yu et al., 2023). However, this analysis incurs a cost. These systems must process large amounts of high-dimensional data. This creates memory and computational overhead on the system and increases the overall cost (C. Chen et al., 2023). The results of these methods are too high. For example, they may identify problems

in the service but do not clearly highlight the exact metrics or resources that caused the issue (Yu et al., 2023; Z. Zhu et al., 2024). Because of this lack of clarity in the root cause, SRE must manually analyze the logs, traces, and metrics to determine the real reason. This significantly reduces the practical value of these systems (Z. Zhu et al., 2024).

2.5.3 Classical Machine Learning Approaches

Capability Shift Beyond the Rules. ML-based techniques represent a significant shift from rule-based methods to learning from data. ML models analyze past and current data to identify normal and abnormal problems. Common ML algorithms used for this classification problem include isolation forests, logistic regression, support vector machines, decision trees, and random forests. These models help detect abnormalities and identify service interactions (Aktaş & Kilinc, 2024; Almajed et al., 2025; Araujo et al., 2023). Therefore, ML-based approaches learn from data, can detect small or early stage abnormalities, and understand service relationships and metrics better. It helps to detect the root cause much faster with better accuracy compared to rule-based systems (Gupta, 2019). Rule-based systems require experts to configure all the rules according to the evolving system; however, this part is automated by ML models, which do not require this tuning (Sarda et al., 2023). However, ML models are effective when trained on quality data (Gupta, 2019). ML techniques are grouped into supervised learning, which requires labeled data; unsupervised learning, which works without labeled data; and reinforcement learning, which improves through feedback.

Supervised Learning: Accuracy vs. Label Scarcity. The model was trained on normal and faulty system behaviors. Anomaly detection and root cause analysis are considered binary clas-

sification (yes/no) problems (Almajed et al., 2025; Gupta, 2019). Common model algorithms used in this approach include decision tree (DT), random forest (RF), support vector machine (SVM), and logistic regression (LR). When high-quality data are available, the performance of the supervised learning model is quite accurate. For example, decision trees achieve 89%-95% accuracy in detecting anomalies from log data. Support vector machines achieve nearly 98% accuracy with system call data (Aktaş & Kilinc, 2024; Almajed et al., 2025; Z. Chen et al., 2022). The MEPFL approach combines several models, such as K-nearest neighbor (KNN), random forest, and multilayer perceptron (Aktaş & Kilinc, 2024; Tien et al., 2019). These combined models were applied to distributed traces, achieving an accuracy of 89.1% (Nobre et al., 2023b).

The real challenge with the supervised learning approach is the unavailability of labeled dataset. In a micro-service environment, collecting a labeled dataset is costly. Human experts have to label the dataset, which is a time-consuming and practically unscalable approach (Cheng et al., 2023; Han et al., 2024; Y. Lin, n.d.). This may introduce bias or any other form of inconsistency due to human error (Cheng et al., 2023). In addition, fault data are rare compared to normal behavioral data. This creates a data imbalance or skewed data. Models trained on such data suffer from accuracy (Cheng et al., 2023). This makes supervised models costly and less feasible in large-scale micro-service environments. This is why real-world RCA systems prefer unsupervised anomaly detection methods (Fernando et al., 2025; Nikiforov, 2018; Xin et al., 2024).

Unsupervised Learning (UL). Scalability versus False Positives. This method primarily solves the problem of labeled dataset. Typically, models learn the data, identify patterns from the data, and flag anything that looks unusual or different (Araujo et al., 2023; Cheng et al., 2023; Fernando et al., 2025; H. He et al., 2024). Basically, it attempts to find data that do not look like

the rest (Xing et al., 2025). Common model algorithms used in this approach include isolation forest (IF), one-class SVM (OCSVM), K-nearest neighbor (KNN), local outlier factor (LOF), and clustering approaches such as DBSCAN (Aktaş & Kilinc, 2024; D'Angelo & d'Aloisio, 2024; Duan, Xue, et al., 2024; Fernando et al., 2025; Ge et al., 2024; Hagemann & Katsarou, 2020b; Petrov et al., 2022; Sun, Lin, et al., 2025; S. Wang et al., 2024; Xin et al., 2024). Among these, IF is effective in identifying 93% of anomalies with accuracy when used to automatically label micro-service data (Aktaş & Kilinc, 2024; W. Gu et al., 2025).

These methods create clusters by defining normal behavior by looking at the data and identifying the cluster where very few data points exist, which are flagged as anomalies (Panwar et al., 2025). This helps them classify and highlight unseen problems; however, without experts, these methods cannot classify which are harmless changes versus real failure. This may result in a high number of false alarms (Cheng et al., 2023; Han et al., 2024). Models such as OCSVM are sensitive to parameter configurations. Poor parameter selection can significantly reduce the accuracy and reliability of microservice environments (Ge et al., 2024).

The advantage of UL methods is their quick production deployment because they do not require a labeled dataset. However, their output needs to be validated by experts to rule out false alarms (Duan, Xue, et al., 2024; Fernando et al., 2025). In a micro-service environment, data are time series, and there is often a causal relationship in the data where order and timing play a role. UL models struggle with time-series data (Duan, Xue, et al., 2024; Ge et al., 2024; Vervaet, 2023). Principal component analysis (PCA) and distance/density-based methods fail to capture complex behavior patterns in dataset (Z. Chen et al., 2022; Duan, Xue, et al., 2024; Vervaet, 2023). In a dynamically changing micro-service environment, subtle or evolving anomalies are often missed, and normal changes may trigger false alarms (Duan, Xue, et al., 2024). Due to these limitations,

UL approaches are combined with advanced or hybrid approaches to better understand event timing and context (N. Chen et al., 2023).

Interpretability and Localization Gaps. ML-based approaches are far better than statistical rule-based approaches. ML-based approaches do not clearly explain decisions and do not identify the exact root cause. As a result, the engineer must investigate the root cause.

ML models act like a black box, where these models provide the output that the service is faulty but do not provide a clear explanation of how and why they have reached this decision (D'Angelo & d'Aloisio, 2024; Gan et al., 2022a; Poghosyan et al., 2023). These models provide service-level insights without much detail. Engineers must spend time debugging the issue to determine the root cause (Z. Chen et al., 2022; Y. Lin, n.d.; C. Zhang et al., 2024; S. Zhang et al., 2026).

The ML model can help identify whether the service is working as usual or whether any failure has occurred in the service. However, these models do not explain why the service failed or the exact problem. Engineers must search through logs, codes, metrics, or traces to identify the root cause. These models lack fine-grained details (Z. Chen et al., 2022; Harutyunyan et al., 2023; Wang et al., 2024; Zhang et al., 2024). SRE/OCE must trust the results, understand the reasoning, and act on the output quickly. Without proper explanation, confidence in ML-based RCA will be low, which will impact operational efficiency (Z. Chen et al., 2022; Harutyunyan et al., 2023). Because this research focuses on interpretability, final grained fault localization, and actionable explanation (W. Gu et al., 2025; T. Wang & Qi, 2024). This gap is addressed using deep learning models with built-in interpretability and Generative AI techniques.

2.5.4 Deep Learning for RCA

Representation Learning and Reconstruction-Based Models. Deep learning models learn the normal system behavior by analyzing the data. Once trained, these models can spot anomalies by identifying the extent to which new data deviate from what they have learned. Two widely used models are autoencoders (Aes) and Variational Autoencoders (VAEs).

An autoencoder (AE) is a neural network that consists of two parts: an encoder and a decoder. The encoder compresses the input data into a simpler form. The decoder reconstructs the original data from the compressed form (Hagemann & Katsarou, 2020a). The key idea is the reconstruction error, which signifies the difference between the original data and what the model has reconstructed (Gupta, 2019; Hagemann & Katsarou, 2020a; Hrusto et al., 2022; Y. Lin, n.d.). Autoencoders (AEs) are trained using data that is free from faults, enabling them to learn the system's normal operational behavior, metric dependencies, and inherent noise. Upon detection of abnormal behavior, the AE model attempts to reconstruct the data, resulting in a significant reconstruction error. This elevated error serves as an indicator of a potential anomaly within the system. AE models are much simpler than LSTM-based autoencoders and can be parallelized efficiently (Fernando et al., 2025). The reconstruction-based approach is robust when the training data contain a small amount of noise compared to the forecasting method (Z. Chen et al., 2022).

Variational Autoencoders (VAEs) extend the idea of autoencoders by learning the probabilistic representation of data over a fixed one. They capture the system data in the latent space, which captures uncertainty and variation (Y. Jin et al., 2025; Xing et al., 2025; Z. Zhu et al., 2024). VAEs detect anomalies by measuring the reconstruction error or reconstruction probability. The reconstruction probability indicates how likely the data are to be under the learned model (N. Chen

et al., 2023; Wang et al., 2024; Z. Zhu et al., 2024). VAEs use probabilities; therefore, they are better at handling changing system conditions, such as daily traffic spikes and seasonal demand. This helps reduce false alarms (C. Ji & Luo, 2025b; Wang et al., 2024; Z. Zhu et al., 2024). The system performs much better with VAEs than with Aes because they can model complex uncertain patterns in time-series data (Gao et al., 2025; Xing et al., 2025; Z. Zhu et al., 2024).

Autoencoders are commonly employed as preprocessing mechanisms, facilitating the extraction of significant features prior to the application of other Root Cause Analysis (RCA) techniques. Both Autoencoders (AE) and Variational Autoencoders (VAE) are integral to the RCA framework, such as the Multi-level Root Cause Analysis (MRCA). VAEs are utilized to rank anomalies, enhance subsequent stages of root cause identification, and mitigate the impact of early detection errors (Wang et al., 2024). These functionalities render AE and VAE indispensable for both anomaly detection and root cause analysis within large-scale micro-service environments (Gao et al., 2025).

Temporal Dependency Modeling (RNN/LSTM/GR). Recurrent neural networks (RNNs), along with their enhanced variants, Long Short-Term Memory (LSTM) and Gated Recurrent Units (GRU), are employed to identify anomalies in time series data, such as system logs and metrics (Adeba et al., 2025; Ge et al., 2024; Girish & Rao, 2023; Hagemann & Katsarou, 2020b; Y. Jin et al., 2025; Liu et al., 2025; J. Qiu et al., 2019; Sun et al., 2024).

These models are designed to learn the sequence of events over time and predict subsequent occurrences. Discrepancies between the predicted and actual data are flagged as anomalies. Among these models, LSTM demonstrates superior performance due to its specialized gates, which enable the retention of information over extended periods (Girish & Rao, 2023; Hagemann & Kat-

sarou, 2020b; Jonnaikode, 2025; J. Qiu et al., 2019; Shetty, Bansal, Kumar, Rao, & Nagappan, 2021). This capability is utilized in root cause analysis systems such as DeepLog and LogAnomaly (Girish & Rao, 2023; X. Ji et al., 2025). In real deployment, the LSTM model fixed over 90% of the connectivity issues in offshore networks (Ahmad et al., 2025). The BLSTM achieved an F1 score of 0.717, which is better than that of the VAE and hidden Markov model (HMM), which showed strong performance in root cause analysis (Chang et al., 2024). RNN and LSTM are good at remembering past events, which helps them connect earlier system behavior with the current one. This is important for the RCA system, and that is why LSTM has become an integral part of hybrid RCA systems, such as LSTM-VAE and LSTM-Generative Adversarial Network (GAN), which detect anomalies across multiple related metrics (N. Chen et al., 2023; Ge et al., 2024; G. Jiang, 2024; Jonnaikode, 2025; Shivakumar, 2025).

Models process the data one step at a time, which makes them slower to train and run. In addition, the models are less suitable for real-time detection. This sequential processing blocks scalability and efficiency (N. Chen et al., 2023; Ge et al., 2024; Jonnaikode, 2025; Shivakumar, 2025). The new approach should aim to maintain the accuracy and improve the speed in the subsequent sections.

Parallel and Convolutional Models (CNN/TCN). Convolutional neural networks (CNN) and temporal convolutional networks (TCN) are effective in identifying patterns within time-series data, such as logs, particularly in detecting abrupt spikes. They encounter challenges with long-term trend analysis. CNNs operate by applying filters across data to extract local patterns, such as log events or performance fluctuations (Adeba et al., 2025; Gan, 2021; Hrusto et al., 2022; J. Qiu et al., 2019). Their parallel data processing capabilities render them more efficient than recurrent

neural networks (RNNs) (G. Jiang, 2024). When integrated with long short-term memory (LSTM) networks, CNNs enhance anomaly detection performance, achieving an F-score of 0.99 compared to 0.84 for CNNs alone (J. Qiu et al., 2019). Additionally, CNNs can extract features and reduce data dimensionality by transforming log keys into numerical representations (Adeba et al., 2025). TCNs employ dilated convolutions to examine long-term patterns without escalating computational demands (Xing et al., 2025). They train more rapidly than RNNs and overcome issues related to unstable gradients (Y. Jin et al., 2025). The TCN autoencoder requires 0.743 kiloseconds for training, in contrast to the 0.815 kiloseconds required by the variational autoencoder (VAE) (Chang et al., 2024). CNN models demonstrate high performance in real-world systems, achieving an accuracy of 94.99% when utilizing time-frequency spectrograms. The combination of CNN and LSTM networks enhances anomaly detection by capturing both local and long-term patterns (J. Qiu et al., 2019). CNNs and TCNs are particularly advantageous when rapid processing is essential (Adeba et al., 2025).

Global Dependency Modeling with Transformers. Transformer models capture relationships across entire sequences simultaneously, unlike LSTM's step-by-step approach. They identify long-distance relationships and early changes impacting later sequences (Hagemann & Katsarou, 2020b; Sun et al., 2024). These characteristics make transformers effective for multivariate time-series anomaly detection with complex metric interactions (C. Gao et al., 1 Aug.1, 2025; Ge et al., 2024).

The transformer handle data in parallel while using a multi-head attention method. This allows the model to analyze multiple relationships simultaneously. These models were trained faster and scaled better for real-time RCA systems. This is important for production-grade RCA systems (F. Chen, zhang, et al., 2024; Zheng et al., 2025). LogBERT and LogAnomaly use a

transformer-based approach to study log messages and identify related patterns across time. This helps identify meaningful patterns. These models converge faster than the LSTM-only approach (N. Chen et al., 2023).

The self-attention mechanism with a transformer checks how every data point relates to every other data point in the sequence. This helps the transformer learn long-range dependencies better than LSTM. These capture the relationship even if two events are far apart (Hagemann & Katsarou, 2020b; Huang et al., 2023). Attentions with transformers operate in parallel and avoid step-by-step processing (N. Chen et al., 2023; Ge et al., 2024; Huang et al., 2023). This makes them suitable for global temporal dependencies and feature interactions in multivariate time series data (F. Chen, zhang, et al., 2024; Gao et al., 2025; Ge et al., 2024).

Transformers reduce the training time and improve stability. RT-SemiVAE combines RNN with transformers and reaches stable training in 500 iterations compared to 800 iterations for LSTM-based approaches (N. Chen et al., 2023). HolisticRCA uses time-aware transformers with positional encoding and multi-head attention to better capture long-term metric behavior (Hou, 2025; Sun et al., 2024). Despite their strengths, standard attention mechanisms can struggle with complex repeating patterns in time-series data (F. Chen, Zhang, et al., 2024). To overcome this, some frameworks combine transformers with graph neural networks (GNN) to model temporal patterns and system dependencies (H. He et al., 2024). Transformers naturally provide explainability. Their attention weights show which part of the input sequence mattered most when making predictions. It helps to understand the model decision, trust the RCA output, and act faster during incidents. Therefore, transformers are widely used in systems attempting to achieve accuracy with interpretable anomaly detection results and maintain low computational costs (Sun et al., 2024).

Structural Dependency Learning with Graph Neural Networks (GNN). Modern micro-services have multiple services connected to each other in complex ways. These can be presented as a graph, where each service acts as a node and the communication path represents the edge (T. Wang & Qi, 2024). GNN are deep learning models designed to learn from such structures. Graph convolution network (GCN) and graph attention network (GAN) can model the propagation of faults or failures between services and detect abnormal behavior (Y. Jin et al., 2025; Somashekar et al., 2024; T. Wang & Qi, 2024).

GNN focuses on the relationship between services, whereas RNN, LSTM, CNN, and Transformer focus on metric changes over time. Each captures a different aspect of the system. The GNN helps to capture the origin of the problem, and temporal models detect when something unusual occurs. Both aspects are important for root cause analysis. Several RCA systems have successfully used GNN. CausalRCA learns causal dependencies using a DAG-GNN and then applies PageRank to rank the likely root causes (Pham et al., 2024a; Pham et al., 2024d). FaaSRCA uses a graph attention network (GAT) autoencoder on a global service call graph. It detects anomalies by identifying a part of the graph that cannot be reconstructed (Huang et al., 2024).

The GNN learns by passing messages between services that share information with their neighbors. This allows the models to learn the interaction pattern, detect abnormal behavior, and understand how the failure propagates. GNN work well with causal graphs that describe how failure propagates (Z. Zhao et al., 2025). This mechanism naturally fits the complex, non-linear structure of micro-services (Somashekar et al., 2024; Z. Zhao et al., 2025). Because they analyze the data sources together, GNN outperforms simple rule-based systems and manual analysis (Z. Zhao et al., 2025). A recent approach combined a GNN with a transformer to capture structural dependencies

using a GNN and temporal behavior with transformers which leads to more accurate results (H. He et al., 2024).

2.5.5 Generative AI and LLM-Driven RCA

Rule Generation and Explainable Detection. Objective of root cause analysis is not only to detect anomalies but to explain why certain anomalies has occurred. GenAI address this challenge by formulating a rule that are interpretable and reproducible. GenAI models, such as ARGOS, employ LLM to construct anomaly detection logic as explicit rules based on opaque model predictions (Y. Gu et al., 2025). These rules function as intermediate, human-readable representations that effectively connect the processes of anomaly detection and root cause reasoning. Rather than classifying the data as normal or abnormal, LLM describe the anomalies in natural language and then convert those descriptions into executable Python code (Y. Gu et al., 2025). The generated rules should explicitly state what abnormal behavior looks like. Engineers should be able to inspect, validate, and trace anomalies to specific metrics or logs. The logic used for detection should be auditable and reproducible. (Y. Gu et al., 2025). This will significantly improve the interpretability of the root cause analysis.

ARGOS has shown significant empirical performance improvement where F1 score is improved by 28.3% over state-of-the-art methods on Microsoft dataset.Rule generated by LLM are found comparable to and sometimes better than human authored rules (Y. Gu et al., 2025). This indicates that explainable rule-based detection does not require a trade-off in accuracy, making it suitable for production systems.

Generative Sequence Modeling. GenAI models are trained on what normal behaviors look like over time. These models can then check whether new data follow a similar trend. These models predict what happens next, and if reality is different, they raise a warning. LLM-based methods operate in the detector mode, where the model examines current and past data points and predicts the next value or event in the time series (Y. Gu et al., 2025). If the actual outcome does not match this prediction, it is termed an anomaly (Y. Gu et al., 2025).

Monitoring data are obtained sequentially over time. These models treat this data as a sentence that needs to be continued. Models are trained on fault-free data so that they learn the usual order and progression of events or metric values. If the system produces an event that does not fit the learned pattern, the model flags it as abnormal. For RCA, this helps to understand when the system behavior first deviated from the normal pattern. For logs, these models work in a similar manner. For example, the LogGPT model predicts the next log message based on previous log entries. If the actual message deviates from model's expectation, it is identified as an anomaly (X. Ji et al., 2025). This methodology is useful in RCA because unexpected log sequences can reveal hidden faults, misconfigurations, or cascading failures.

GenAI models can detect faults early before major incidents occur (Y. Gu et al., 2025). This shift from reactive diagnosis to proactive prevention complements traditional detection stages and enables intervention before anomalies escalate.

LLMs for Reasoning, Explanation, and RCA Automation. Large Language Models (LLMs) are skilled at generating explanations in straightforward, human-readable language, which helps engineers in understanding the nature and cause of issues. These models independently scan diagnostic data to pinpoint the most likely root cause with detailed explanation (Y. Chen et al.,

2023). For example, RCACopilot can predict the root cause along with the reasoning behind the prediction (Y. Chen et al., 2023).

LLM do not require large amounts of data for training. Using few-shot learning, they can adapt to specific system domains by learning a small number of examples (Y. Chen et al., 2023; Shan et al., 2025; Zhang et al., 2024). This is useful when historical incident data are limited or specific to a particular system. This is contradictory to traditional models, which are black boxes and provide predictions without explanation (Shan et al., 2025; Sun, Luo, et al., 2025). LLM show reasoning steps in a structured and logical manner (Sun, Luo, et al., 2025). This approach enhance explainability (Poghosyan et al., 2023) and foster trust in the result (Sun, Luo, et al., 2025).

Large Language Models (LLMs) improve root cause diagnosis by explaining their conclusions which enables engineers to verify results (Sun, Luo, et al., 2025). In-context learning helps LLMs to incorporate new examples during analysis which reduces cost of retraining (Shan et al., 2025; Zhang et al., 2024).

By combining high accuracy, fast response time, and clear explanation, LLM-based RCA significantly reduces the time and expert efforts required to diagnose an incident (Y. Chen et al., 2023; Sun, Luo, et al., 2025). It is well-suited for real-time incident root cause analysis, helping teams resolve issues faster with greater confidence (Y. Chen et al., 2023; Xiang et al., 2025).

Hallucination-resilient multi-agent. GenAI models are powerful, but they have a known weakness of hallucination. This means that the model can produce answers that may sound correct but are actually incorrect. To make AI reliable for RCA, modern sources combine different external knowledge sources and cooperating AI agents.

Instead of relying on what LLM models remember from training, Retrieval-Augmented Generation (RAG) allows models to retrieve verified information from specialized databases such as run books, historical incidents, or system documentation (Pei et al., 2025; Shan et al., 2025; T. Wang & Qi, 2024; W. Zhang et al., 2024). RAG reduces hallucinations and improves accuracy by referring to real, historical, and verifiable data (Shan et al., 2025; Sun, Zheng, et al., 2025). Advance GenAI framework use multiple AI agents each with specific role. These agents work together to cross-check the results and improve the decision quality. For example, the Flow-of-Action framework uses standard operating procedures (SOPs) to guide agent behavior and embed SRE best practices into the system (Pei et al., 2025). This ensures that the analysis follows the proven operational logic. MABC employs blockchain technology for voting purposes, wherein agents assess outputs to reach a consensus on reliable results (W. Zhang et al., 2024). Graph frameworks such as MetaGraph and StateGraph integrate LLM with graph databases to model system relationships, thereby facilitating the prediction of cascading failures (Xiang et al., 2025).

In multi-agent systems, distinct agents are designated to manage specific tasks, such as CodeAgent for code analysis and JudgeAgent for validation (Pei et al., 2025). This specialization reduces the dependence on single model. This reduces the hallucination impact and improves robustness. By combining external knowledge with coordinated, specialized multi-agent systems, the GenAI framework produces more accurate, explainable, and verifiable results. This will reduce the error rate and make RCA much faster and more dependable (Pei et al., 2025; Shan et al., 2025; W. Zhang et al., 2024).

2.5.6 Multi-modal Data Fusion and Hybrid Intelligence

Fusion Strategies (Early, Intermediate, Late). Micro-service systems in modern cloud environments produce many types of data, such as logs, traces, metrics, and topological data. Each of these data points presents one part of the system behavior. On its own, no single data source is sufficient to fully explain why a failure occurs. Multimodal data fusion solves this problem, leading to a more accurate and reliable RCA.

In early feature-level fusion, raw data from different sources are merged and analyzed together (Lee et al., 2023b; Yu et al., 2023; S. Zhang et al., 2023) Systems such as Nezha and DiagFusion combine metrics, logs, and traces into a single unified representation, such as an event graph or shared embedding space (Yu et al., 2023). Using this, Nezha achieved an accuracy of 89.77. In this case, learning occurs across all data types, which helps the system discover relationships between traces, metrics, and logs. Simultaneously, combining the data into a unified format may lose or hide some essential data characteristics (Tao et al., 2024).

In representation-level fusion, each data type is processed separately to extract important features, which are then combined (Lee et al., 2023b; Tao et al., 2024; S. Zhang et al., 2026). Eadro and HolisticRCA follow this method. They model behavior within individual services and interactions across services while preserving the unique characteristics of metrics, logs, and traces. It retains modality-specific information, allowing integration across data sources. However, it makes the system architecture more complex than early fusion (Tao et al., 2024).

In late decision-level fusion, each data type is analyzed independently. Different methods that suit the respective data are used. The final root cause decision is made by combining individual results through voting or weighted scoring (Tao et al., 2024; Yu et al., 2023). For example, Pdiag-

nose identifies the root cause separately from the metrics and logs and then fuses the outcomes to produce the final diagnosis (Yu et al., 2023). This keeps the data type fully independent and easier to implement; however, it loses the deeper relationships between data types (Tao et al., 2024).

Studies consistently shown that using multiple data type together produce better RCA results than relying on single source (Lee et al., 2023b; S. Xie et al., 2026; Yu et al., 2023). Systems such as TrioXpert, which integrate multimodal data with LLM, have shown improved speed and accuracy of diagnosis in real-world environments (Sun, Luo, et al., 2025).

Causal Graph Construction from Multi-modal Signals. The advanced RCA system synthesizes a comprehensive system view by integrating data into causal graphs that illustrate problem propagation. MULAN transforms log data into a time-series format utilizing LLM to facilitate analysis alongside metric data. It employs contrastive learning to detect abnormal behaviors and utilizes KPI-aware attention to highlight key indicators. MULAN integrates metrics and logs to construct causal graphs that elucidate the manner in which anomalies precipitate system failures (Zheng et al., 2024).

TVDiag focuses on building specific diagnostic tasks within multi-modal environments. Its causal learning method connects metrics and logs to understand failures. Causal graphs from multi-modal data make diagnosis more explainable and actionable (S. Xie et al., 2026; Zheng et al., 2024).

Hybrid Learning and Transfer Learning Approaches. Hybrid approaches improve root cause analysis by combining different learning methods. Instead of relying on one algorithm, a hybrid system combines models that are good at understanding a specific aspect of system behavior,

as follows.- CNN are effective in identifying spatial patterns, such as correlations in metrics or components. - LSTM is good at capturing temporal patterns over time. - GNN models are effective in identifying the relationship between services and call graphs. - CNN + LSTM captures both spatial and time-based behaviors. - The transformer + GNN captures long-range temporal patterns with a service-level structure.

These hybrid combinations achieve better accuracy than a single diagnostic model (H. He et al., 2024; J. Qiu et al., 2019).

Training the RCA model from scratch is costly and slow. Transfer learning addresses this issue by allowing models trained on one system to adapt to similar systems. For example, MAD-DoC is an unsupervised transfer learning approach. This clusters similar storage components and transfers the learned knowledge between them. This helps reduce the cost by 60-70% while maintaining or improving accuracy (Boesel et al., 2023). Similarly, the cluster model (CM) approach groups similar services and shares models within each group. This reduces the number of models by two-thirds and reduces computational and resource costs (Fernando et al., 2025). For large-scale micro-service environments, more accurate, scalable, and cost-effective approaches are required, and together, these approaches work efficiently while maintaining high diagnostic quality (Boesel et al., 2023; Fernando et al., 2025; H. He et al., 2024; J. Qiu et al., 2019).

2.5.7 AIOps as an End-to-End RCA Paradigm

Automated Data Ingestion. AIOPS systems automatically collect data from micro-services systems. The data include metrics, logs, traces, and topological data. Different failures behave differently, and the system reacts accordingly. For example, RCACopilot uses a special-

ized incident handler for different types, which ensures that each problem is analyzed using a suitable diagnostic strategy (Y. Chen et al., 2023). Similarly, CloudRCA combines logs, KPI's and service topology to gain a holistic system view (Y. Zhang et al., 2021). Eadro and TrioXpert analyze within-service behavior and cross service dependencies avoiding incorrect conclusions derived from single modal data (Lee et al., 2023b; Sun, Luo, et al., 2025). Rather than collecting everything, AIOPS intelligently collects data. ML models determine which signals are most important for RCA, which reduces storage and computational costs without losing accuracy (Y. Chen et al., 2023).

Intelligent Diagnosis Pipeline. Once data are collected, AIOPS coordinates multiple diagnostic techniques to identify the root cause.- ML, DL, and causal inference methods build dependency, topology, and causal graphs. These graphs show how services and metrics influence each other (H. Wang et al., 2022; T. Wang & Qi, 2024; Z. Zhao et al., 2025).

- Graph-based methods, such as TraceDiag, remove unnecessary components from service graphs using reinforcement learning, which simplifies diagnosis (Ding et al., 2023). BALANCE, another graph-based technique, groups anomalies into understandable categories, such as database errors and container faults, and explains why the cause is ranked highly (C. Chen et al., 2023).
- Causal discovery techniques, such as Granger causality and the PC algorithm, identify cause-and-effect relationships between metrics and services. Ranking algorithms, such as PageRank, highlight the most likely root causes (Pham et al., 2024a; Shan et al., 2025; Wang et al., 2024).

Instead of manual investigation, these methods produce a list of likely root causes, which reduces the Mean Time To Resolve (MTTR). This also helps reduce reliance on human expertise (Y. Chen et al., 2023; Ding et al., 2023; Soldani & Brogi, 2023).

Human-readable explanation. Finding the root cause is not enough. It must be understandable and actionable by the user. GenAI and LLM help produce complex outputs with clear explanations. LLM analyze diagnostic evidence, identify root cause categories, and explain their reasoning in human-understandable language (Y. Chen et al., 2023; S. Jin et al., 2024). SynergyRCA uses an LLM with RAG to produce RCA and recommend fixes (Xiang et al., 2025).

GenAI can automatically perform Five Why analysis. Knowledge graphs and live system data can be used to trace failures to underlying causes, such as coding mistakes or missing automation (S. Jin et al., 2024). This explanation turns technical RCA results into clear, actionable guidance, enabling faster remediation and decision-making (C. Chen et al., 2023; Shan et al., 2025; T. Wang & Qi, 2024). This aligns with the AIOPS goal of automated detection and resolution (Sarda et al., 2023).

Operational Impact: Reducing MTTR. The real value of AIOPS is the reduction in MTTR. SynergyRCA identifies the root cause in approximately 2 min (Xiang et al., 2025). RCA-Copilot achieves an accuracy of 0.7666 and enables fast and reliable diagnosis (Y. Chen et al., 2023). Higher accuracy means fewer false alarms and less time wasted on incorrect hypotheses. Some systems go beyond diagnosis and enable automated remediation processes. For example, the RCA output can be converted into ansible playbooks for direct execution (Esposito et al., 2025; Sarda et al., 2023). Transfer learning reduces the cost of maintaining RCA models across hundreds

of services (Boesel et al., 2023; Fernando et al., 2025). Multi-agent systems and voting mechanisms increase confidence in results and make automated actions safer (Pei et al., 2025; W. Zhang et al., 2024).

2.6 Economics

Organizations invest in RCA systems to reduce financial and reputational losses due to service downtime. The impact can be significant. For example, a Facebook outage in 2021 caused by a software defect resulted in an estimated loss of $ 60 million, while 37 minutes of downtime at YouTube led to approximately $ 1.7 million in lost advertising revenue (Sun, Luo, et al., 2025). Losses can occur if services are slow, not just when they are unavailable. Even a 400 ms increase in latency can reduce Google search traffic by 0.5% (Bansal et al., 2020). Traditional RCA rely on manual efforts and take a long time to identify the root cause. In contrast, GenAI-based solutions have shown better performance by reducing the mean time to detect (MTTD) and mean time to resolve (MTTR). These systems also suggest automated remediation that engineers can understand. This helps build trust and reduce financial impact (Sun et al., 2024).

Although traditional RCA are cost-effective, they have low accuracy and limited flexibility. They assume a simple linear relationship between variables, such as metrics, logs, or alerts. They used a fixed threshold to detect abnormal behavior. As a result, they struggle to adapt to dynamic environments (Fernando et al., 2025; Girish & Rao, 2023; J. Qiu et al., 2019). Empirical studies have shown the poor performance of statistical techniques, such as principal component analysis (PCA) or vector auto regression (VAR), with F1 scores between 0.4 and 0.5 (Z. Chen et al., 2022; Fernando et al., 2025; S. Jin et al., 2024). This means that alerts can be skipped or recurring incidents can be raised (Ge et al., 2024; Panwar et al., 2025; J. Qiu et al., 2019).

Classical ML uses algorithms such as support vector machines, decision trees, and isolation forests to detect issues in systems. It shows an accuracy of 90-95% in identifying issues in controlled settings. However, they require a labeled dataset, and the cost of labeling a dataset is very high. These models may suffer from high data imbalance, where normal pattern logs will appear most of the time, and error exceptions are one of the cases. Deep learning approaches further improve the performance using models such as autoencoders or LSTM (Fernando et al., 2025; C. Zhang, Peng, Sha, et al., 2022). This helps reduce the MTTD and failure impact, resulting in a measurable operational return on investment (ROI) (Hrusto et al., 2022; Jha et al., 2025).

Causal inference and Explainable AI (XAI) help improve interpretability and stability by providing trust and expert validation (L. Wang et al., 2023; Yu et al., 2023). For example, BALANCE is an attribution-based method that outperforms state-of-the-art models. This reduces the computational overhead and enables transparent reasoning (C. Chen et al., 2023). The LLM model provides a human-readable explanation. These models can automate reasoning, rule generation, and knowledge retrieval. Empirical evidence has shown a 45% reduction in incidents, which provides major operational benefits (S. Jin et al., 2024). Despite its benefits, GenAI incurs additional training costs, such as inference expenses, limited context windows, and hallucinations (Y. Chen et al., 2023; Zhang et al., 2024).

AIOPS delivers the highest ROI by reducing the mean time to resolve (MTTR) and labor dependency. Production deployment has observed RCA time reduction up to 96.5% and improvement in service availability and SLA compliance (Ding et al., 2023; W. Gu et al., 2025).

2.6.1 ROI Framework

Framework can be broken into three main component.

Quantifiable Losses. RCA framework can help in reducing financial loss due to downtime or service interruption. This can be measured using direct financial loss, prevention of SLA penalties and regulatory risk mitigation. For example, cloud providers such as AWS can give 10% service credit if up-time has dropped below 99.99% and 30% credit for up-time below 99.0% (Bansal et al., 2020; Gulenko, 2020). Monetary Authority of Singapore (MAS) has fined DBS Bank with six months pause on non essential business activities due to repeated failure (S. Jin et al., 2024). Table summarizes the financial loss impact due to slow RCA.

Table 8
Financial and Operational Impact of IT Service Disruptions

Metric	Financial Impact / Statistic	Source
Hourly Outage Cost	A single hour of downtime for Amazon on major shopping days can cost up to $100 million.	(Chakraborty et al., 2024)
Daily Outage Cost	A 24-hour incident affecting mission-critical microservices on AWS could result in $3.4 billion in revenue loss.	(Chakraborty et al., 2024)
Per-Minute Cost	The average cost of downtime for enterprise-level services exceeds $5,600 per minute.	(Chakraborty et al., 2024)

Metric	Financial Impact / Statistic	Source
Specific Incident Cost	A 37-minute downtime incident on YouTube was estimated to cost Google $1.7 million in advertising revenue alone.	(Chakraborty et al., 2024)
Latency Impact	A 400 ms increase in latency can cause a 0.5% drop in Google Search volume, directly affecting revenue.	(Bansal et al., 2020)
SLA Penalties	Cloud providers such as AWS offer 10% to 30% service credits when uptime drops below specified thresholds, such as 99.99% or 99.0%, respectively.	(Bansal et al., 2020)

Metric	Financial Impact / Statistic	Source
Regulatory Risk	Operational failures can lead to severe regulatory actions, such as the six-month pause on non-essential business activities imposed on DBS Bank by the Monetary Authority of Singapore.	(S. Jin et al., 2024)
Global Impact	IT downtime is estimated to cost $26.5 billion in lost revenue annually.	(C. Chen et al., 2023)

Operational Efficiency. Framework measure mean-time-to-resolve (MTTR) by drastically reducing the root cause identification. For example, Adobe has deployed the AI driven solution for root cause identification which reduced the overall analysis time from 3 hours to 15 minutes (Guntupalli, 2025). 41% SRE has reported that their work is manual and repetitive due to lack of automation (Hwang et al., 2021). AI driven RCA attempt to fix the root cause by fixing underlying configuration or code issues and prevent recurring failures (S. Jin et al., 2024).

Table 9
Operational Efficiency and Performance Improvements from AI-Driven Root Cause Analysis

Metric	Improvement / Efficiency Gain	Source
MTTR Reduction (Adobe)	Reduced Root Cause Analysis (RCA) time from 3 hours to less than 15 minutes.	(Guntupalli, 2025)
MTTR Reduction (Banking)	Reduced diagnosis time in a commercial bank from 30 minutes to 5 minutes.	(Chakraborty et al., 2024)
MTTR Reduction (GenAI)	The TrioXpert framework achieved diagnosis in an average of 26 seconds, compared with 2.5 hours for human experts.	(Sun, Luo, et al., 2025)
Investigation Savings	The DeCaf system saved engineers an average of 20 hours per investigation.	(Bansal et al., 2020)
Toil Reduction	Site Reliability Engineers (SREs) report that 41% to 50% of their work is manual “toil”; AI automation can help reclaim this time.	(Chakraborty et al., 2024)

Metric	Improvement / Efficiency Gain	Source
Incident Reduction	GenAI-driven RCA led to a 45% reduction in major incidents and a 45.5% reduction in change failure rates over one year in a global financial company.	(S. Jin et al., 2024)
Accuracy Improvement	AI-driven methods such as CloudRCA can help SREs reduce failure-resolution time by more than 20% compared with manual methods.	(Y. Zhang et al., 2021)
Actionable Guidance	GenAI copilots such as Nissist can achieve a 98.93% reduction in Time-to-Mitigate (TTM) for simple incidents by providing guidance to engineers.	(An et al., 2024)

Cost Of Implementation. Framework has to take into account continuous model training cost. If model's are mot trained regularly they may show drift the output (Guntupalli, 2025). Deep

learning model can be computationally expensive and resource hungry (Aktaş & Kilinc, 2024; Fernando et al., 2025). Additionally these models could be black box in nature. SRE may not be readily accepting its output. This cost vs marginal accuracy gains needs to be evaluated with trust factor and used judiciously (Chakraborty et al., 2024; Z. Chen et al., 2020).

GENERATIVE AI / LLM COST ANALYSIS – DETAILED.

Baseline Cost Calculation.

- Log processing: ~2,000 tokens (logs)
- Metric processing: ~1,000 tokens (time-series data)
- Trace analysis: ~2,000 tokens (request graph)
- Context/instructions: ~500 tokens
- **Total per attempt: ~5,500 tokens**

GPT-4 Pricing (as of 2025).

- Input: $0.03 per 1,000 tokens
- Output: $0.06 per 1,000 tokens
- **Cost per attempt:** (5.5 × $0.03) + (1.5 × $0.06) ≈ **$0.26**

Typical RCA Requires Multiple Attempts.

- Initial diagnosis attempt: 1 × $0.26
- Hypothesis refinement: 1–2 × $0.26 (if results are unclear)
- Verification queries: 1 × $0.26

- **Average attempts per incident: 3**
- **Cost per incident:** 3 × $0.26 ≈ **$0.78**

Table 10
Annual Inference Cost by Incident Volume

Incidents per Month	Incidents per Year	Estimated Cost per Incident	Estimated Annual Inference Cost
10	120	$0.78	$93.60
20	240	$0.78	$187.20
100	1,200	$0.78	$936.00
500	6,000	$0.78	$4,680.00

Table 11
Cost and Resource Requirements of AI-Driven Root Cause Analysis

Cost Component	Details & Estimates	Source
Inference Cost (LLM)	Identifying a single fault type using a model such as GPT-4 can cost approximately $0.04 per input; this can become expensive at high log volumes.	(Han et al., 2024)

Cost Component	Details & Estimates	Source
Training Resources	Fine-tuning large models (e.g., GPT-3-scale models) is resource-intensive, requiring clusters such as 16 V100 32 GB GPUs for training and 4 V100 GPUs for inference.	(Zhang et al., 2024)
Data Labeling	Obtaining high-quality labeled data for supervised learning is "significantly tricky" and incurs substantial labor costs for expert annotation.	(Y. Chen et al., 2023; Cheng et al., 2023; Duan, Bao, et al., 2024)
MLOps Debt	Continuous retraining is required to prevent "model staleness" caused by system evolution and concept drift; failing to do so can make maintenance more expensive than the value provided.	(Guntupalli, 2025)

Cost Component	Details & Estimates	Source
Computational Overhead	Data preparation modules, such as those used in MicroRank, can consume up to 60% CPU utilization on a single core to mine detailed trace information.	(Yu et al., 2021)
Instrumentation Cost	Heavy instrumentation, such as kernel-level tracking required by some proactive models, can be impractical in large production systems because of performance overhead.	(Gan et al., 2021, 2022b)
Knowledge Base	Constructing the necessary Knowledge Graph to support GenAI agents requires end-to-end automation of the SDLC and represents a continuous, resource-intensive effort.	(S. Jin et al., 2024)

```
ROI = ((Revenue Loss Avoided + SLA Penalties + Labor Cost Reduction) -
(Compute cost + MLOps maintenance))/(Compute cost + MLOps maintenance)
```

Where:

- Revenue Loss Avoided = Downtime cost/hour × hours saved/year
- Service Level Agreement (SLA) Penalties Prevented = Annual SLA credit reductions
- Labor Cost Reduction = (Current SRE time – New SRE time) × SRE cost/hour
- Compute Cost = GPU/infrastructure annual expense
- MLOps Maintenance = Model retraining + data collection + monitoring
- Interpretation:
  - ROI > 100% = Positive return (benefits exceed costs)
  - ROI = 0% = Break-even (benefits equal costs)
  - ROI < 0% = Negative return (costs exceed benefits)

Example: ROI of 1,000% means for every $1 spent, you get $10 back. Over a year, a $50k investment with 1,000% ROI generates $500k in benefits. Below table summarizes the impact of AI based RCA from operational, cost and implementation perspective.

Table 12
Comparison of Traditional and AI-Driven Root Cause Analysis Approaches

Metric	Traditional/Manual RCA	Advanced AI/GenAI RCA
Resolution Time (MTTR)	Hours (2–3 hours average) (Guntupalli, 2025; Sun, Zheng, et al., 2025)	Minutes/seconds (<5 minutes) (W. Gu et al., 2025; Sarda et al., 2023)

Metric	Traditional/Manual RCA	Advanced AI/GenAI RCA
Cost of Downtime	High (e.g., $100M/hour exposure)	Minimized through rapid mitigation
Operational Cost	High (human labor/toil)	Lower through automation
Inference/Compute Cost	Negligible	High ($0.04/query, plus GPU costs)
Setup/Data Cost	Low	High (data labeling, Knowledge Graph construction)
Accuracy/Resolution	Coarse (service-level)	Fine-grained (code/metric-level)
Recurrence Rate	High (symptom-focused remediation)	Potentially lower through root-cause elimination (Guntupalli, 2025)

2.7 Gaps and Future

The literature has identified operational, technical, and economic gaps in the RCA systems. Lets explore them.

2.7.1 Technical and Data Challenges

Root cause localization and multi-modal fusion. Current RCA systems face challenges in analyzing data from different sources, such as metrics, logs, or traces, in sufficient depth. However, multi-modal data may be inconsistent or contain noise. The state of the system is not captured correctly owing to the incomplete handling of multi-modal data (Y. Chen et al., 2023; Fu et al., 2025).

The velocity of the observability data is very high, which creates challenges for both traditional and modern RCA systems (Wang et al., 2024; Yu et al., 2023). Although multi-modal data processing is widely accepted, it requires additional data fusion techniques (Sun, Luo, et al., 2025). Sometimes, one modality data can hide the symptoms of other modality data (Fu et al., 2025).

Traditional machine learning approaches have provided service-level results that lack code or configuration-level insights for accurate fault localization (Aktaş & Kilinc, 2024; Wang et al., 2024; Xin et al., 2024). Engineers must manually investigate the service to identify the fine-grained root cause (Yu et al., 2023; Z. Zhu et al., 2024).

Overall, achieving a high-quality integration of multi-modal data for precise root cause localization is challenging.

Data, organization, and MLOPS barriers. The lack of labeled dataset is a major challenge in AI-based RCA systems. Labeled faults require efforts from expert engineers, and real fault data are occasionally rare (Harutyunyan et al., 2023). This limits the ability of the supervised model to perform, and the unsupervised model performs with fewer labels, but it generates false positive data (C. Chen et al., 2023; Hardt et al., 2024; Y. Lin, n.d.; Soldani & Brogi, 2023; Sun,

Luo, et al., 2025). Organizations are reluctant to proceed with autonomous cloud deployments because of the lack of a standardization framework and limitations in cross-vendor usage (Ahmad et al., 2025; Shetty et al., 2024). Frequent updates may cause concept drift, requiring continuous training. This creates additional costs and infrastructure burdens (Shan et al., 2025; Xiang et al., 2025; Zhang et al., 2024). In-context learning (ICL) helps reduce training costs, but requires a robust foundational system (Ahmad et al., 2025; Zhang et al., 2024).

Interpretability and Explainability. Many RCA systems fail to gain the trust of service reliability engineers (SRE), who must perform remediation actions (Harutyunyan et al., 2023). These systems do not produce interpretable and actionable insights for SRE. Deep Learning and Machine Learning algorithms are black box for SRE. They do not provide clear reasoning or causal relationships behind remediation actions (C. Chen et al., 2023; Sun, Luo, et al., 2025). However, traditional RCA provide high-level outcomes, such as identifying failure services (Aktaş & Kilinc, 2024; Wang et al., 2024; Yu et al., 2023). These methods fail to identify the exact root cause. The SRE must perform additional steps to determine the root cause (Yu et al., 2023; Z. Zhu et al., 2024). This highlights the need for clear, actionable, and transparent explanations of failures.

Temporal causality and causal inference in dynamic state. Identifying the cause-and-effect relationship with respect to time is a challenge. Many ML/DL-based systems ignore the temporal relationship owing to computational cost or performance. Typically, these systems fail to capture event sequences. For example, the CPU usage of a service increases before an increase in the latency (Hou, 2025; C.-M. Lin et al., 2024; L. Wang et al., 2023). Additionally, multi-modal fusion ignores the time-based nature of fault propagation, which does not reflect the cause-and-

effect relationship in the causal graph (Hou, 2025). Causal discovery algorithms, such as the PC algorithm, are very slow and resource-intensive. In addition, their performance is poor when the data are limited (C. Chen et al., 2023; Pham et al., 2024d).

The problem of temporal dependencies exacerbates in large-scale micro-service environments, where dependencies are dynamically changed. These changes occur because of deployment, autoscaling of services, or configuration updates. Service dependencies and causal relationships can shift while an anomaly occurs (Fu et al., 2025; L. Wang et al., 2023). In such scenarios, statistical rule-based methods struggle to perform, but deep learning algorithms can still handle the situation; however, their learning and training costs are often high (Fu et al., 2025; W. Gu et al., 2025).

Metric-based techniques can produce misleading signals. Correlated signals are not necessarily causally related to each other. This creates false alarms and incorrect root cause identification (W. Gu et al., 2025; Hardt et al., 2024; Soldani & Brogi, 2023). Graph-related systems that create dependency graphs from static rules require manual efforts to define traces and rules. Creating a reliable causal graph from operational data is difficult (Gan, 2021; Pham et al., 2024d). The graph becomes unstable when false correlations build causal links under dynamic conditions. RCA must adapt to changing system topologies and identify how faults evolve over time. The high cost and complexity of building causal links between dependencies create challenges in fully autonomous remediation flows.

Model Hallucination and Reliability. A major challenge with GenAI-based RCA is model hallucination. The model can produce fabricated outputs, and acting on them may result in misinterpretation in the analysis (Cornacchia et al., 2025). LLM are effective in processing huge logs, but

they lack deterministic behavior,which makes their output unpredictable (Y. Gu et al., 2025; Pei et al., 2025). Data are often processed in chunks, and processing large volumes of data at once is not possible. This limits their usefulness in end-to-end RCA (Zhang et al., 2024). LLM actions are constrained by predefined tools that are costly (Shi et al., 2025). To reduce this, external guardrails and a human-in-the-loop approach are required. Fully autonomous remediation remains immature and unstable (Cheng et al., 2023; Esposito et al., 2025; Sarda et al., 2023).

2.7.2 Operational Gap

Simplistic or Isolated Models. Many existing RCA methods rely on overly simplistic anomaly detection techniques like N-Sigma or BIRCH clustering (Pham et al., 2024a; Wang et al., 2024). Studies often consider AD and RCA as different stages (Lee et al., 2023b; Wang et al., 2024). When these simple detectors provide incorrect results, such as noisy labels, they significantly reduce the success and accuracy of subsequent steps in identifying the root cause (Lee et al., 2023b; Pham et al., 2024a; Wang et al., 2024).

Although various studies have focused solely on identifying the root cause, they have assumed the existence of an anomaly detection module that can accurately detect failures and trigger RCA (Pham et al., 2024a). In practice, current RCA methods commonly use simple techniques such as N-sigma, even when the data are not normally distributed (Wang et al., 2024). Based on the accuracy of anomaly detection, further localization steps were performed (Wang et al., 2024).

This highlights the need for a unified end-to-end solution for RCA, such as BARO and KPIRoot+ (W. Gu et al., 2025; Pham et al., 2024a; Pham et al., 2024d).

2.7.3 Economic Gap

False Alarm and missed anomalies. False positives occur when the system classifies normal events as problems, and false negatives occur when the system fails to highlight the actual issues (Soldani & Brogi, 2023; S. Zhang et al., 2026). In the first case, resources and time are wasted on analyzing an issue that did not occur, and in the second case, failure to identify the actual issue leads to prolonged downtime. This also breaks the subsequent RCA pipeline flow, such as the localization and remediation steps (Wang et al., 2024). Both issues incur additional costs and have economic impacts.

Economic Impact of Advanced AI Models. GenAI and DL models require incremental training using the latest dataset. This adds to the training and resource consumption costs. In addition, the GenAI model can process a limited set of tokens, and multi-modal data may be sufficiently large for processing and generating accurate results. The cost of a single input for identifying one fault using GPT-4 is approximately $0.04 (Han et al., 2024). The economic viability of the GenAI model is constrained by a shift in development efforts to per-inference token consumption (Han et al., 2024; Shan et al., 2025). Data collection requires a high CPU, memory, and disk storage. In comparison, traditional RCA are slow but cost-effective. Organizations must be careful in choosing LLM models and comparing cost-benefit analyses. To address this gap, current research is focusing on in-context learning (ICL) (Zhang et al., 2024, 2024).

2.7.4 Research Gap

Fragmented AIOps System. Currently, anomaly detection (AD), root cause localization (RCL), and automated remediation (AR) are treated as separate steps. Currently, no end-to-end platform seamlessly connects RCA with automated execution. An automated solution is effective when anomaly detection and root cause localization are accurate and reliable. Fragmented RCA phases create challenges in building fully automated AIOps systems. In addition, the remediation engine should correctly interpret the insights and execute appropriate recovery actions (Cheng et al., 2023). According to (Sarda et al., 2023), future research should focus on integrated, self-contained AIOps systems.

Addressing root causes. The focus of SRE is to restore the service as quickly as possible and reduce damage. This leads to fixing the symptoms over the actual root cause, such as restarting the service or patching the software (S. Jin et al., 2024). This may leave the actual configuration, code, or design flaws unaddressed, which creates technical debt (S. Jin et al., 2024; Zhang et al., 2024). Using GenAI with the Five Why technique can help in finding the actual root cause. This may reveal hidden issues from the code and help prevent the same problem from occurring again (S. Jin et al., 2024).

Benchmark and continuous learning are also important aspects of this field. There is a lack of publicly available dataset to identify the root cause. It is difficult for researchers to compare different RCA methods and produce experimental results. Some simulated dataset fail to capture the variability and complexity of actual production systems (Cheng et al., 2023; S. Zhang et al.,

2026). Another issue is the dynamic system change or concept drift. The system evolves over time, and static anomaly detection and RCA models quickly lose accuracy owing to the lack of training (Soldani & Brogi, 2023). Most RCA approaches do not utilize human feedback and knowledge to improve the model's performance (Cheng et al., 2023; C. Wang et al., 2024). This leads to repeated errors and declining reliability. In addition, there is no widely accepted benchmark to evaluate the RCA performance (Cheng et al., 2023). The AIOps lab provides a holistic evaluation framework with support for fine-grained root cause analysis and comprehensive assessment of agents (Cheng et al., 2023). The lack of standard benchmark and real-world dataset prevents the fair and reliable evaluation of RCA systems.

Human-in-the-loop Optimization. Most RCA systems do not use human knowledge to improve model accuracy via a human feedback loop. Human factors, such as organizational and team structures, are under explored (Cheng et al., 2023; L. Wang et al., 2023). These dependencies can significantly affect system reliability. For example, the T-CausalRCA framework introduced human expertise via hierarchical reinforcement learning with human feedback (HRLHF) to support causal discovery (L. Wang et al., 2023). The SRE has domain expertise and a good understanding of causal relationships and system behavior. Effective AIOps should support human-in-the-loop feedback for better performance. Agentic AI can assist in service decoupling strategies to reduce failure propagation (Esposito et al., 2025). Future research should focus on developing methods to quantify, capture, and apply human expertise (Esposito et al., 2025; L. Wang et al., 2023; S. Zhang et al., 2026).

Model Adaptability. Future research should focus on adaptable models. When dynamic system changes occur, the model should be adapted to new types of failures. This will help move away from dynamic, rigid models. Model retraining incurs additional costs and resources (Pei et al., 2025). Research increasingly highlights few-shot and meta-learning techniques. This helps the model learn and adapt to new changes without any concept drift. This also reduces the need for large dataset and heavy computational resources (Duan, Bao, et al., 2024). The RCA system should be able to highlight the failure, even if no prior information is available on the failure (Zhang et al., 2024). Future studies should prioritize adaptive learning methods (Duan, Bao, et al., 2024).

3 Research Method and Design

3.1 Introduction

This chapter specifies the research design and methodological framework for the doctoral study Optimizing Software Defect Management, Costs and Operational Efficiency Using AI-Driven Root Cause Analysis and Automated Triage in Micro-service Environments. The study will produce a single primary research artifact — the Heuristic Learning for Integrated Observability Systems framework (HELIOS) — whose construction, ablation, and empirical evaluation will constitute the central contribution of the dissertation. Because the intended output is a purposeful, novel artifact rather than a theoretical generalization about a pre-existing phenomenon, the methodological foundation is Design Science Research (DSR), in the formulation articulated by (Hevner et al., 2004), (Hevner, 2007), and (Peffers et al., 2007). The methodological apparatus comprises three interlocking elements: a pre-registered ablation study of eight confirmatory variants exercised across an AIOpsLab-grounded benchmark corpus with cross-benchmark probes, against eight ablation hypotheses (A-H1 through A-H8) under Holm–Bonferroni family-wise error control with pre-registered rank ordering; a pre-registered baseline-comparison evaluation against five priority-ranked AIOps systems on eight hypotheses (B-H1 through B-H8) corrected as a separate Holm–Bonferroni family; and ten exploratory hypotheses (E-H1 through E-H10) that will generate descriptive evidence informing future confirmatory work without entering the confirmatory hypothesis families.

The chapter is organized as follows. Section 3.2 positions the research philosophically. Section 3.3 specifies the research design, the DSR mapping, the intervention, the unit of analysis, the time horizon, and the construct-to-variable operationalization. Section 3.4 describes the research

setting. A business evaluation layer then maps the technical RCA and triage outcomes onto organizational value after deployment, and specifies the managerial use, the staged adoption path, and the decision rights that go with that deployment. Section 3.5 presents the population and sampling strategy. Section 3.6 specifies HELIOS as the experimental artifact with six named contributions (C1 through C6) decomposed into architectural, algorithmic, and methodological-discipline categories. Section 3.7 describes data-collection procedures, including the exploratory human evaluation that addresses Research Question 4. Section 3.8 specifies the data-analysis plan, and Sections 3.9 through 3.11 discuss validity and reliability, ethical considerations, and methodological limitations.

3.2 Research Philosophy

The philosophical stance of this research is critical realism, tempered by a pragmatist orientation toward artifact evaluation. Critical realism, as developed by Bhaskar (2008/1975) and applied in information-systems research by (Mingers, 2004), holds that reality is stratified into empirical, actual, and real domains, and that generative mechanisms in the real domain produce events in the actual domain, only some of which are observed in the empirical domain. This stance is well suited to the present research domain because microservice failures are produced by real, structural generative mechanisms — fault propagation through typed dependency chains, resource-exhaustion cascades across shared infrastructure, agent reasoning failures in the cognitive substrate — that yield observable symptoms in telemetry streams, yet no single observation constitutes a complete picture of the underlying cause. The epistemic task of a root-cause-analysis (RCA) system is precisely to infer the structural mechanism from the partial telemetry trace it has available; HELIOS will be built on the realist premise that such inference is meaningful because the mechanisms

exist, and on the epistemological premise that such inference is fallible because the mechanisms are only partially observable.

Within this realist frame, the evaluation of HELIOS will adopt a pragmatist stance following the DSR tradition (Hevner et al., 2004; Peffers et al., 2007). The validity of the artifact is not established through correspondence with a pre-existing theory or through the successful prediction of a natural-kind phenomenon; it is established through the artifact's demonstrated utility in solving a specified design problem, measured against pre-registered hypotheses on pre-specified metrics using pre-specified statistical procedures. This pragmatist commitment justifies the emphasis throughout the chapter on falsifiability at the level of each named contribution: every contribution C1 through C6 is ablatable by a feature flag, every ablation corresponds to a hypothesis, and every confirmatory hypothesis is powered on a pre-registered minimum-detectable-effect target.

Two implications follow. The research will not claim that the mechanisms HELIOS infers constitute a complete characterization of microservice faults; it will claim only that HELIOS, under the pre-registered evaluation protocol, will produce inferences that are more accurate, faster, or cheaper than those produced by comparator systems under identical inputs. Nor will the research claim that the architecture of HELIOS is optimal; it will claim only that the architecture is ablatable, that the ablations will be informative about the mechanism of each contribution, and that the pattern of ablation evidence will be consistent with the hypotheses advanced. This modest epistemic posture is appropriate to a doctoral research artifact and protects against the overclaiming that has been a persistent critique in the AIOps sub-field (Pham et al., 2024c; T. Wang & Qi, 2024).

3.3 Research Design

3.3.1 Research Design Classification

The study is classified as Design Science Research with embedded empirical ablation and controlled quantitative comparison. It is not a case study, not an exploratory qualitative inquiry, and not a traditional hypothesis-confirmation experiment on a pre-existing system. It is a construction-and-evaluation study in which the construction of the artifact is itself a research contribution and the evaluation will be structured to falsify, at the level of each architectural mechanism, the claim that the mechanism contributes to the artifact's overall performance. The design integrates three empirical elements: an internal-mechanism ablation study isolating each of the five core contributions of HELIOS (C2–C6); an external comparative evaluation against five priority-ranked AIOps baselines; and descriptive reporting of exploratory variants that cannot, for well-specified reasons, enter the confirmatory hypothesis family under the pre-registration discipline. The methodological consequences of this classification — that the unit of analysis is the paired incident (Section 3.3.4), that the time horizon is cross-sectional with respect to the corpus and longitudinal with respect to ORAR's two-stage reward (Section 3.3.5), and that several mechanisms cannot be confirmatory validated on the benchmark corpus as shipped (Section 3.6.8) — are developed in the corresponding subsections.

3.3.2 Design Science Research (DSR) Framework

The research is organized around the three-cycle DSR framework of (Hevner, 2007). The relevance cycle is instantiated through the dissertation's motivating context: the economic and operational cost of manual triage in cloud-native microservice environments and the emergent class

of agentic-AI failure modes for which no current AIOps framework provides first-class diagnostic support. The rigor cycle is instantiated through the systematic literature review in Chapter Two, through the explicit differentiation of each HELIOS contribution from its most proximate comparator (CHASE for UEG-C, mABC for MAHC, FrugalGPT for ORAR, Istio traffic mirroring for Fidelity-Gated Shadow Validation (FGSV), integrated control-plane self-monitoring patterns for IOL), and through the contribution-back-to-knowledge-base of the runtime-enforced evaluation-rigor discipline (C1) and the five design principles specified in Section 3.6.2. The (Peffers et al., 2007) six-activity DSR process is mapped to the dissertation as follows: problem identification and motivation in Chapters One and Two; objectives in Section 3.8.1 and the pre-registered hypotheses; design and development in the architecture chapter and Section 3.6; demonstration in the pre-registered ablation protocol (eight confirmatory variants × five benchmarks × 40 faults × 10 seeds = 16,000 planned runs) and the 174-fault baseline-comparison protocol; evaluation through the statistical procedures specified in Section 3.8.3; and communication through the dissertation and the Open Science Framework (OSF) pre-registration that will precede and govern data collection.

DSR evaluation rigor will be operationally enforced through architectural commitments that constitute Contribution C1 (Section 3.6.8). First, a Variant Control Layer (VCL) will read a content-hashed variant manifest at process start and emit an immutable feature-flag vector that gates runtime control paths; a flag flip changes which pipelines execute, which consensus rule fires, and which routing arm is selected. Second, a snapshot hash registry will record a content hash of every Unified Evidence Graph with Cognitive Nodes (UEG-C) snapshot and will be queried by downstream pipelines to confirm consumption of the declared snapshot. These two components — specified as Class 1 components of the Evaluation plane (runtime-affecting), as distinct from

Class 2 components (observation-only) — together with the metric integrity gate, the exclusion ledger, the deviation log, and the reconciliation ledger, constitute a research-method-as-system-architecture contribution that converts DSR evaluation rigor from a reporting convention into a runtime invariant of the artifact. Operational enforcement substantially strengthens, but does not by itself dissolve, every threat to paired-observation validity (see Section 3.9.1).

3.3.3 The Intervention Strategy

The intervention is the HELIOS artifact itself, deployed as a treatment in a comparative evaluation design. HELIOS will intervene in the RCA process by replacing existing monolithic or sequential diagnostic pipelines with a peer-pipeline architecture in which deterministic statistical reasoning (D-pipe), graph machine-learning reasoning (G-pipe), and multi-agent large language model reasoning (L-pipe) execute concurrently on the same content-hashed UEG-C snapshot and emit typed verdicts that are fused by the Multi-Agent Hybrid Consensus (MAHC) mechanism. The intervention is not the introduction of any single reasoning modality (each has precedent in the literature) but the structural co-deployment of all three under identical inputs with a cost-aware Operational-Reality-Aware Adaptive Routing (ORAR) mechanism that will select among them based on incident features, per-pipeline historical accuracy priors, and a pre-routing Action-Class Predictor (ACP). What changes under the intervention is the granularity at which RCA reasoning is adjudicated: comparator systems commit to a single architectural reasoning identity, whereas HELIOS shifts the reasoning choice to runtime under cost and latency constraints, with MAHC reconciling verdicts when multiple pipelines execute and the Chain-of-Evidence (CoE) ledger rendering the selection auditable.

The intervention will be executed under six strict control commitments that together constitute the failure-as-covariate discipline and the static-policy commitment:

- **Single-binary variant production**. All eight ablation variants will be produced from a single HELIOS binary (commit SHA fixed at OSF freeze) via feature-flag toggles; no variant is a separate code base.
- **Snapshot identity across variants**. The same UEG-C snapshot, identified by content hash, will be consumed by every active pipeline for the same incident–seed pair.
- **Seed register**. The pre-registered seed register (seeds 10,000,000 through 10,009,999) is binding.
- **Metric integrity gate**. Incident runs with missing measurements will be rejected rather than zero-imputed; excluded runs will be recorded to a cryptographically-signed, append-only exclusion ledger.
- **Price-book freeze**. Per-token and per-compute-second cost coefficients will be frozen at pre-registration; post-freeze price changes will be ignored for confirmatory analysis and noted as an external-validity threat.
- **Static-policy commitment**. The ORAR LinUCB per-arm parameters (A_a, b_a) for a □ {D, G, L, D+G, D+L, G+L, Full} and the Action-Class Predictor weights will be frozen at a pre-warmed state for the entire confirmatory protocol, so that the k-th incident's routing decision is a deterministic function of incident features and frozen parameters and is not conditional on the preceding k−1 incidents' outcomes. This static-policy commitment will preserve the within-pair independence assumption of the paired-comparison design (see Section 3.6.5).

The pre-warmed state will be constructed from a dedicated pre-training corpus that is disjoint from the 174-fault confirmatory corpus and from the 16,000-run ablation corpus. The pre-

training corpus will comprise 1,200 synthetic incidents generated by Chaos Mesh on an isolated three-service synthetic microservice testbed (one stateful database service, one stateless API service, one client-emulator service) that does not appear in any of the five evaluation benchmarks (AIOpsLab, DeathStarBench, Train-Ticket, PetShop, OpsEval) and that is not derived from any of them. The deliberately reduced topology is chosen so that the router will observe fault-class transitions rather than benchmark-specific (fault, service) instances. The synthetic testbed, its fault-injection script, the seed sequence governing the pre-training pass, and the resulting parameter artifacts will be deposited at OSF as a tagged immutable release; their content hashes will be recorded in the same registry that hashes the confirmatory corpus, so that a SHA-256 intersection check at the (fault-type, service-family, injection-parameter) granularity will be executed at OSF freeze. A non-null intersection at freeze would vacate the static-policy commitment, and the affected portion of the confirmatory protocol would be moved to exploratory status before any analysis is conducted. The Variant Control Layer will verify the hash of each loaded parameter artifact against the hash recorded at OSF freeze at every confirmatory run start; hash divergence will block execution.

A second disclosure level addresses the construction protocol itself, distinct from the contents-level disclosure above. The fault-class proportions, severity distribution, and injection-parameter ranges that govern generation of the 1,200-incident pre-training corpus will be pre-registered at OSF prior to corpus generation, and a SHA-256 hash of the construction script — together with the random seed sequence governing fault sampling on the synthetic testbed — will be recorded at freeze and re-verified at any corpus regeneration. This protects against post-hoc selection of construction parameters that would favor any particular routing arm of the LinUCB policy. The chapter acknowledges as a residual construction-validity threat that pre-registration of construction parameters does not, on its own, guarantee that the parameter choices were

neutral with respect to HELIOS's architecture, since the synthetic testbed and the construction script were designed by the same researcher who designed HELIOS; external replication of the pre-warming pass on an independently constructed synthetic testbed is identified in Section 3.11.5 as a priority direction for follow-on research, and is the only intervention that fully neutralizes the construction-validity threat at the process level.

A residual structural-similarity threat is also acknowledged. Even with hash-verified disjointness at the (fault-type, service-family, injection-parameter) granularity, structural similarities between the synthetic pre-training testbed and the AIOpsLab benchmark family — for example, shared HTTP request-response semantics, shared OTEL instrumentation conventions, or shared anomaly-class taxonomy at the conceptual level — could in principle transmit information from the pre-training distribution to the confirmatory distribution through the routing policy, even where no specific fault instance is shared. This residual exposure is bounded by the small dimensionality of the LinUCB feature space (approximately 12 to 18 features comprising incident features and the ACP stateful-action feature block) relative to the diversity of the confirmatory corpus, and by the deliberate construction of the synthetic testbed to expose the router to fault-class transitions rather than to specific instances; neither bound is a guarantee. The threat is reported in Section 3.9.1 (Threat 8) and Section 3.11.5; the chapter identifies external replication on a structurally distinct microservice paradigm — for example, an event-driven serverless mesh in place of a request-response Kubernetes deployment — as the only intervention that fully neutralizes the threat at the process level, and does not claim to dissolve it through the present disclosure regime alone.

3.3.4 Unit of Analysis

The primary unit of analysis is the paired incident instance: a single fault injection, executed under a specified seed, producing paired observations on HELIOS-Full and on each comparator variant or baseline system under identical telemetry access. Pairing at the incident level is both statistically and philosophically necessary. Statistically, paired observations reduce variance attributable to fault heterogeneity and admit the non-parametric Wilcoxon signed-rank test as the primary inferential instrument; the power analysis grounded in CHASE's reported HR@3 performance (Cohen's h = 0.276; derivation in Section 3.8.3) yields a minimum required n of 73 paired faults at Holm–Bonferroni $\alpha = 0.00625$ and 80 percent power, inflated to n = 84 under a 15 percent attrition allowance, which justifies the planned 174-fault delivered corpus at 2.1× the attrition-inflated minimum (see Section 3.5 and Section 3.8.3). The validity of the paired design relies on within-pair independence of treatment effects across incidents, which is operationally protected by the static-policy commitment in Section 3.3.3. Philosophically, the paired-incident unit respects the critical-realist premise that each incident is a distinct generative event, and the pragmatist premise that each incident is a distinct problem instance on which the artifact's utility will be measured.

Two secondary units supplement the primary unit. The first is the variant–benchmark–metric cell (the atomic unit of confirmatory inference), with eight confirmatory variants × five benchmarks × five confirmatory metrics yielding 200 cells in the pre-registered results matrix. A cell will be eligible for confirmatory inference only if at least 320 of 400 planned paired observations (80 percent) are included under the inclusion criteria specified in the pre-registration; cells below this threshold will be reported as "insufficient data" with the completed count and the reason,

and they will not enter the hypothesis tests. The second supplementary unit is the contribution–ablation pair: the binding of each named core contribution (C2–C6) to its designated primary ablation variant under the discipline specified in Section 3.6.7 and Section 3.6.8.

3.3.5 Time Horizon

The primary time horizon of the empirical evaluation will be cross-sectional. The benchmark corpus will be fixed at pre-registration freeze (with SHA-256 hashes published alongside the reserve-pool hashes for replacement instances), and all variants will be evaluated against the same corpus under seed-controlled replication. Each of the five benchmarks will contribute a pre-specified set of fault instances drawn from the fault categories documented in the pre-registration appendix; no benchmark result will be aggregated across benchmarks without explicit per-benchmark reporting alongside.

A secondary longitudinal element exists within the ORAR routing mechanism. The router will emit a provisional reward r□¹□ at approximately one to five minutes after actuation and a terminal reward r□²□ when the post-mortem label arrives (hours to days later). The Reward Reconciliation Error (RRE) metric is defined as the discrepancy between the sign of the provisional reward and the sign of the reconciled terminal reward. Because the benchmark corpus does not produce naturally delayed labels (ground-truth root-cause is available at the chaos-mesh injection time, so provisional and terminal rewards collapse to the same instant), and because the LinUCB parameters will be frozen for the confirmatory protocol under the static-policy commitment (Section 3.3.3), neither the confirmatory RRE analysis nor the on-line ORAR learning dynamics will be evaluated on the confirmatory corpus. The RRE measurement gap will be addressed through synthetic delayed-label injection on a pre-registered 25 percent subset of AIOpsLab and Train-Ticket

incidents, parameterized as a log-normal distribution with $\mu_{log} = 8.7$ (in seconds) and $\sigma_{log} = 1.4$, yielding a median delay of approximately 1.7 hours, a 25th-percentile delay of 38 minutes, a 75th-percentile delay of 4.6 hours, and a 95th-percentile delay of approximately 19 hours. This parameterization reflects the 'hours to days' production-incident reality that the RRE metric is designed to characterize and is grounded in the post-mortem arrival distributions reported in (Pham et al., 2024a) and in the cross-platform retrospective summarized in (T. Wang & Qi, 2024). ORAR on this 25 percent subset will operate with thawed LinUCB updates and will be reported as a disjoint exploratory analysis; the RRE metric and the on-line learning behavior will be reported only on these exploratory subsets and will not constitute confirmatory tests.

3.3.6 Variables and Constructs

The research operationalizes six primary constructs corresponding to Contributions C1 through C6, each mapped to an architectural mechanism of HELIOS and each measured through pre-specified metrics. The constructs, their architectural source, their independent-variable and dependent-variable operationalization, and their measurement instrumentation are summarized in following Table.

The construct-to-component binding is itself an instance of Contribution C1: each independent variable corresponds to a feature flag (or, for C1, to an architectural commitment) and each dependent variable is recorded at a specific architectural location, so the operational definition of every construct is fixed at runtime by the architecture rather than at analysis time by the researcher. This architectural fixing is a reliability commitment — it eliminates the researcher-degrees-of-freedom risk that the same nominal construct could be measured at two different architectural locations producing two different operational definitions — and is in this respect a positive

contribution to the chapter's methodological discipline. The chapter does not, however, claim that architectural fixing enforces construct validity in the stronger sense of guaranteeing alignment between the operational measurement and the theoretical construct it is intended to reflect. The architectural binding strengthens the reliability of construct measurement (every measurement of construct X will reflect the same operationalization across runs and variants) without claiming to strengthen the validity of that operationalization as a reflection of the underlying theoretical construct.

Table 13
Constructs, Variables, and Operationalization in HELIOS Evaluation

Construct	Architectural Source	Independent Variable	Dependent Variable	Measurement Location
Runtime-enforced evaluation rigor	C1 — Variant Control Layer + snapshot hash registry + ledger apparatus	Architectural commitment; not feature-flag-toggled	Variant-identity verification rate; snapshot-consumption verification rate; metric-integrity-gate completion rate; deviation-log entry count	Evaluation plane; runtime hash-verification logs

Construct	Architectural Source	Independent Variable	Dependent Variable	Measurement Location
Cross-domain evidence integration	C2 — UEG-C with cognitive nodes	`p4_cognitive` flag (L-pipe scope)	HR@3 on agentic-AI fault classes	Plane 4 MAHC ranked output
Shadow validation fidelity	C3′ — FGSV with KS gate	`fgsv_ks_gate` flag	False-accept rate of invalid actions	L4 FGSV verdict record
Uncertainty-aware consensus	C4 — MAHC with hierarchical priors	`mahc_hierarchical` flag	HR@3; consensus-confusion rate	Plane 4 MAHC ledger
Meta-observability	C5 — IOL self-telemetry	`iol_self_telemetry` flag	Degradation detection latency	L1 `tenant=self` partition
Cost-aware adaptive routing	C6 — ORAR (Contextual Bandit Router (CBR) + ACP + reconciliation)	`cbr_routing`, `acp_stateful`, `reconcile_terminal`, `his_tier` flags	CpR at matched HR@3; RRE; A1 latency per HIS tier	ORAR arm-selection log; reconciliation ledger

Note. HR@3 = hit rate at 3; CpR = cost per resolution; RRE = Reward Reconciliation Error; HIS = hardware-isolation tier; ACP = Action-Class Predictor. With the exception of C1 (which is an architectural commitment to evaluation-rigor infrastructure rather than a feature-flag-toggled mechanism), each independent variable is a feature flag toggled at the Variant Control Layer. Each

dependent variable is recorded at a specific architectural location and cannot be measured elsewhere without violating the integrity of the paired observation.

A specific instance of the architectural-fixing distinction warrants explicit treatment. HR@3 will be measured at the MAHC ranked-output module in HELIOS-Full. In HELIOS-noConsensus, MAHC will be ablated and replaced by a vote-only aggregator that emits a structurally identical ranked-output tuple at the same Plane 4 architectural location under the same content-addressable schema. The measurement location is therefore architecturally identical across HELIOS-Full and HELIOS-noConsensus, but the mechanism producing the measurement differs (uncertainty-aware hierarchical consensus in Full; scalar-vote aggregation in noConsensus). The chapter accepts that this is a single-construct operationalization rather than two distinct constructs, and that A-H4 (no-consensus degrades HR@3 on ambiguous incidents) is a comparison of the same construct under two consensus mechanisms. The construct-validity asymmetry that this resolution does not dissolve — that the noConsensus measurement reflects a different aggregation mechanism even though it occurs at the same architectural location — is acknowledged in Section 3.9.2 and is treated there as a residual threat that the architectural-fixing argument bounds but does not eliminate.

Beyond the five core constructs, three secondary constructs will be measured descriptively but will not enter the confirmatory hypothesis set: hallucination rate, decomposed into factual and structural components (H_fact and H_struct) with independent-rater labeling against pre-specified rubrics; time-to-triage (TTT), measured as the interval between the anomaly sentinel event timestamp and the publication of the MAHC provisional verdict; and SRE interpretation quality, to be measured in the exploratory HELIOS-noExpl variant for the user study accompanying Research Question 4.

3.4 Research Setting and Context

3.4.1 The Architectural Setting

The research is situated in cloud-native microservice environments orchestrated on Kubernetes and instrumented with OpenTelemetry (OTEL)-compliant observability tooling. The HELIOS artifact will enforce a strict physical separation between the Data Plane, which hosts production microservices, serverless functions, and agentic AI applications, and the Control Plane, which hosts HELIOS itself. The Control Plane will execute on a dedicated Kubernetes node pool tainted against Data Plane workloads, with ResourceQuota caps of eight CPU cores and 32 GiB of memory per incident, and OTEL collectors will be deployed as DaemonSets on Data Plane nodes with CPU limits of 100 millicores per node, ensuring that observability instrumentation cannot directly exacerbate resource-exhaustion incidents. HELIOS will be organized as a five-layer execution hierarchy (L0 telemetry ingress; L1 UEG-C construction; L2 peer reasoning pipelines; L3 coordination and consensus; L4 action and learning) orthogonal to five governance planes (Data, Reasoning, Coordination, Learning, Evaluation), with the Evaluation plane spanning all layers; the per-component specification is given in Section 3.6.3. The architectural setting is a methodological precondition for causal identifiability of each architectural contribution under the bounded scope conditions of the study: peer-pipeline co-deployment on identical inputs is the minimum structural commitment that turns the question "what does LLM reasoning contribute to RCA?" into a tractable question, because HELIOS-noLLM will leave the D-pipe and G-pipe as a functioning residual system against which HELIOS-Full can be compared on identical inputs.

3.4.2 The Business and Operational Context: High-Stakes Reliability

The operational context is characterized by the economic and reputational consequences of service degradation at cloud scale, with industrial incident reports documenting outage costs reaching tens of millions of dollars per hour in high-traffic financial-sector and e-commerce deployments (Pham et al., 2024a). The research addresses environments in which service-level objective (SLO) breach penalties are contractually specified, multiple tenants with heterogeneous service-class requirements share cluster infrastructure, and the cost of large language model inference for AI-driven diagnosis has itself become a material operational expense that must be architecturally managed rather than treated as a parameter to be minimized post hoc. The context shapes the research design in three ways: economic proportionality is operationalized through the Cost per Resolution (CpR) metric (Hypothesis A-H5 of the ablation family and Hypothesis B-H7 of the baseline-comparison family) as a technical proxy for organizational outcomes (on-call burden, customer-facing recovery, cloud spend) grounded in industrial retrospectives; safety is operationalized through the FGSV shadow validation mechanism and the Human-in-the-Loop (HITL) gate stratified by dynamic autonomic threshold α; and adversarial robustness is operationalized through the Adversarial Robustness Monitor that inspects incoming telemetry for prompt-injection payloads and verifies agent integrity via periodic attestation challenges. These three operational emphases are architectural commitments required by the high-stakes context and will be ablatable through the FGSV, HITL, and Adversarial Robustness Monitor (ARM) feature flags respectively (Section 3.6.7). The proxy nature of the technical metrics is acknowledged as an external-validity boundary in Section 3.11.2.

3.4.3 The Data Context: The Three Pillars of Observability

The research is grounded in the three canonical observability pillars established in cloud-native practice: metrics (P1, collected via Prometheus exporters and OTEL counters), distributed traces (P2, via Jaeger and OTEL with standard semantic conventions), and logs (P3, parsed via Drain3 template extraction (Duan, Xue, et al., 2024) with LogEDL evidential deep learning for vacuity estimation). HELIOS extends this canonical framework with two additional pillars specified as first-class signals rather than optional supplementary data: cognitive telemetry (P4 — agent reasoning-step spans under the Model Context Protocol convention, tool-call schema-validation records, goal-drift embedding cosine-distance measurements, and loop-depth counters) and business telemetry (P5 — transaction-level revenue signals, SLO breach penalty coefficients, and service-class tier assignments).

HELIOS will consume all five pillars through a single UEG-C representation with four typed edge classes (structural, behavioral, causal, economic) and formal composition rules, such that cross-domain path queries — causal composed with economic, behavioral composed with structural — will be first-class operations on the graph. This integration distinguishes HELIOS from prior single-pillar systems (Liu, 2021; Pham et al., 2024c) and from prior multi-modal systems that maintain pillar-specific pipelines emitting separate verdicts (Yu et al., 2023; Z. Zhao et al., 2025; Zheng et al., 2024). The five-pillar extension is presented as a conceptual-architectural contribution rather than a strict ontological claim that P4 and P5 must be separate pillars; the substantive content is the integrated UEG-C treatment with formal composition rules across the five signal classes. Whether the integration is the source of HELIOS's empirical performance advan-

tage, or whether the advantage is carried by the pipeline architecture alone, is the question that the HELIOS-noStructural ablation is designed to probe under hypothesis A-H8 (Section 3.6.7).

3.4.4 The Technological Setting: The Emergence of AIOps

The research is situated at the intersection of three technological trajectories: the maturation of Kubernetes as the de facto microservice orchestration standard, the proliferation of agentic AI systems built on large language models, and the emergence of AIOps as a research and industrial discipline. The AIOps landscape surveyed in Chapter Two encompasses a growing body of academic systems for microservice RCA and a complementary body of commercial platforms providing anomaly detection enriched with varying degrees of LLM-assisted explanation. A systematic gap persists across the surveyed landscape: no existing system simultaneously satisfies five methodological desiderata necessary for causally identifiable evaluation of AI-driven RCA:

- **D1:** Peer-pipeline co-deployment of statistical, graph-ML, and LLM reasoning on identical inputs.
- **D2:** First-class observability for LLM agent reasoning failures alongside infrastructure faults.
- **D3:** Economic arbitration of diagnostic effort through cost-aware routing grounded in observed accuracy priors.
- **D4:** Deferred two-stage reward reconciliation that accounts for the asynchronous arrival of post-mortem labels.
- **D5:** Ablation-first architectural discipline that produces variant equivalence at the binary level rather than through retrained model weights.

Table 14
Comparative Analysis

System	D1 Peer pipelines on one input	D2 Own reasoning observed	D3 Cost routed by accuracy	D4 Two-stage reward	D5 Same-build ablation
CHASE	No	No	No	No	No
mABC	Partial	No	No	No	No
TAMO	No	No	No	No	No
Flow-of-Action	No	No	No	No	No
FrugalGPT	No	No	No	No	No
HELIOS	Proposed	Proposed	Proposed	Proposed	Proposed

The desiderata are derived from examination of methodological gaps in the comparator corpus rather than from the HELIOS specification itself; HELIOS is one possible artifact that will satisfy all five, with D1 corresponding to Contribution C1 (peer-pipeline integration enforced by C1's runtime apparatus), D2 to C2, D3 and D4 to C6, and D5 to C1's ablation-first architectural discipline.

The C5 (meta-observability) contribution further differentiates HELIOS from concrete control-plane self-monitoring patterns — Prometheus self-scrape, OpenTelemetry self-instrumentation, and the meta-monitoring features of commercial observability platforms such as Datadog and New Relic — through the integrated-graph treatment of self-telemetry as a tenant=self partition in the same UEG-C that the production-diagnostic pipeline consumes

(Section 3.6.8). Because each comparator baseline satisfies some but not all of the five desiderata, cross-comparator generalizations will be interpretive rather than strictly inferential; the analysis plan in Section 3.8 therefore treats the baseline-comparison family as a separate hypothesis family from the ablation family, corrected separately under Holm–Bonferroni to avoid combining incommensurable inference targets into a single family-wise error budget.

3.5 Business Evaluation Layer

A DBA dissertation that stops at hit-rate and latency has not finished the methods argument. HELIOS is meant to sit in a production estate after the evaluation, and the organization that would pay for it does not buy HR@3. It buys shorter incidents, cheaper resolution, fewer SLO credits written, and diagnoses that people will actually act on. This section is the methods account of that translation. It is not a new confirmatory family. The laboratory still measures the technical variables in Section 3.3.6 and Section 3.8.4. What follows is how those variables are read as business outcomes, what the architecture already records for that reading, and what remains a post-deployment claim rather than a result of the 16,000-run protocol.

The mapping is one-way and lossy, which is worth saying before the table. A drop in log-MTTR on a chaos-injected fault is not the same thing as recovered revenue on a shopping day. CpR is assembled from token spend, infrastructure, storage, human minutes, and opportunity cost at the Critic Agent; it is a cost object, not a P&L line. The chapter already treats these metrics as proxies (Section 3.4.2). The business layer does not pretend otherwise. It specifies which proxy stands for which organizational effect, so that Chapter Four can report the technical tests without leaving the business reading implicit, and so that a later industrial deployment has a documented path from those tests to value.

Table 15
Technical outcomes to business outcomes after deployment

What HELIOS measures	Hypothesis / instrument	Business reading after deployment	Who sees it
HR@3, Avg@5	A-H1–A-H4, A-H6, A-H8; B-H1–B-H5, B-H8	Fewer wrong first diagnoses; less time chasing the wrong service; fewer repeat incidents from a missed root cause	SRE lead; incident commander
log-MTTR	B-H6	Shorter customer-visible impact window; less SLO burn; lower contractual service-credit exposure	Service owner; finance
CpR / log-Cost at matched accuracy	A-H5; B-H7	Lower cost per closed incident, including LLM inference, so diagnosis does not become a second opex problem	Platform owner; FinOps

What HELIOS measures	Hypothesis / instrument	Business reading after deployment	Who sees it
Time-to-triage (TTT)	E-H2 (exploratory)	Faster hand-off from alert to a named cause; less on-call queueing	On-call rotation
Correct interpretation, trust, accept / escalate / reject	E-H7 user study	Decision effectiveness: whether the output is usable enough that a person will act, escalate, or stop a bad remediation	SRE; service owner; academic reviewer of the design
Hallucination rates (H_fact, H_struct)	A-H7	Residual risk of a fluent but wrong narrative driving a bad change	Risk / change-advisory
FGSV false-accept; HITL on high-risk actions	E-H4; Monitor Agent	Blast-radius control; the organization is not buying unsupervised writes to production	Engineering leadership

Four of those rows are the ones DDR is asking for, in ordinary language.

Operational efficiency is MTTR and TTT. If HELIOS-Full is faster to a usable diagnosis than the best baseline (B-H6) and faster to a provisional verdict than uniform routing (E-H2), the

operational story after deployment is fewer engineer-hours per incident and a shorter wait before someone can start a fix. Published deployments in this literature have claimed diagnosis moving from hours to minutes (Chakraborty et al., 2024; Guntupalli, 2025). Those figures are not HELIOS results. They are the class of benefit the MTTR and TTT instruments are built to speak to, once the system is on a live estate.

Incident-resolution cost is CpR. A-H5 asks whether turning ORAR off makes that cost worse at the same accuracy; B-H7 asks whether HELIOS-Full is cheaper than the best baseline. After deployment the same object is what FinOps would recognize: tokens, cluster time, storage, and the human minutes still spent on the ticket. The design commitment in Chapter Two was that generative RCA can be accurate and still too expensive to run on every alert. ORAR exists because of that. The business layer therefore treats a CpR reduction at matched HR@3 as the organizational value of C6, not as a side-effect.

Service availability and impact are not measured as uptime percentages in the confirmatory corpus, because the corpus is injected faults with a known start, not a year of production traffic. The available proxy is the duration of the diagnostic and recovery path (log-MTTR) together with HR@3 (whether the recovery path is aimed at the right component). P5 business telemetry — revenue signals, SLO penalty coefficients, service-class tiers — and the economic edge type in UEG-C are how HELIOS would consume those signals if they exist on a live estate. In the laboratory they are specified so that a later deployment can attach a dollar figure to an SLO burn without changing the diagnostic core. Recovered-revenue numbers will not be reported from chaos-mesh runs.

Decision effectiveness is the RQ4 study and the HITL log. Accuracy on a ranked list is necessary and not sufficient. If the Chain-of-Evidence form is no easier to read than the raw list, the organization has bought a faster wrong conversation. E-H7 is exploratory and n = 24 is mixed

(practitioners, academics, business participants) for that reason: after deployment the output is not only read by the on-call engineer. A service owner who has to approve a rollback, or a reviewer who has to believe the write-up, is part of the value path. The study does not estimate adoption in the field. It asks whether the narrative is usable enough to change interpretation, trust, and the accept / escalate / reject choice.

3.5.1 Managerial application

The table above names who reads each number. The rest of this subsection says what they are deciding. The incident process keeps its ordinary shape: detect, diagnose, decide, remediate, learn. The architecture in Section 3.6.3 is that sequence drawn as boxes.

Detection is L0. Intake takes the telemetry the estate already emits — metrics, traces, and logs — and puts two further signals on the same incident: what the firm's own agents were doing, and the business weight of the service (revenue, SLO penalty, service tier). Which services are in that intake is settled by the platform owner and the service owners. HELIOS does not draw the estate boundary for them.

Diagnosis holds the two decisions that change the cost of the incident. The ranked list is which service to open first. A wrong first guess is paid for by the SRE lead and the incident commander, which is why they sit on the HR@3 and Avg@5 row. The router is the other decision: whether this alert earns a language-model call, or whether the cheaper pipelines are enough. The platform owner and FinOps already know that object as cost per resolution. A-H5 and B-H7 are the tests. Until they are run, the managerial claim is only that the cost is written down in parts those roles recognize: tokens, cluster time, storage, human minutes, and opportunity cost.

The decide step is the evidence trail out of L3. A ranked list that nobody will sign does not shorten the incident. It shortens the argument and then leaves the argument where it was. E-H7 asks whether a person can read the trail and accept it, escalate it, or reject it. After deployment that person is the on-call engineer, and also the service owner who has to allow a rollback, and the reviewer who has to stand behind the write-up. They are already on the decision-effectiveness row.

Remediation is L4, and it is fenced. The shadow test runs before a production write. A database change, a network-policy change, or a rollback waits for a named person. Engineering leadership's stake in that row is the blast radius: the firm is paying for a diagnosis it can refuse. Learning is the outcome record on the same layer. In the confirmatory protocol that record does not move the router's parameters. The update path is there for the exploratory delayed-label subset and for a later deployment. Chapter 4 will not report it as an effect of the 16,000 runs.

3.5.2 Organizational adoption

Whether a firm adopts HELIOS is not a result this study will estimate. Chapter 1 already lists culture and management among the limits on that outcome. H3b is smaller than a deployment claim. If trust in the output rises, stated willingness to accept the diagnosis inside the user study may rise with it. That is a question about the exercise.

What can be specified now is the order in which the controls would be opened. (Rogers, 2003) is the reference for the plain conditions under which an operations team tries a tool: the gain has to be visible, the tool has to fit what the team already runs, and the team has to be able to try it without putting a production write at risk. The three stages below are those conditions, assembled from controls the architecture already has.

Shadow is the trial. HELIOS writes the diagnosis and the cost per resolution, and stops. The human gate stays shut. Nothing is handed to production. A platform team can put the ranked cause, the time, and the cost next to what the on-call engineer actually did.

Recommendation is the mode the study can speak about. Every action waits for a person, and the evidence trail is what they read. E-H7 is this mode. Twenty-four participants — practitioners, academics, and business readers — can show whether the narrative changes interpretation, trust, and the accept / escalate / reject choice. They cannot show that an organization will keep the tool. Assumption A3 already says the same thing in operational terms: findings should not be carried to a firm that has not had a 30-day period of human calibration. That month is this stage. Assumption A5 sits beside it. With a thin incident history, SOP matching stays weak for about 90 days, and early recommendations should be read with that in view.

Automatic action is the firm's option, and it is optional. The firm sets the risk threshold α. Actions under the line may run without a person in the loop. A rollback, a data change, or a network-policy change still waits, and the shadow copy still has to agree with live telemetry before a write. HELIOS does not arrive with α filled in.

The proxies remain proxies. (Kohli & Grover, 2008; Melville et al., 2004) treat an IT measure as a stand-in for an organizational effect, with the effect itself still unobserved on the firm's books. A lower log-MTTR on a chaos-injected fault is still not recovered revenue. Putting a rollout order around the artifact does not change that.

3.5.3 Governance

The five planes in Section 3.6 — Data, Reasoning, Coordination, Learning, and Evaluation — place components. They do not say who is allowed to let an action through. That second question belongs to the firm, and the third stage above is empty until it is answered.

Table 16
Decision rights for a HELIOS deployment

Rule	Who decides	What the artefact already records
Which alerts are in scope	Platform owner, with the service owners	Sampling tier and incident signature
When the language model may be called	FinOps with the SRE lead, through the cost target	Cost per resolution, and the router's choice
Which actions may run unattended	The firm, by setting α	Action class and the threshold
Who approves a high-risk change	A named person, for a rollback, a data change, or a network-policy change	Human gate and the Chain-of-Evidence
When a fix may touch production	The change owner, after the shadow copy agrees with live telemetry	Shadow-validation verdict
What personal or tenant data may reach the model	The data owner	Privacy gateway and tenant separation

Rule	Who decides	What the artefact already records
How a fluent but wrong diagnosis is caught	Risk reviewer	Factual and structural hallucination checks, and the ethics entry on the evidence ledger
Who reviews overrides and spend	SRE lead, platform owner, and risk, on the firm's existing incident cadence	Override, exclusion, and cost records

The person or function that sets α owns the decision that some actions may run unattended. The Chain-of-Evidence is how that decision is reconstructed: which threshold was in force, who was asked, what the shadow test returned, and what was written. An audit trail supports reconstruction. It does not settle legal liability. Who answers for a bad automatic change is outside this study.

Privacy, consent, and the requirement that a high-risk action wait for a person remain in the ethics section. That section is about the research. The table here is about the firm that would run the artefact afterwards. Overrides, exclusions, and cost per resolution are already on the ledger for the SRE lead, the platform owner, and risk to read. The study does not convene that review.

What this layer will and will not claim in Chapter Four follows from the above. Confirmatory tests may support (or fail to support) lower log-MTTR, lower log-Cost, and higher HR@3. Those results, if obtained, license a bounded business reading: under the corpus and the frozen protocol, the artifact is cheaper and/or faster and/or more accurate than the named comparators, which

is the usual mechanism by which RCA tooling earns its keep. They do not license a claim that a named firm will save a named sum, that availability will rise by x nines, or that SRE toil will fall by the 41 percent reported in other settings (Hwang et al., 2021). Those claims need production traffic, a price book for SLO credits, and a time-on-ticket measure that the present instruments only approximate. The adoption stages and the decision-rights table specify how a firm would open the controls and who would set the thresholds. They are not evidence that any firm will. An industrial replication, if one is engaged, is where that last step belongs (Section 3.11.5). Until then the business layer is a documented translation, not a second results chapter.

3.6 Population and Sampling

The target population for the quantitative evaluation is the set of fault scenarios representable in cloud-native microservice environments as exercised through chaos-engineering fault injection and workload-driven degradation. The confirmatory population is operationalized through AIOpsLab (Y. Chen et al., 2025) as the primary benchmark, with cross-benchmark probes drawn from DeathStarBench (Gan et al., 2021), Train-Ticket, PetShop (Hardt et al., 2024), and OpsEval (Liu et al., 2025) supporting external-validity assessment but not confirmatory cross-benchmark inference (see Section 3.9.3). The five benchmarks are selected because they provide publicly available, independently replicable fault dataset with documented annotation protocols (a precondition for reproducibility under the DSR framework), they span heterogeneous microservice architectures (hotel-reservation, social-network, train-ticket, e-commerce, and IT-operations task suites) supporting external validity across application domains, and they exhibit different fault-injection regimes (Chaos Mesh, ChaosBlade, functional fault injection) that prevent the evaluation from being over-fit to the failure mode of a single injection tool.

The sampling strategy combines stratified and purposive elements under the binding constraint of the a priori power analysis. Each benchmark will contribute 40 pre-specified fault instances drawn from the fault categories documented in the pre-registration appendix (resource exhaustion, network partition, service degradation, dependency failure, database contention, and benchmark-specific fault classes). With ten random seeds per fault instance, each variant–benchmark cell will produce 400 paired observations, for a full ablation protocol of eight variants × five benchmarks × 40 faults × 10 seeds = 16,000 incident runs. The 16,000-run ablation protocol will be paired with a 174-fault delivered corpus for the baseline comparison, sized to 2.1× the 84-fault statistical minimum determined by power analysis (Section 3.8.3).

The 174-fault corpus will be constructed across four augmentation tiers to reconcile the conflicting demands of statistical power, external validity, and cognitive-fault coverage. Tier 1 will comprise 96 paired faults drawn from two AIOpsLab replicas under distinct random seeds, integrated in analysis through a generalized linear mixed-effects model (GLMM) with ‘problem_id‘ as a random intercept to prevent pseudoreplication inflation of degrees of freedom. Tier 2 will comprise 36 paired faults from a Chaos-Mesh parametric extension of AIOpsLab (9 fault types × 3 severity levels × 2 concurrency levels in a balanced factorial). Tier 3 will comprise 30 paired faults from Train-Ticket and DeathStarBench under ChaosBlade injection, contributing cross-benchmark external validity. Tier 4 will comprise 12 paired cognitive-agentic synthetic faults constructed through NIKA-style structured Model Context Protocol instrumentation (Z. Wang et al., 2025), comprising six hallucination-class faults (invalid tool schema, non-existent API invocation, argument fabrication, RAG-contamination, self-contradiction, context-leakage) and six goal-drift-class faults (task-objective decay, looping, premature termination, delegation misrouting, persona violation, ethics-bypass attempt). Tier 4 faults will be layered onto a subset of AIOpsLab incidents

through cognitive-agentic instrumentation rather than constructed on an independent synthetic substrate; the cognitive-node telemetry is injected into the AIOpsLab incident harness, and the resulting paired observations therefore inherit AIOpsLab's microservice topology, fault-injection regime, and labeling discipline.

The 174-fault corpus exhibits a substantial concentration in the AIOpsLab benchmark family: Tier 1 and Tier 2 together comprise 132 of 174 instances, or approximately 76 percent of the corpus. The methodological consequence is that confirmatory inferences will be bounded to the class of AIOpsLab and its parametric extension; cross-benchmark results in Tiers 3 and 4 will be reported as external-validity probes that suggest, but do not confirm, that the observed effects extend to environments outside the AIOpsLab specification (see Section 3.9.3). The rank-based mixed-effects treatment of problem_id as a random intercept will control for pseudoreplication within AIOpsLab but does not, on its own, repair the bound on the population to which findings can be generalized.

Tier 4 is not optional within the corpus — the C2 cognitive-agentic claim requires agentic-AI fault instances that no baseline system can diagnose — but two methodological consequences are explicitly accepted: the n = 12 sample size is below the 73-fault statistical minimum, and Tier 4 instances are constructed by the researcher specifically to exercise cognitive-node reasoning. These two consequences together motivate the reclassification of the C2 cognitive-agentic evaluation as exploratory empirical validation under E-H1 rather than confirmatory inference (Section 3.6.8 and Section 3.8.2). Tier 4 evidence supports an existence-proof claim — that HELIOS can diagnose cognitive-agentic faults that comparator baselines cannot — but does not support a population-level efficacy claim about cognitive-agentic fault diagnosis in general; an externally constructed

cognitive-agentic fault corpus on a non-AIOpsLab substrate is identified in Section 3.11.5 as the priority direction for upgrading E-H1 to confirmatory status.

Four statistical-validity safeguards will protect inferential validity under augmentation: ‘benchmark_source‘ will be included as a fixed effect in the GLMM to prevent between-source heterogeneity from inflating Type I error; Tier 1 replicates will share ‘problem_id‘ as a random intercept to avoid pseudoreplication; the GLMM specification, corpus partition, and Holm–Bonferroni hypothesis ordering will be locked at pre-registration freeze and deposited at OSF; and results will be reported at ρ □ {0.4, 0.5, 0.6} to demonstrate robustness to the paired-correlation assumption embedded in the sample-size calculation.

The 174-fault baseline-comparison corpus is a subset of the 200-fault ablation corpus drawn from the AIOpsLab and Train-Ticket benchmarks (132 from Tier 1+2 AIOpsLab, 30 from Tier 3 Train-Ticket and DeathStarBench, and 12 from Tier 4 cognitive-agentic on AIOpsLab); the remaining 26 ablation faults are PetShop and OpsEval instances used in the ablation protocol but not in baseline comparison because the baseline systems do not all support those benchmark substrates. The corpus relationship is documented in the OSF-deposited fault-instance manifest with full SHA-256 hashes for each instance’s membership in each protocol.

The pseudoreplication-control specification deserves explicit binding under the hypothesis schema. To preserve coherence with the non-parametric Wilcoxon primary tests (Section 3.8.3), pseudoreplication on Tier 1 (the 96 paired faults from two AIOpsLab replicas under distinct seeds) will be controlled via a rank-based mixed-effects analysis: the paired outcome differences will be rank-transformed within each (variant, benchmark, metric) cell prior to fitting a linear mixed model with problem_id as a random intercept and benchmark_source as a fixed effect. Variance components will be estimated by restricted maximum likelihood (REML); degrees-of-freedom ad-

justment will be Kenward–Roger; the residual covariance for the seed-paired observations within each problem_id will be unstructured. As a concurrent sensitivity analysis, the non-parametric repeated-measures procedure of (Noguchi et al., 2012) the nparLD package in R) will be applied to the same Tier 1 subset and reported alongside. Neither analysis is the binding confirmatory test for any hypothesis; the binding tests are specified in Section 3.8.3 (Wilcoxon signed-rank for the ablation family; McNemar's exact for B-H1 through B-H5; Wilcoxon signed-rank for B-H6 through B-H8 with paired t on log-transformed values reported as sensitivity analysis). The pseudoreplication-control analyses verify that the Wilcoxon inference does not shift materially when the within-Tier-1 dependence structure is modeled explicitly; if the rank-based mixed model and the Wilcoxon results disagree on direction or significance for any A-Hx, both will be reported and the disagreement treated as a sensitivity-analysis flag rather than as confirmatory inference. The pseudoreplication-control specification will be locked at OSF freeze and any departure recorded in the deviation log.

The sampling is not intended to support population-level generalization to every microservice environment in industry. It is intended to support the narrower and more defensible claim that, within the class of AIOpsLab-representative microservice environments and across the four corpus tiers, HELIOS's architectural contributions will produce the hypothesized effects on the pre-specified metrics at the pre-specified effect sizes, and that cross-benchmark observations in Tiers 3 and 4 will be consistent with — but will not confirm — generalization beyond AIOpsLab and its parametric extension.

3.7 The AI Artifact Development (The Intervention)

3.7.1 Research Artifact Definition

The research artifact is HELIOS, the Heuristic Learning for Integrated Observability Systems framework, specified as a multi-agent, multi-modal, ablation-first RCA system for cloud-native microservice environments that co-exist with agentic AI applications. HELIOS's definitional scope is bounded by six explicit commitments: it will operate on OTEL-instrumented microservice estates of at most approximately 100 services within the benchmark regime; it will assume an enterprise-scale control-plane compute budget of 5 to 10 percent of cluster capacity; it will require seeded accuracy-calibration data from pre-production fault-injection campaigns; it will assume LLM-access tier 2 or higher (logprob-API access) for full P4 cognitive telemetry; it will assume the existence of historical incident records for the Knowledge Cortex warm tier; and it will treat each incident as independent, not modeling multi-incident cascade correlation.

HELIOS is not claimed as a general-purpose RCA solution; it is claimed as an experimental artifact engineered to support falsifiable hypothesis testing on six named contributions (C1 through C6) spanning four contribution types in the Gregor and Hevner (Gregor & Hevner, 2013) taxonomy: an instantiation (HELIOS itself as an integrated artifact), a methodological-discipline contribution (C1 — runtime-enforced evaluation rigor), constructs and models (C2 — UEG-C with cognitive nodes; aspects of C4), and methods (C3′, C4, C5, C6). Confirmatory or exploratory status for each contribution is determined by the corpus feasibility documented in Section 3.5 and Section 3.6.8. In the planned evaluation, HELIOS's absolute performance against baselines is of interest but is not the primary inferential target; the primary inferential target is the pattern of ablation effects, which will license the claim that each contribution individually provides the hypothesized mechanism under

the conditions evaluable on the available corpus. The ablation-first framing distinguishes HELIOS from comparator systems engineered for production deployment first and evaluated for research contribution second, and it will justify the depth of internal instrumentation (Variant Control Layer, snapshot hash registry, reconciliation ledger, exclusion ledger) that is architectural in HELIOS but absent from the comparator corpus.

3.7.2 Architectural Design Principles

Five design principles will govern the HELIOS architecture and, through it, the methodology of the study; the principles are intended as transferable design knowledge contributions in the (Gregor & Hevner, 2013) sense, generalizable beyond HELIOS to AIOps research-method discipline more broadly.

Pipeline independence. The three reasoning modalities — D-pipe, G-pipe, L-pipe — will be peer pipelines that consume the same evidence snapshot but share no runtime state; any subset can be disabled by feature flag without affecting the operation of the others. This is the minimum structural commitment that admits causal identifiability of each paradigm's contribution to RCA performance, subject to the caveat that the absence of cross-pipeline runtime state coupling does not preclude shared selection biases inherited from the common UEG-C input.

Ablation-first design. Every mechanism claimed as a contribution must be ablatable via a feature flag, and every flag must flip exactly one well-defined code path without composing with another flag as a side effect. The twelve feature flags specified in the architecture (‘l2c_llm‘, ‘p4_cognitive‘, ‘mahc_hierarchical‘, ‘cbr_routing‘, ‘l2b_graph‘, ‘acp_stateful‘,

‘reconcile_terminal‘, ‘fgsv_ks_gate‘, ‘iol_self_telemetry‘, ‘coe_narrative‘, ‘his_tier‘, ‘sampling_regime_tag‘) will produce eight confirmatory variants through pre-registered combinations and will be subject to a hidden-coupling audit verifying that each flag toggles disjoint code paths. The audit has been conducted at design freeze: it identified one hidden coupling in the original ‘p4_cognitive‘ specification (silent removal of cognitive nodes from G-pipe’s input), which has been resolved by restricting the flag to L-pipe scope, and confirmed that the remaining eleven flags toggle disjoint code paths.

Bounded reproducibility under explicitly disclosed determinism conditions. D-pipe and G-pipe will be reproducible to the level of bitwise output equality under the binding hardware configuration (NVIDIA A100 80GB SXM4; CUDA 12.1; PyTorch deterministic-algorithm flags enabled via ‘torch.use_deterministic_algorithms(True)‘ and ‘CUBLAS_WORKSPACE_CONFIG=:4096:8‘; cuDNN deterministic mode enabled). L-pipe reproducibility will be specified at two protocol levels rather than as bitwise. Under Protocol A (self-hosted open-weight model), the binding model is Llama-3.1-70B-Instruct served via vLLM 0.6.x under greedy decoding (temperature = 0, top-p = 1, top-k = 1) with ‘enforce_eager=True‘, ‘disable_custom_all_reduce=True‘, and single-replica serving; under these conditions the emitted token sequence will be reproducible across runs, but token-level logits will exhibit small floating-point variance ε due to non-deterministic CUDA attention-kernel selection and cuBLAS workspace allocation. ε will be measured during the Chapter Four pilot pass on a 200-prompt probe set and reported in Chapter Four with its standard error and measurement date; the measurement is conducted prior to chapter submission so that ε is a delivered empirical result at viva. Under Protocol B (commercial API access), neither token-sequence nor logit-level determinism is

achievable through documented means; Protocol B will be reported as a sensitivity analysis with inter-run variance characterized by the same probe-set methodology. Container digest, weights SHA-256, and tensor-parallel-degree configuration will be deposited at OSF and verified at runtime by the Variant Control Layer.

Separation of diagnosis from action. The decision layer will emit a ranked root-cause list and a proposed remediation plan; the action-and-learning layer will be the sole execution boundary. This separation allows diagnostic quality to be evaluated independently of action safety, which is methodologically necessary because the RCA accuracy hypotheses target the decision layer alone, whereas the safety and validation hypotheses target the action layer.

Failure-as-covariate discipline. Every failure mode — sampling downgrade, pipeline unavailability, consensus non-convergence, twin staleness, privacy-budget exhaustion, adversarial input, hardware contention — will be converted into a first-class observable covariate that stratifies the evaluation, not into a silent fallback. The principle is enforced by the metric integrity gate at the Variant Control Layer, which will refuse to accept a run whose covariates are not fully populated. The proportion of incidents with complete covariate records is itself a reported metric with a target of 100 percent.

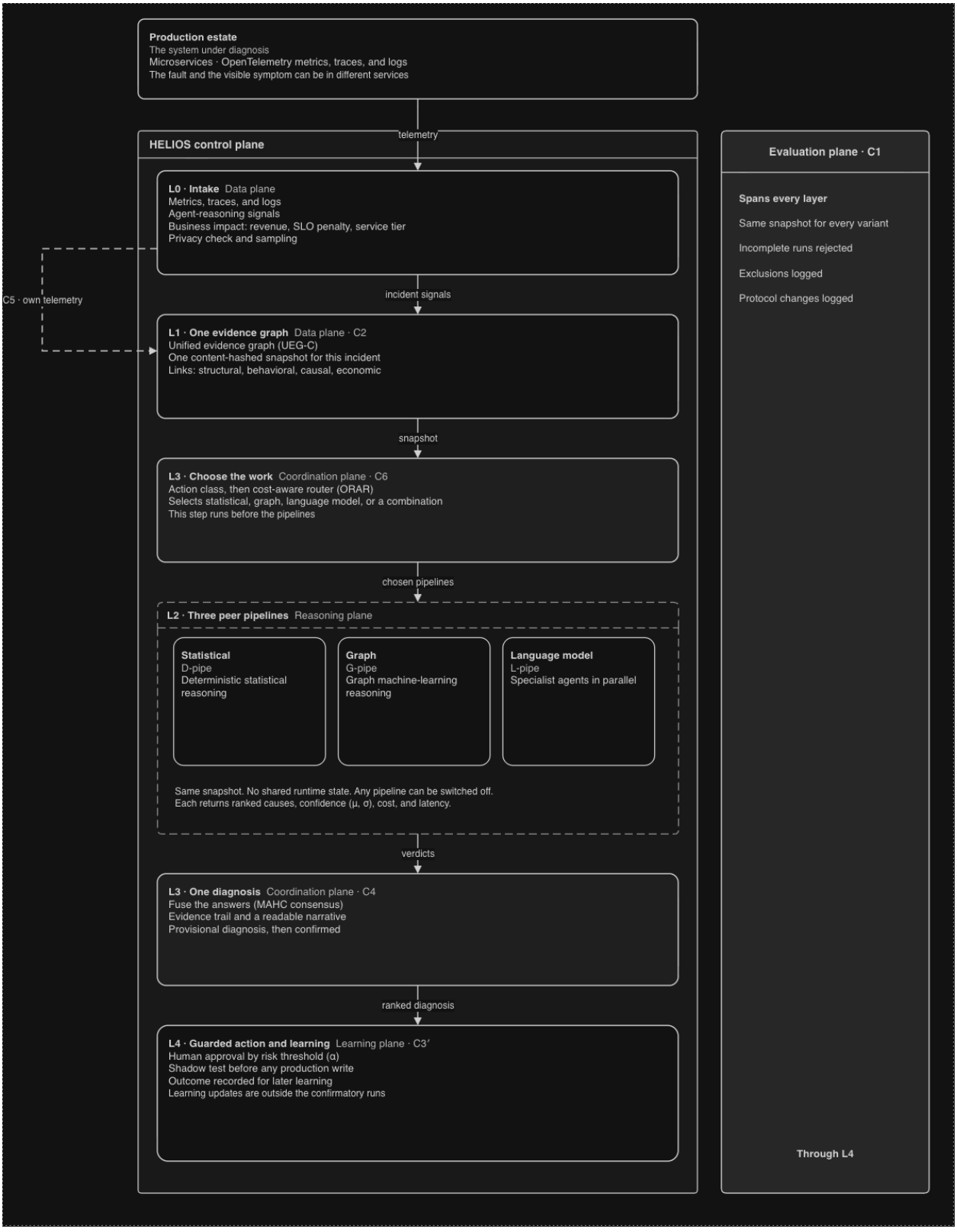
3.7.3 System Architecture Specification

The following describes the HELIOS architecture as specified at design freeze. Evaluation activities described in subsequent sections will exercise this architecture under the pre-registered

protocol; present-tense statements in this section therefore describe the design specification, not currently-running infrastructure. Figure 1 shows one incident moving through that hierarchy.

HELIOS will be organized as a five-layer execution hierarchy (L0–L4) orthogonal to five governance planes (Data, Reasoning, Coordination, Learning, Evaluation), with the Evaluation plane spanning all layers. Layer L0 will host telemetry ingress (OTEL collectors, the Privacy Gateway, the Adversarial Robustness Monitor, Tiered Adaptive Sampling, and the Internal Observability Loop, which writes HELIOS's own control-plane telemetry into the L1 graph under a tenant=self partition). Layer L1 will construct and maintain the UEG-C hypergraph, with the K-hop Pruner extracting incident-local subgraphs through personalized PageRank with Pearson correlation pruning. Layer L2 will host the three peer reasoning pipelines, each consuming the same content-hashed UEG-C snapshot and emitting a typed verdict of the form (ranked_RCA_list, μ, σ, cost, latency). Layer L3 will execute coordination: the Action-Class Predictor will emit a stateful-action feature block that the ORAR router consumes; the ORAR router will select the pipeline subset subject to cost minimization under accuracy and latency constraints; the MAHC consensus will aggregate the pipeline verdicts under hierarchical priors with (μ, σ) vote representation; the Chain-of-Evidence ledger will record the decision audit trail; and the Narrative Engine will render the CoE into human-readable form. Layer L4 will execute the action-and-learning cycle: the HITL gate will stratify by dynamic autonomic threshold α; the FGSV shadow validator will execute the proposed remediation in a Kolmogorov–Smirnov-gated digital twin; the SEAN library will provide versioned remediation templates; and the reconciliation ledger will receive provisional and terminal rewards and apply compensating updates to the ORAR parameters during exploratory subsets only.

Figure 1
HELIOS Architecture for One Incident



Each component is specified by its inputs, outputs, and ablation controllability. The OTEL collectors and Privacy Gateway are supporting infrastructure rather than ablatable contributions. The UEG-C graph store (C2) is ablated at the L-pipe-visibility level by ‘p4_cognitive‘ and at the structural level by ‘ueg_structural‘. The three L2 pipelines are ablated individually by ‘l2c_llm‘, ‘l2b_graph‘, and the implicit D-pipe-only configuration. The ACP is ablated by ‘acp_stateful‘; the CBR by ‘cbr_routing=uniform‘ (with auxiliary ablations for ‘reconcile_terminal‘ and ‘his_tier‘); MAHC (C4) by ‘mahc_hierarchical‘; FGSV (C3′) by ‘fgsv_ks_gate‘; and IOL (C5) by ‘iol_self_telemetry‘.

The L-pipe multi-agent system warrants specific attention because its architectural differentiation from prior multi-agent RCA systems is central to the contributions claimed in this dissertation. The L-pipe is organized around a Reflective Orchestrator that loads matching Standard Operating Procedures from the Knowledge Cortex, evaluates the Adaptive Specialization Gate decision function, and dispatches worker agents with Persona-Profile-bound tool permissions. The worker mesh comprises specialist agents for metric-domain anomalies, trace-domain propagation analysis, log-domain evidential reasoning, cognitive-telemetry sentinel monitoring, value-impact assessment, ethics auditing, and reward computation. This organization differs from mABC (W. Zhang et al., 2024), in which agents vote with scalar contribution × expertise weights, by emitting (μ, σ) tuples that propagate uncertainty into the MAHC consensus rule; from TAMO (Zhang et al., 2025), in which an expert agent synthesizes tool outputs into a single narrative, by treating agents as peer voters rather than as a single synthesizer; and from Flow-of-Action (Pei et al., 2025), in which a MainAgent orchestrates sub-agents sequentially, by operating specialists in parallel with independent evidence streams entering the MAHC consensus.

3.7.4 Data Flow and Control Flow

Data flow and control flow are deliberately separated to facilitate causal identifiability at each stage of the incident lifecycle. The data flow will commence with signal emergence at L0, where five-pillar telemetry is continuously collected at the Tier A baseline rate of 0.1 percent via eBPF kernel probes. Anomaly detection at the D-pipe will trigger sampling escalation to Tier C within the affected K-hop neighborhood, with backpressure-aware downgrade to Tier B on resource pressure (Section 3.7). The K-hop Pruner will extract a subgraph of 50 to 200 nodes with four typed edge classes, content-hash the snapshot, and publish the snapshot identity to the registry for downstream consumers.

Control flow, which determines which pipelines execute on the snapshot, will be routed through the ACP and the ORAR CBR. The ACP will read the snapshot and the incident signature and emit a stateful-action feature block including schema-change probability, data-mutation probability, config-change probability, a log-scale write-ahead-log (WAL) size estimate, and a blast-radius tier classification. The ORAR router will combine incident features, per-pipeline accuracy priors, and the ACP output into a constrained-bandit decision over the action set {D, G, L, D+G, D+L, G+L, Full}, minimizing expected cost subject to an accuracy-probability constraint at target HR@3 and a latency constraint. During the confirmatory protocol, the LinUCB per-arm parameters are frozen at the pre-warmed state (Section 3.3.3 and Section 3.6.5), so the routing decision is a deterministic function of incident features and frozen parameters, and the sequence in which incidents are processed does not influence subsequent routing. If the constraint set is empty, the router will fall back to HELIOS-Full with an explicit `constraint-empty-fallback` covariate logged to the CoE.

Once the pipeline subset is dispatched, the pipelines will execute concurrently and each will produce a typed verdict. The MAHC consensus at L3 will aggregate the verdicts under hierarchical priors (fault-type → service-family → global, with backoff triggered by minimum-sample thresholds) and emit a Fast-Path PROVISIONAL verdict within 30 seconds and a CONFIRMED verdict upon full consensus convergence; non-convergence within 90 seconds will trigger elevated-uncertainty PROVISIONAL publication plus HITL escalation, and the non-convergence count will be a reported metric per cell rather than an evaluation failure.

Action flow will proceed through the FGSV at L4, which will test the proposed remediation in the shadow twin only when the Kolmogorov–Smirnov two-sample test on pillar-matched samples confirms that twin telemetry is statistically close to production. A KS-gate failure will yield a STALE verdict, which is itself a first-class covariate; incidents with `shadow_suspended=true` will be reported separately and no action-safety claims will be made for them. Actions that pass FGSV and clear the HITL gate during evaluation runs will proceed to actuation; the actuation result will feed the provisional-reward computation approximately one to five minutes later, and post-mortem labels arriving hours to days later will drive the terminal-reward computation, the compensating ORAR parameter update, the RRE logging, and the weekly bias-audit recalibration. These on-line update operations will be performed only on the disjoint exploratory delayed-label subset described in Section 3.3.5 and will not modify the frozen LinUCB state used in the confirmatory protocol.

3.7.5 Formal Algorithms

Four formal algorithms are named as core intellectual content of HELIOS, with one supporting algorithm (the K-hop Pruner). Each is summarized below at a level sufficient for methodological grounding; full pseudocode is documented in the architecture chapter (Chapter Four). Al-

gorithm specifications use present tense in the sense of computer-science algorithm description (algorithms describe a temporal computations); evaluation activities that exercise the algorithms during the confirmatory protocol are stated in future tense.

Algorithm 1. ORAR Router. Solves a cost-minimizing constrained bandit problem at each incident. Given incident features x (augmented with the ACP stateful-action feature block), per-arm LinUCB parameters (A_a, b_a) for a □ {D, G, L, D+G, D+L, G+L, Full}, and thresholds (τ_acc, τ_lat), the router will compute the expected cost μ_cost(a) with an upper-confidence-bonus correction, evaluate a calibrated accuracy probability p̂_acc(a) and a latency estimate ℓ̂(a), construct the feasible action set, and select the feasible arm minimizing the cost-minus-bonus objective. After actuation, a provisional reward at T + Δ□ ≈ 1–5 min will be computed from the HITL-acceptance indicator, verdict self-consistency, and realized CpR; per-arm parameters will be tentatively updated only on the disjoint exploratory subsets used for ORAR learning analyses. At T + Δ□ (hours to days later), a terminal-reward correction Δr will be computed from post-mortem-label correctness; an idempotent compensating update will be applied; and the RRE per incident will be logged.

The static-policy commitment in Algorithm 1 binds in the confirmatory protocol: for the 16,000-run confirmatory ablation protocol and the 174-fault baseline-comparison protocol, the LinUCB parameters (A_a, b_a) will be frozen at the pre-warmed state established prior to OSF freeze (Section 3.3.3), and no on-line updates will be applied. Under the static policy, the k-th incident's routing decision is a deterministic function of incident features and frozen parameters, preserving the within-pair independence assumption of the paired-comparison design. The pre-warming pass operates LinUCB under thawed updates with the same exploration coefficient LinUCB used in the confirmatory protocol on the 1,200-incident synthetic corpus in pre-registered seed order; the ACP

is trained by minimizing cross-entropy against synthetic ground-truth remediation labels with early stopping on a 20 percent held-out validation subset. The pseudocode, hyperparameters, and seed sequence are deposited at OSF with content hashes recorded at freeze.

Algorithm 2. Action-Class Predictor. A small supervised classifier that pre-predicts the probable remediation class of an incident before pipeline dispatch. Given the L1 snapshot and the incident signature, it will extract structural features (node count, edge-class counts, anomaly-seed type, recent-deployment overlap, fault-signature hash, pillar-availability flags) and produce softmax probabilities over {schema change, data mutation, config change}, an MLE estimate of log expected WAL-replay size, and a blast-radius tier classification. In production deployment, the ACP would be retrained weekly on accumulated (incident, realized remediation) pairs; for the confirmatory protocol, the ACP weights will be frozen at the pre-warmed state established prior to OSF freeze (Section 3.3.3 and Algorithm B.1 in Appendix B), consistent with the LinUCB parameter freezing under the static-policy commitment. The ACP is ablated by HELIOS-noACP, which replaces it with a uniform latency prior.

Algorithm 3. Multi-Agent Hybrid Consensus. Aggregates the verdicts of active pipelines into a ranked root-cause list with calibrated uncertainty. For each pipeline p in the active set, MAHC will select a prior weight from the hierarchical prior (fault-type priors if sufficient data, else service-family priors, else global priors) and apply an uncertainty adjustment that reduces the effective weight as σ_p grows. Candidate causes from the union of pipeline ranked lists will receive a weighted Borda aggregation score; their variance will be computed as a weighted sum of squared σ_p. An entropy regularizer will test the normalized score distribution: if entropy is below

a pre-registered floor λ, MAHC will emit PROVISIONAL plus a HITL request rather than auto-executing. The mechanism is ablated by HELIOS-noConsensus, isolating the effect of uncertainty-aware hierarchical consensus on HR@3 and on the consensus-confusion rate.

Algorithm 4. Fidelity-Gated Shadow Validation. Validates a proposed remediation in the shadow twin only when twin-to-production distributional distance, measured by the two-sample Kolmogorov–Smirnov test on pillar-matched samples, falls below a pre-registered threshold $\alpha_ks = 0.05$ (one-sided rejection; with companion Type II bound $\beta_ks = 0.20$ for a Cohen's $d = 0.3$ target on the KS-statistic effect). When the KS gate passes, the remediation will be executed in the twin under the Action-Scoped Replay Engine (ASRE) mode (structural, WAL-replay, or probabilistic sketch) selected by the remediation's action class; post-action twin telemetry will be captured, and blast-radius will be computed against the pre-action baseline. Blast-radius exceedance triggers a FAIL verdict; KS-gate failure triggers a STALE verdict tracked as a first-class covariate.

Algorithm 5. K-hop Pruner (supporting). Given the UEG-C graph, seed nodes, a hop limit K, a Pearson threshold (pre-registered at $r = 0.7$), and an edge budget of 10,000, it expands the seed set by K-hop traversal over structural and behavioral edges, retains top-n nodes by personalized PageRank restart probability, drops causal and behavioral edges below the correlation threshold, and content-hashes the resulting subgraph for snapshot registry storage. The snapshot hash is the content-addressable identity under which between-variant comparisons are valid.

3.7.6 Observability Planes and Layers

HELIOS maintains two orthogonal structural decompositions: the five-layer execution hierarchy (L0–L4; Section 3.6.3) and a five-plane governance decomposition (Data, Reasoning, Coordination, Learning, Evaluation). Each component has a unique layer and a unique plane; the Evaluation plane deliberately spans every layer because it observes and gates every layer. Within the Evaluation plane, Class 1 components affect runtime behavior and together constitute Contribution C1 (Section 3.6.8): the Variant Control Layer (binary-level variant identity), the snapshot hash registry (input identity across variants), the metric integrity gate (runtime rejection of incomplete cells), the exclusion ledger (cryptographically-signed append-only missingness record), the deviation log (cryptographically-signed append-only protocol-change record), and the reconciliation ledger (provisional-and-terminal reward correspondence). Class 2 components observe but do not modify runtime behavior and are experimental infrastructure operating under OSF pre-registration discipline. The IOL self-telemetry path deliberately spans all layers under the tenant=self partition, enabling the methodological claim that HELIOS's own degradation will be observable through the same diagnostic apparatus that HELIOS applies to production. The empirical validation of C5 rests on synthetic injected control-plane degradation and is reported as exploratory rather than confirmatory evidence (Section 3.6.8).

3.7.7 Ablation-Aware Architecture

The ablation-aware architecture is the methodological core of the artifact and therefore of the research design. The twelve feature flags documented in Section 3.6.2 will produce eight confirmatory variants consistent with the OSF pre-registration (Table 2) and seven exploratory variants

documented below. None of the flags composes with another as a side effect, subject to the hidden-coupling audit reported in Section 3.6.2.

Table 17
Feature-Flag Matrix for the Eight Confirmatory Variants

Variant	L2C (LLM)	P4 (Cogn.)	MAHC	CBR	L2B (Graph)	ACP	Reconcile	UEG-C Struc-tural	Primary Test
HELIOS-Full	ON	ON	ON	ON	ON	ON	ON	ON	Reference
HELIOS-noLLM	OFF	ON	ON (D+G)	ON {D, G}	ON	ON	ON	ON	A-H1 — LLM neces-sity
HELIOS-LLM-only	ON	ON	passthru	passthru	OFF	OFF	passthru	ON	A-H2 — LLM suffi-ciency

Variant	L2C (LLM)	P4 (Cogn.)	MAHC	CBR	L2B (Graph)	ACP	Reconcile	UEG-C Struc-tural	Primary Test
HELIOS-D	OFF	ON	passthru	passthru	OFF	OFF	passthru	ON	A-H3 — Statistical floor
HELIOS-noConsensus	ON	ON	OFF (vote-only)	ON	ON	ON	ON	ON	A-H4 — Uncertainty consensus value
HELIOS-noRouter	ON	ON	ON	uniform	ON	ON	ON	ON	A-H5 — C6 routing value

Variant	L2C (LLM)	P4 (Cogn.)	MAHC	CBR	L2B (Graph)	ACP	Reconcile	UEG-C Struc-tural	Primary Test
HELIOS-G	OFF	ON (graph enc.)	passthru	passthru	ON	OFF	passthru	ON	A-H6 — Graph-only on narrative-heavy
HELIOS-noLLM (factual hallu-cination)	OFF	ON	ON (D+G)	ON	ON	ON	ON	ON	A-H7 — Hal-lucina-tion zero with-out LLM

Variant	L2C (LLM)	P4 (Cogn.)	MAHC	CBR	L2B (Graph)	ACP	Reconcile	UEG-C Struc-tural	Primary Test
HELIOS-noStructural	ON	ON	ON	ON	ON	ON	ON	OFF (com-posi-tion + train-ing dis-abled)	A-H8 — C2 struc-tural and integra-tion value

Note. "passthru" denotes that the named mechanism is inactive in the variant because its upstream dependencies are absent. The p4_cognitive flag is scoped to L-pipe visibility per the hidden-coupling audit (Section 3.6.2).

When set to OFF in HELIOS-noStructural, the ueg_structural flag replaces the four typed edge classes with a single untyped edge class, disables the subgraph-aware training discipline (each of D-pipe and G-pipe is trained on the full UEG-C rather than on the K-hop pruned subgraph consumed at inference time), and demotes Agent-Reasoning-Step nodes to flat textual descriptions presented to L-pipe agents. HELIOS-noStructural retains all other HELIOS-Full mechanisms and is therefore the closest minimal ablation isolating C2's structural and integration contribution. A-H7 is tested on the same HELIOS-noLLM variant as A-H1 because the hallucination-zero hypothesis

logically follows from removing the LLM; the two hypotheses use the same variant configuration but evaluate distinct dependent variables (HR@3 for A-H1, H_fact and H_struct for A-H7) and are therefore testable on the same paired observations without methodological conflict. A-H8 is a bundle-level confirmatory test: HELIOS-noStructural simultaneously ablates the four typed-edge composition rules, the subgraph-aware training discipline, and the structured Agent-Reasoning-Step node specification, and a single confirmatory hypothesis cannot identify which of these three sub-mechanisms carries any observed effect. Within-bundle attribution is reported descriptively under exploratory hypothesis E-H10 (Section 3.8.2) and is not part of the confirmatory family. Across all eight confirmatory variants, the ORAR LinUCB parameters and the ACP weights are frozen at the pre-warmed state (Sections 3.3.3 and 3.6.5).

Seven exploratory variants are also defined, each addressing a hypothesis demoted to exploratory status because the confirmatory corpus does not support its evaluation under the pre-registration discipline: HELIOS-noCogni (E-H1, cognitive-agentic HR@3 on the n = 12 Tier 4 subset, below statistical minimum), HELIOS-noACP (E-H8, latency-constraint behavior), HELIOS-noReconcile (E-H5, RRE on the delayed-label subset, requiring synthetic delayed-label injection), HELIOS-HIS3 (E-H9, hardware-isolation contention bias, an engineering-validity probe), HELIOS-noFGSV (E-H4, false-accept rate under synthetic fidelity-break perturbation), HELIOS-noIOL (E-H6, degradation-detection latency under injected control-plane degradation), and HELIOS-noExpl (E-H7, RQ4 user-study within-subjects narrative outcome). Exploratory variants will be reported with point estimates and 95 percent Bias-corrected and accelerated (BCa) bootstrap confidence intervals, clearly labeled and visually separated from the confirmatory cells, and will not enter the Holm–Bonferroni correction family. The reasons for exploratory placement are corpus-feasibility constraints documented in Section 3.6.8: RRE requires delayed-label

scenarios absent from the benchmarks as shipped (addressed via synthetic delayed-label injection on a 25 percent subset); FGSV and IOL require synthetic perturbation protocols (fidelity breaks and injected control-plane degradation) that warrant separate future pre-registration; HIS3 is an engineering-validity measurement; and noExpl belongs to the Research Question 4 user study.

3.7.8 Embedded Novelty

The artifact's novelty is embedded in six named contributions C1 through C6, ablation-tested where corpus feasibility permits and exploratorily validated where it does not. The present section summarizes the novelty attribution in a disciplined format: what is new precisely, how it differs from its most proximate comparator, how it is ablated or evidenced, and the evidential status (confirmatory or exploratory) of its empirical evaluation under the present corpus. Contribution-type classification follows (Gregor & Hevner, 2013): C1 is a methodological-discipline contribution; C2 is a construct-and-model contribution; C3′ is an evaluation-infrastructure-discipline contribution; C4 is a method contribution; C5 is an architectural-pattern contribution; and C6 is a method contribution with a cross-field-transfer profile from the delayed-feedback bandit literature.

C1 — Runtime-Enforced DSR Evaluation Rigor. Novel in the architectural enforcement of pre-registration discipline through the integrated Class 1 component apparatus: the Variant Control Layer enforces variant identity at the binary level via content-hashed feature-flag manifest reading at process start; the snapshot hash registry enforces input identity across variants via content-addressed UEG-C snapshots consumed by every active pipeline; the metric integrity gate enforces measurement completeness via runtime rejection of incomplete cells; the exclusion ledger enforces transparent missingness via cryptographically-signed append-only logging of ev-

ery excluded run; the deviation log enforces protocol-change transparency via cryptographically-signed append-only logging of every departure from the pre-registered plan; and the reconciliation ledger enforces deferred-reward correctness via provisional-and-terminal correspondence audit. Not novel in feature-flagging, content-hashing, ledger architectures, or pre-registration as concepts taken individually; the novelty is the integration as a research-method discipline that transforms DSR evaluation rigor from a reporting convention into a runtime invariant of the artifact. The most proximate comparator is the comparator AIOps evaluation literature (CHASE, mABC, FrugalGPT, TAMO, Flow-of-Action), in which ablations are reported in prose without binary-level variant identity, snapshot identity is asserted but not verified, missingness is silently imputed, and pre-registration is absent. The C1 contribution is methodological rather than algorithmic: it does not improve diagnostic accuracy by any pipeline mechanism, but it makes the falsifiability of every other contribution operationally enforceable. C1 is evaluated descriptively rather than via a confirmatory ablation hypothesis, since the contribution is the runtime-enforcement apparatus itself rather than a flag-toggleable mechanism; the enforcement is observed by the verification-rate, completion-rate, and ledger-entry-count metrics specified in Table 1 and reported as descriptive evidence in subsequent chapter. C1 is also a transferable-knowledge contribution in the Gregor and Hevner (Gregor & Hevner, 2013) sense, generalizable beyond HELIOS to AIOps research-method practice and arguably to DSR research-method practice more broadly.

C2 — UEG-C with cognitive nodes. Novel in the introduction of Agent-Reasoning-Step nodes and Transaction-or-KPI nodes into the evidence graph, the four typed edge classes (structural, behavioral, causal, economic) with formal composition rules, and subgraph-aware training under identical pruning at training and inference time. Not novel in the heterogeneous graph-

neural-network backbone (adopted from CHASE; (Z. Zhao et al., 2025), in personalized PageRank (adopted from MicroRCA; (L. Wu et al., 2020), or in hypergraph learning. The novelty is architectural-plus-system-integration. The mechanism is decomposed into two distinguishable claims with separate evidential pathways: a structural-and-integration claim (typed-edge composition rules, Agent-Reasoning-Step node specification, subgraph-aware training discipline), tested confirmatorily through HELIOS-noStructural under hypothesis A-H8 (Table 2); and a cognitive-agentic-accuracy claim (improvement on agentic-AI fault classes specifically), reclassified as exploratory empirical validation under E-H1, because the agentic-AI fault corpus consists of the 12 researcher-constructed Tier 4 instances — below the 73-fault statistical minimum, constructed through NIKA-style instrumentation specifically to exercise cognitive-node reasoning, and layered onto an AIOpsLab substrate that further bounds E-H1's exploratory evidence to "AIOpsLab plus cognitive-agentic instrumentation" rather than to cognitive-agentic faults in general. A-H8 is positioned as a bundle-level confirmatory test of the structural-and-integration contribution as an integrated whole. The variant ablates three sub-mechanisms simultaneously — typed-edge composition, subgraph-aware training, and structured Agent-Reasoning-Step nodes — and the chapter accepts that within-bundle attribution among these three sub-mechanisms is exploratory rather than confirmatory; descriptive within-bundle attribution is reported under E-H10 in the exploratory family.

C3′ — FGSV. An evaluation-infrastructure-discipline contribution with exploratory empirical validation. Novel in (a) the KS-gated shadow-suspension protocol with pre-registered $\alpha_{ks} = 0.05$ and $\beta_{ks} = 0.20$ for a Cohen's $d = 0.3$ target on the KS-statistic effect, (b) the STALE verdict as a first-class covariate enabling stratified analysis of shadow-validation availability rather

than a silent fallback, and (c) the bundling with action-class routing across the three ASRE modes (structural, WAL-replay, probabilistic-sketch). Not novel in traffic mirroring (an Istio deployment pattern), in the KS test, in shadow-testing as a concept, or in any individual ASRE mode. The contribution is evaluation-infrastructure discipline rather than algorithmic invention: FGSV converts shadow-testing from an engineering pattern into a falsifiable research instrument with pre-registered Type I and Type II bounds and a structurally tracked stale-fidelity covariate. The mechanism will be exercised by HELIOS-noFGSV under a synthetic fidelity-break perturbation protocol that does not arise spontaneously from the public benchmarks. Empirical evidence on the false-accept rate will be reported as exploratory (E-H4) and motivates a separate future pre-registration with a pre-specified effect size; the present chapter does not claim confirmatory empirical validation of C3′ on the available corpus.

C4 — MAHC. Novel in the (μ, σ) vote representation, the three-level hierarchical priors with backoff, and the entropy regularizer against overconfident unanimity. Not novel in weighted voting (adapted from mABC; (W. Zhang et al., 2024) or in Borda aggregation. The novelty is algorithmic, the primary comparator is mABC, and the mechanism will be ablated confirmatorily by HELIOS-noConsensus on HR@3 (ambiguous-incident stratum) and on the consensus-confusion rate under hypothesis A-H4. A deterministic synthetic counterexample — three-agent agreement at $\sigma = 0.1$ versus three-agent agreement at $\sigma = 0.8$, yielding identical scalar votes but distinguishable (μ, σ) representations — will be reported alongside the confirmatory inference as a conceptual illustration of the representation gap, but is not positioned as confirmatory evidence; the gap exists conceptually, but whether it matters empirically on the corpus is the question A-H4 and A-H7 are designed to answer.

C5 — IOL. An architectural contribution with exploratory empirical validation. Novel in writing self-telemetry to the same L1 graph under tenant=self and in consuming that telemetry through the same L2 pipelines that perform production diagnosis, without a separate monitoring stack. Not novel in self-monitoring as a concept. The most proximate comparators are concrete control-plane self-monitoring patterns rather than AIOps RCA systems: Prometheus monitoring its own scrape pipeline through self-instrumentation, OpenTelemetry self-instrumentation of its collector and exporter components, and the meta-monitoring features of commercial observability platforms such as Datadog and New Relic. Each of these patterns produces self-telemetry as a separate signal stream consumed by a separate monitoring layer; what differentiates IOL is not self-monitoring as a concept but the integrated-graph treatment in which self-telemetry is written under tenant=self into the same UEG-C consumed by the production-diagnostic pipeline. The integrated-graph treatment supports cross-tenant causal-path queries that flat self-monitoring patterns cannot express because their self-telemetry is structurally segregated from production telemetry. Three illustrative query classes are: (i) a path query from a production-tenant anomaly node through a behavioral edge to a control-plane-tenant degradation node within a single graph operation (e.g., correlating an L-pipe latency anomaly with an LLM Gateway saturation event); (ii) a typed-edge composition query joining causal edges from production-tenant anomalies with economic edges from business-tenant SLO breaches and structural edges from control-plane-tenant component dependencies (e.g., quantifying the SLO-breach contribution attributable to control-plane reasoning failures rather than data-plane infrastructure failures); (iii) a cross-tenant entropy query measuring whether a control-plane reasoning failure (e.g., MAHC consensus non-convergence) propagates to multiple production tenants simultaneously, distinguishing systemic from tenant-specific control-plane degradation. The C5 contribution is therefore a comparative claim with an articulated

mechanism rather than a uniqueness-by-absence claim, even though the empirical validation of the mechanism remains exploratory pending injected control-plane degradation under E-H6.

C6 — ORAR. A method contribution with a cross-field-transfer profile from the delayed-feedback bandit literature (Joulani et al., 2013) to AIOps RCA, where the source literature does not include the integrated extensions claimed here. Novel in (a) the two-stage provisional-and-terminal reward with reconciliation ledger, transferred from the delayed-feedback tradition but absent from the RCA literature; (b) the pre-routing Action-Class Predictor for latency-aware arm selection, with no prior AIOps system precedent; and (c) hardware-isolation-tier (HIS) tagging of every latency and cost measurement, a measurement-validity contribution with no precedent in the comparator set. Not novel in LinUCB (Li et al., 2010), in cost-aware cascading (FrugalGPT (L. Chen et al., 2023)), or in the contextual-bandit-routing concept. The primary comparator is FrugalGPT. The mechanism will be ablated confirmatorily by HELIOS-noRouter (A-H5) under the static-policy commitment, and exploratorily by HELIOS-noACP, HELIOS-noReconcile, and HELIOS-HIS3 (E-H8, E-H5, E-H9). The static-policy commitment narrows the confirmatory inference for C6 to the cost-aware static routing mechanism; the on-line learning behavior is reserved for the exploratory subset, and a confirmatory test of the on-line dynamics would require a future pre-registration that resolves the paired-inference confound of cross-incident learning.

The contribution-level evidential status under the present empirical evaluation is summarized as follows: C1 — methodological-discipline contribution evidenced descriptively through verification-rate, completion-rate, and ledger-entry-count metrics (Section 3.3.6 Table 1); C2 — confirmatory for the structural-and-integration claim through A-H8, with concurrent confirmatory evidence on hallucination rates through A-H7; exploratory for the cognitive-agentic-accuracy

claim through E-H1; exploratory for within-bundle attribution through E-H10; C3′ — evaluation-infrastructure-discipline contribution with exploratory empirical validation through E-H4; C4 — confirmatory through A-H4 (HR@3 on ambiguous incidents); C5 — architectural-pattern contribution with exploratory empirical validation through E-H6; C6 — confirmatory for static cost-aware routing through A-H5, exploratory for on-line learning dynamics through E-H5. This profile reflects explicit alignment between corpus feasibility (Section 3.5) and contribution claims, and is mapped to the hypothesis structure in Section 3.8.2 and to the limitations in Section 3.11.

3.8 Data Collection Procedures

Data collection will operate at two distinct temporal and architectural scales: continuous telemetry ingestion during each incident run, and per-run experimental-record aggregation across the 16,000-run ablation protocol and the 174-fault baseline-comparison corpus. The two scales are coupled by the run orchestrator, which will manage the dispatch schedule, record completion state per variant–benchmark–metric cell, and block aggregation until a cell is either completed or formally excluded under the inclusion criteria specified in the pre-registration.

Telemetry ingestion will proceed continuously across all five pillars via OTEL-compliant protocols (OTLP gRPC and HTTP). The Privacy Gateway will apply pattern-based personally identifiable information (PII) detection and named-entity-recognition-based redaction at the Data-Plane-to-Control-Plane boundary before any signal is forwarded to Plane 1 aggregators; redacted tokens will be replaced with deterministic placeholders that preserve grammatical structure for downstream LLM parsing. Per-tenant differential-privacy budgets ($\varepsilon = 1.0$ per tenant, $\delta = 10^{\square\square}$, with budget renewal at 24-hour rolling windows) will cap total information leakage from aggregated Knowledge Cortex queries; the (ε, δ) values are pre-registered at OSF freeze and any devia-

tion recorded in the deviation log. The Adversarial Robustness Monitor will inspect all ingress in parallel for prompt-injection patterns, telemetry-poisoning signals, and anomalies in the telemetry emission process itself. Data-collection volume is governed by the three-tier adaptive sampling strategy (Tier A baseline at 0.1 percent via eBPF kernel probes; Tier B dynamic adjustment based on runtime SLO pressure; Tier C anomaly-triggered escalation to 100 percent within the affected K-hop neighborhood). Tier C will downgrade automatically to Tier B when node memory utilization exceeds 80 percent or CPU exceeds 90 percent; the downgrade will be logged as a first-class covariate (`sampling_regime=downgraded`) so that downstream analysis can stratify by sampling regime and avoid attributing sampling-induced signal loss to an algorithmic effect.

Fault dataset will be sourced from the five pre-specified benchmarks (Section 3.5) with identical telemetry access granted to all evaluated systems, and ground-truth root-cause labels will be sourced directly from each benchmark's published annotation; no annotation will be regenerated or adjudicated for HELIOS-specific criteria. The 174-fault augmentation corpus will combine these benchmark sources with the Chaos-Mesh parametric extension (Tier 2) and the NIKA-style cognitive-agentic synthetic faults (Tier 4), with SHA-256 hashes of every fault instance and reserve-pool instance published at pre-registration freeze.

Experimental-record aggregation will operate at the variant–benchmark–metric cell granularity. Each cell will accumulate 400 paired observations (40 faults × 10 seeds) for the ablation protocol or the tier-determined count for the baseline-comparison corpus. The metric integrity gate at the Variant Control Layer will verify at run completion that every metric expected for the cell has been produced, will reject runs with missing measurements rather than silently impute zeros, and will record exclusions in the cryptographically-signed, append-only exclusion ledger. The minimum completed-run threshold for cell inclusion in confirmatory inference is 320 of 400 (80

percent); cells below this threshold will be reported as "insufficient data" with the completed count and the reason, and will not enter the hypothesis tests. No hypothesis will be dropped silently.

The 80 percent threshold (320 of 400 paired observations per cell) is justified on three grounds: (i) it preserves $n \geq 320$ paired observations per cell, exceeding the per-cell statistical minimum of $n = 73$ by 4.4× (Section 3.8.3); (ii) failed runs are characterized by infrastructure-level mechanisms (out-of-memory kills, network partitions, hardware contention, container-runtime errors) rather than fault-class-correlated mechanisms, supporting a missing-completely-at-random (MCAR) assumption; (iii) the MCAR assumption will be empirically tested in Chapter Four by comparing fault-class proportions and severity distributions between included and excluded runs at $\alpha = 0.05$ (chi-square test, two-sided). If the MCAR test rejects, the affected cells will be supplemented with sensitivity analysis using inverse-probability weighting on the propensity to be excluded, with weights estimated from the exclusion-ledger covariates; this conditional sensitivity analysis is pre-registered at OSF freeze. Hallucination labeling for H_fact and H_struct will follow the same inter-rater protocol as the RQ4 user study (two independent raters blind to variant, Cohen's $\kappa \geq 0.7$, third-adjudicator re-labeling triggered below 0.7, rubrics deposited at OSF freeze); raters will be calibrated on a 50-instance training subset prior to confirmatory labeling.

3.8.1 Exploratory Human Evaluation (Research Question 4)

The automated ablation and baseline protocols answer whether HELIOS is more accurate or cheaper than its comparators. They do not say whether a person looking at the output can actually use it. Research Question 4 is about that second problem: whether the Chain-of-Evidence narrative helps someone interpret a diagnosis, compared with a raw ranked list. The human evaluation is therefore an exploratory component of the design, not a third confirmatory family. It is bound

to hypothesis E-H7 in Table 3, uses the HELIOS-noExpl variant (HELIOS-Full with narrative rendering switched off), and is reported with point estimates and interval estimates only. It does not enter Holm–Bonferroni correction, and n = 24 is not treated as a basis for claiming how SREs, academics, or managers in general would behave.

The planned sample is 24 people, mixed on purpose. Incident output is not read only by on-call engineers. Academics who work on this literature, and business or service owners who approve tooling and live with downtime cost, also have to make sense of a diagnosis. The target mix is twelve practitioners with current or recent incident-response duties, six academic participants in information systems or software engineering, and six business or operations stakeholders (service owners, product managers, or equivalent). If a cell is hard to fill, the study will run with the mix actually recruited and the composition will be reported; roles will not be reclassified after the fact to make the table look balanced.

Recruitment will go through the researcher's professional network and through open calls on practitioner and academic lists. Screening is role-specific. Practitioners need at least three years of production incident work and a working familiarity with traces or equivalent observability tooling. Academic participants need postgraduate training or a faculty appointment in a related field and enough background to read an incident write-up. Business participants need a role in which they consume incident reports or decide on operational tooling. Anyone who helped design HELIOS, or who can identify the two output conditions from prior exposure, is excluded. A fixed honorarium, set at about one hour of professional consulting time and documented in the IRB protocol, will be paid; it is meant to compensate time, not to pressure anyone to take part.

The design is within-subjects. Each person sees both output conditions on a stratified set of twelve fault scenarios taken from the confirmatory corpus: HELIOS-Full (ranked list plus Chain-

of-Evidence narrative) and HELIOS-noExpl (the same underlying diagnosis with the narrative stripped, leaving the ranked list). Order is counterbalanced with a Latin square over output type and scenario stratum so that fatigue and learning are not loaded onto one condition. Sessions are expected to last about 60–75 minutes, remote or in person, under the same task script.

For each scenario the participant is given the RCA output and a short incident stem (what broke, roughly when, which service the user saw fail). The task is not to debug the live system. It is to (1) name the service they believe is the root cause, (2) name a fault category, (3) point to at least one piece of evidence in the output that supports that reading, and (4) choose accept, escalate, or reject as if they were the human in the loop. After each item they rate trust in that output on a five-point scale. Ground truth for scoring comes from the benchmark labels already used in the confirmatory corpus; participants do not see those labels during the task.

Three measures are collected. Correct interpretation is a binary score per person × scenario: the written answer has to get the ground-truth service, the ground-truth category, and one corroborating evidence item. Two raters, blind to condition, will score against a rubric deposited at OSF; Cohen's $\kappa \geq 0.7$ is required, and a third rater is brought in below that. Time-to-decision is the clock from presentation of the output to submission. Trust is the Likert rating, read alongside the accept / escalate / reject choice, which is logged and cannot be edited later. The Likert item is easy to talk up; the logged choice is there because of that.

Analysis follows E-H7. The binding comparison for correct-interpretation rate is a paired Wilcoxon signed-rank test. Time-to-decision is tested with a paired t on the log scale if Shapiro–Wilk at $\alpha = 0.05$ does not reject, otherwise Wilcoxon. Trust is reported as median and a 95 percent bootstrap interval per condition. Role (practitioner / academic / business) is a descriptive split, not a

confirmatory factor; cells of six to twelve people are too thin for that. As an exploratory hypothesis, E-H7 also carries a 95 percent BCa interval and stays out of the Holm families.

The study is a probe of whether the narrative rendering is usable by the kinds of people who would actually see HELIOS output, under twelve constructed scenarios. It is not a population survey of SREs, not a market study, and not evidence that organizations will adopt the system. Mixing roles will add variance that the original n = 24 calculation ($d_z = 0.6$, one-sided $\alpha = 0.05$, 80 percent power, n = 22 plus 10 percent attrition) did not fully price in; that calculation is a planning number, not a license to generalize. Findings are bounded to the people who take part and to the scenarios shown. A larger, role-stratified confirmatory study would be a separate pre-registration.

The protocol, recruitment text, consent form, debrief, and analysis plan go to the Golden Gate University IRB before any data are collected. Approval status and protocol number will be written into the OSF deviation log at freeze. Ethics handling for participants is specified in Section 3.10.

Principal data-collection instruments will be aligned with the dependent variables specified in Section 3.3.6 and the evaluation metrics specified in Section 3.8.4. Per-plane OTEL span duration histograms will provide time-to-detect (TTD), time-to-triage (TTT), and per-plane inference latency at p50, p95, and p99. The MAHC consensus output module at Plane 4 will provide the ranked root-cause list against which ground-truth labels are evaluated for HR@k, mean reciprocal rank (MRR), and macro F1. The Plane 3 LLM Gateway token counter will provide per-episode token spend for CpR computation, and the Plane 7 Critic Agent post-incident computation module will aggregate token, infrastructure, storage, human-minute, and opportunity-cost components into the five-component CpR sum. The P4 cognitive-telemetry pipeline will provide schema-validation failure counts for structural hallucination rate (H_{struct}); the Narrative Engine output, parsed by a

deterministic claim-extraction parser, will provide atomic factual claims for independent-rater labeling of the factual hallucination rate (H_fact). The Monitor Agent audit output will provide ethics-compliance rates, the Plane 5 MAML-Quick execution will provide retraining overhead in CPU-hours and GPU-hours, and the multi-judge Faithfulness-Accuracy-Explanation Score (FAE)-Score ensemble (three judge models plus 10 percent human-anchored review) will provide narrative-quality scoring for the Research Question 4 user study. Each instrument is specified by its architectural location and its event-generation contract, supporting independent replication.

3.9 Data Analysis Plan

3.9.1 Research Questions and Hypotheses

The research questions that will orient the empirical evaluation are derived from the dissertation's stated objective and from the architectural commitments of the HELIOS artifact.

Research Question 1. To what extent will the co-deployment of statistical, graph-machine-learning, and large language model reasoning as peer pipelines under identical inputs improve root-cause-analysis accuracy and reduce cost per resolution compared with baseline systems that deploy a single reasoning modality or a sequential pipeline?

Research Question 2. To what extent will the integration of self-telemetry into the same diagnostic apparatus enable the detection and mitigation of reasoning failures within the AI system itself, beyond the detection of failures in the production workloads the AI system is diagnosing?

Research Question 3. To what extent will each named architectural mechanism of HELIOS — cognitive-telemetry integration (C2), fidelity-gated shadow validation (C3′), uncertainty-aware hierarchical consensus (C4), meta-observability (C5), and cost-aware adaptive routing (C6) — individually contribute to diagnostic accuracy, cost efficiency, and safety, as determined by ablation?

Research Question 4. To what extent will the Chain-of-Evidence narrative rendering improve SRE interpretation of RCA output, measured by correct-interpretation rate, time-to-decision, and self-reported trust in a within-subjects user study?

The pre-registered hypotheses are organized into three families, each corrected for family-wise error under Holm–Bonferroni with pre-registered rank ordering specified in Section 3.8.3. The ablation family (A-H1 through A-H8) tests the internal contribution of each architectural mechanism against the corresponding feature-flag-gated variant; it contains eight confirmatory hypotheses, evaluated on the eight ablation variants specified in Section 3.6.7 Table 2 (note that A-H1 and A-H7 are tested on the same HELIOS-noLLM variant but evaluate distinct dependent variables — HR@3 for A-H1 and hallucination rates for A-H7 — and are therefore independent confirmatory tests). The baseline-comparison family (B-H1 through B-H8) tests the external comparative advantage of HELIOS-Full against five priority-ranked AIOps baselines on HR@3 (B-H1 through B-H5), log-MTTR (B-H6), log-Cost (B-H7), and Avg@5 (B-H8); the baseline family is corrected separately from the ablation family because the two families answer different questions (internal mechanism contribution vs external comparative advantage) and would violate the inferential logic of a single correction family if combined.

The exploratory family (E-H1 through E-H10) does not enter Holm–Bonferroni correction and is reported with point estimates and 95 percent BCa bootstrap confidence intervals only; it in-

cludes the C2 cognitive-agentic test (E-H1, reclassified from the ablation family because the Tier 4 corpus of n = 12 is below the 73-fault statistical minimum and is constructed to exercise cognitive-node reasoning per Section 3.6.8); the demoted near-tautological hypothesis E-H2 (uniform routing increases TTT, which follows from the deselection of the latency-aware routing constraint) and the related E-H3 (latency-constraint behavior); the FGSV false-accept rate (E-H4) and IOL detection latency (E-H6), both requiring synthetic perturbation protocols absent from the public benchmarks; the RRE measurement on the delayed-label subset (E-H5); the RQ4 user-study CoE narrative outcomes (E-H7); the HELIOS-noACP latency-constraint and HELIOS-HIS3 contention exploratory variants (E-H8, E-H9); and the C2 within-bundle attribution probe (E-H10) introduced under the bundle-level framing of A-H8 (Section 3.6.8). The schema is exhaustive; every pre-registered hypothesis receives exactly one label, and the prefixed labels are binding from OSF freeze forward. C1 is evidenced descriptively rather than via a confirmatory or exploratory hypothesis (Section 3.6.8) and therefore does not appear in any of the three families.

3.9.2 Research Question–Hypothesis–Architecture Mapping

The binding mapping from research questions to hypotheses to architectural components and ablation variants is presented in Table 3. Every confirmatory hypothesis corresponds to exactly one variant–metric cell; every variant corresponds to exactly one set of feature-flag configurations as specified in Section 3.6.7; every metric has a named measurement location in the architecture as specified in Section 3.7. Exploratory hypotheses appear in the table for completeness and are clearly demarcated. The mapping is operational rather than documentary: it is enforced by the Variant Control Layer at pre-registration freeze, and the run orchestrator will block aggregation of any cell that fails to receive its full complement of paired observations under the inclusion criteria.

Table 18
Research Question to Hypothesis to Architecture to Variant to Metric to Statistical-Test Mapping

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ1	**A-H1:** If the L-pipe is removed (HELIOS-noLLM) rather than retained (HELIOS-Full), then HR@3 will be lower.	L2c L-pipe	HELIOS-noLLM	HR@3	Wilcoxon signed-rank	Ablation / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ1	**A-H2:** If only the L-pipe is used (HELIOS-LLM-only) rather than the hybrid peer pipelines (HELIOS-Full), then HR@3 will be lower.	L2c L-pipe	HELIOS-LLM-only	HR@3	Wilcoxon signed-rank	Ablation / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ1	**A-H3:** If only the D-pipe is used (HELIOS-D) rather than HELIOS-Full, then HR@3 will be lower.	L2a D-pipe	HELIOS-D	HR@3	Wilcoxon signed-rank	Ablation / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ1	**A-H5:** If ORAR is replaced by uniform routing (HELIOS-noRouter) rather than retained, then CpR at matched HR@3 will be higher.	L3 ORAR CBR	HELIOS-noRouter	CpR at matched HR@3	Wilcoxon signed-rank	Ablation / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ1	**B-H1–B-H5:** If HELIOS-Full is used in place of baseline k (k = 1…5, priority-ranked), then the paired binary HR@3 outcome will favor HELIOS-Full.	Full system	HELIOS-Full vs. 5 Baselines	HR@3 (paired binary)	McNemar's exact	Baseline / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ1	**B-H6:** If HELIOS-Full is used in place of the best baseline, then log-MTTR will be lower.	Full system	HELIOS-Full vs. Best	log-MTTR	Wilcoxon signed-rank (paired t on log values reported as sensitivity analysis)	Baseline / Confirma-tory
RQ1	**B-H7:** If HELIOS-Full is used in place of the best baseline, then log-Cost (CpR) will be lower.	Full system	HELIOS-Full vs. Best	log-Cost	Wilcoxon signed-rank (paired t on log values reported as sensitivity analysis)	Baseline / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ1	**B-H8:** If HELIOS-Full is used in place of the best baseline, then Avg@5 will be higher.	Full system	HELIOS-Full vs. Best	Avg@5	Wilcoxon signed-rank (paired t reported as sensitivity analysis)	Baseline / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ2	**E-H6:** If IOL is disabled (HELIOS-noIOL) rather than retained, then detection latency for injected LLM/control-plane degradation will be higher.	L0/L1 IOL	HELIOS-noIOL + injection	Detection latency	BCa bootstrap CI	Exploratory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**A-H4:** If MAHC uncertainty-aware consensus is replaced by vote-only aggregation (HELIOS-noConsensus) rather than retained, then HR@3 on the ambiguous-incident stratum will be lower.	L3 MAHC	HELIOS-noConsensus	HR@3 (Ambigu-ous)	Wilcoxon signed-rank	Ablation / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**A-H6:** If only the G-pipe is used (HELIOS-G) rather than HELIOS-Full, then HR@3 on narrative-heavy incidents will be lower.	L2b G-pipe	HELIOS-G	HR@3 (Narrative-heavy)	Wilcoxon signed-rank	Ablation / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**A-H7:** If the L-pipe is removed (HELIOS-noLLM) rather than retained, then factual and structural hallucination rates (H_fact, H_struct) will be at or near zero, versus greater than zero under HELIOS-Full.	L/G-pipes	HELIOS-noLLM (DV: hallucination)	H_fact, H_struct	Sign test	Ablation / Confirmatory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**A-H8:** If UEG-C typed-edge composi-tion, subgraph-aware training, and Agent-Reasoning-Step nodes are disabled (HELIOS-noStructural) rather than retained, then HR@3 will be lower.	L1 UEG-C structural sub-mechanisms	HELIOS-noStructural	HR@3	Wilcoxon signed-rank	Ablation / Confirma-tory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**E-H1:** If P4 cognitive nodes are removed (HELIOS-noCogni) rather than retained, then HR@3 on the Tier 4 agentic-AI subset will be lower.	L1 UEG-C cognitive nodes	HELIOS-noCogni	HR@3 (Tier 4 subset)	BCa bootstrap CI	Exploratory (reclassi-fied)

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**E-H2:** If uniform routing is used (HELIOS-noRouter) rather than ORAR, then time-to-triage (TTT) will be higher.	L3 ORAR CBR	HELIOS-noRouter	TTT	BCa bootstrap CI	Exploratory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**E-H3:** If the Action-Class Predictor is removed (HELIOS-noACP) rather than retained, then latency-constraint satisfaction will be lower.	L3 ACP	HELIOS-noACP	Latency-constraint satisfaction	BCa bootstrap CI	Exploratory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**E-H4:** If FGSV is disabled (HELIOS-noFGSV) rather than retained, then the false-accept rate under synthetic fidelity-break perturba-tion will be higher.	L4 FGSV	HELIOS-noFGSV	False-accept rate	BCa bootstrap CI	Exploratory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**E-H5:** If reward rec-onciliation is disabled (HELIOS-noReconcile) rather than retained, then Reward Reconcilia-tion Error on delayed-label incidents will be non-zero (and higher than under HELIOS-Full).	L4 Reconcile	HELIOS-noReconcile	RRE per incident	BCa bootstrap CI	Exploratory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**E-H8:** If the Action-Class Predictor is removed (HELIOS-noACP) rather than retained, then latency-constraint satisfaction will be lower.	L3 ACP	HELIOS-noACP	Latency-constraint satisfaction	BCa bootstrap CI	Exploratory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**E-H9:** If hardware isolation is reduced to HIS-3 (HELIOS-HIS3) rather than the design-committed isolation tier, then per-incident latency will be higher.	Engineering-validity	HELIOS-HIS3	Δ-latency per HIS tier	BCa bootstrap CI	Exploratory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ3	**E-H10:** If the C2 structural bundle is ablated (HELIOS-noStructural), then the ΔHR@3 will be attributable across typed-edge composi-tion, subgraph-aware training, and ARS-node sub-mechanisms.	L1 UEG-C structural sub-mechanisms	HELIOS-noStructural (descriptive sub-analysis)	ΔHR@3 attribution across typed-edge / training-discipline / ARS-node sub-mechanisms	Descriptive with BCa bootstrap CI	Exploratory

RQ	Hypothesis	Component	Variant	Primary Metric	Statistical Test	Family / Status
RQ4	**E-H7:** If Chain-of-Evidence narrative is shown rather than a raw ranked list (HELIOS-Full vs HELIOS-noExpl), then SRE correct-interpretation rate and trust will be higher and time-to-decision will be lower.	L3 Narrative	HELIOS-noExpl vs. Full	Interp. rate; TTT; trust	Within-subjects study (paired Wilcoxon)	Exploratory

Note. McNemar's exact test is used for paired binary HR@3 outcomes in the baseline-comparison family (B-H1 through B-H5); Wilcoxon signed-rank is the binding test for paired continuous outcomes throughout (ablation family; B-H6, B-H7, B-H8). For B-H6, B-H7, and B-H8, paired t-tests on log-transformed outcomes are reported as sensitivity analysis only and are not the binding inference. Exploratory hypotheses receive bias-corrected and accelerated bootstrap confidence intervals but do not enter the Holm–Bonferroni family-wise correction. HR@3 in HELIOS-noConsensus is measured at the same Plane 4 architectural location as in HELIOS-Full, where the vote-only aggregator replaces the MAHC consensus rule but emits a structurally identical ranked-output tuple; the construct-validity asymmetry that this measurement-location identity does not dissolve is acknowledged in Section 3.9.2. C1 (runtime-enforced evaluation rigor) does not appear in this table because it is evaluated descriptively through verification-rate, completion-rate, and ledger-entry-count metrics (Section 3.6.8 and Table 1) rather than via a confirmatory or exploratory hypothesis.

For every contribution C1 through C6 with confirmatory hypothesis A-Hy or B-Hy, failure to reject the null hypothesis at the pre-registered Holm–Bonferroni-adjusted α implies lack of empirical support for the contribution under the conditions specified in this chapter; negative confirmatory outcomes do not invalidate the architectural specification of the contribution but bound the empirical evidence base, and would prompt a corresponding revision of the contribution's evidential status in the final dissertation. For exploratory hypotheses, no confirmatory inference is drawn; exploratory results inform but do not test the corresponding contribution claim and inform the design of future confirmatory pre-registrations. Specific to A-H4 (no-consensus on ambiguous incidents) and A-H8 (HELIOS-noStructural bundle), the per-hypothesis power audit in Section 3.8.3 indicates these tests are underpowered at the delivered corpus; the pre-registered inferential

rule for these two hypotheses is asymmetric — rejection at $\alpha = 0.00625$ supports the contribution; non-rejection is reported as "inconclusive at delivered power" rather than "no support." This asymmetric rule is locked at OSF freeze.

3.9.3 Statistical Methods

The statistical analysis will be structured around five methodological commitments, summarized here and detailed below: (i) within-subjects paired design with Wilcoxon signed-rank as primary test; (ii) family-wise error control through separate Holm–Bonferroni correction within each hypothesis family with pre-registered rank ordering; (iii) effect-size derivation grounded in minimum-detectable-effect framing rather than operational-significance midpoint; (iv) BCa bootstrap confidence intervals for every variant–benchmark–metric cell; (v) pre-registration discipline with deviation-log enforcement.

First, paired design. The experiment will be a within-subjects paired design: every fault instance will be processed by HELIOS and by each comparator variant or baseline, so each fault will act as its own control. The validity of the paired-comparison test relies on within-pair independence of treatment effects across incidents, which is operationally protected by the static-policy commitment documented in Section 3.3.3 and Section 3.6.5. The on-line LinUCB and ACP learning dynamics will be evaluated separately on the disjoint exploratory delayed-label subset and do not contribute to confirmatory inference. The primary test for paired non-parametric comparisons of HR@k, MTTR, and CpR across variant–benchmark cells will be the Wilcoxon signed-rank test, in one-sided form where a directional expectation is pre-registered and two-sided form otherwise. The rank-based mixed-effects analysis specified in Section 3.5 (with nparLD as concurrent sen-

sitivity check) is applied on Tier 1 only to verify pseudoreplication-controlled robustness of the Wilcoxon inference and does not supersede the primary confirmatory result.

Second, family-wise error control. Family-wise error rate will be controlled through Holm–Bonferroni correction applied separately within each hypothesis family with pre-registered rank ordering. The rank-1 adjusted α is 0.00625 for the eight-hypothesis baseline-comparison family and 0.00625 for the eight-hypothesis ablation family.

The Holm rank order is binding from OSF freeze and is established by a priori expected effect size, largest first, so that the most-likely-to-reject hypothesis is tested at the strictest α and the procedure terminates early if H_(1) fails to reject. The pre-registered rank order for the ablation family is: A-H3 (D-pipe statistical floor; expected ΔHR@3 ≥ 0.15) > A-H7 (hallucination zero without LLM; expected H_fact ≈ 0 vs > 0) > A-H1 (LLM necessity; expected ΔHR@3 ≥ 0.10) > A-H2 (LLM-only sufficiency; expected ΔHR@3 ≥ 0.10) > A-H6 (graph-only on narrative-heavy; expected ΔHR@3 ≥ 0.12) > A-H5 (uniform routing increases CpR; expected Δlog-CpR ≥ 0.60) > A-H4 (no-consensus on ambiguous; expected ΔHR@3 ≥ 0.05, underpowered) > A-H8 (HELIOS-noStructural bundle; expected ΔHR@3 ≥ 0.08, underpowered). The pre-registered rank order for the baseline-comparison family is: B-H1 through B-H5 (HELIOS exceeds five baselines on HR@3) ordered by expected discordance with HELIOS-favorable ratio, followed by B-H7 (cost reduction; expected δ = 0.80) > B-H6 (MTTR reduction; expected δ = 0.60) > B-H8 (Avg@5; expected ΔAvg@5 ≥ 0.10). Holm–Bonferroni is selected over plain Bonferroni because several hypotheses in each family are correlated. Family-wise error control under Holm–Bonferroni is governed by the number of hypotheses tested within a family, not by the sample size of the underlying paired observations; the symmetric rank-1 α across the two eight-hypothesis families is therefore the pro-

cedurally correct correction regardless of corpus-size asymmetry, with per-test power (rather than family-wise error) accounting for the corpus-size difference. Benjamini–Hochberg false discovery rate at q = 0.05 is reported as a sensitivity analysis; confirmatory inference is FWER-controlled.

Third, effect-size derivation under minimum-detectable-effect framing. The binding effect size is specified as the minimum effect detectable at the delivered corpus under the pre-registered statistical procedure, rather than as a researcher-chosen operational-significance threshold. At the delivered n = 174 paired observations, one-sided Holm–Bonferroni-adjusted $\alpha = 0.00625$, paired correlation $\rho = 0.5$, and 80 percent power, the protocol detects a paired-difference effect of Cohen's $h \geq 0.276$ (computed as $h = 2 \cdot \arcsin(\sqrt{p_1}) - 2 \cdot \arcsin(\sqrt{p_0})$ using IEEE-754 double precision, rounded to three decimal places). Against the AIOpsLab-equivalent CHASE baseline of HR@3 = 0.60 (derivation in Table 5 Panel A), this corresponds to a HELIOS target of HR@3 ≥ 0.73, or an absolute improvement of 13 percentage points.

The 13-percentage-point framing is presented as a consequence of the minimum-detectable-effect specification rather than as a researcher-chosen operational-significance midpoint. Industrial-deployment retrospectives (Shan et al., 2025; T. Wang & Qi, 2024) describe a qualitative noise-significance transition in production AIOps deployments where HR@3 improvements small enough to be obscured by operational noise produce no detectable downstream effect in MTTR or CpR telemetry, while improvements large enough to clear that noise floor produce persistent and measurable production effects; the protocol's minimum-detectable effect of 13 pp falls within the qualitative band described by these retrospectives, supporting the practical relevance of the detection threshold without grounding it numerically in the citations. Sensitivity is reported across $h \in [0.213, 0.339]$ in Table 6 Panel B.

The CHASE-to-AIOpsLab adjustment factor that will translate CHASE's published HR@3 (0.72) into the AIOpsLab-equivalent baseline applies three adjustments: service-count dilution (−0.04 to −0.06), fault-taxonomy expansion (−0.02 to −0.04), and labeling-discipline strictness (−0.02 to −0.04), totaling −0.08 to −0.14 with midpoint −0.12 and yielding an AIOpsLab-equivalent CHASE baseline of HR@3 = 0.60. Each adjustment component, its citation source, and the sensitivity of the binding effect size to ±25 percent perturbation of each component will be reported in Table 5-8 and locked at OSF freeze.

Applying the minimum-detectable-effect framing to the adjusted baseline of HR@3 = 0.60 yields a HELIOS target of HR@3 = 0.73 and the corresponding Cohen's h = 0.276 (one-sided; arcsine-transformation paired difference). At one-sided $\alpha = 0.00625$, paired correlation $\rho = 0.5$, and 80 percent power, the required paired sample size will be n = 73 faults per pair; inflated 15 percent for attrition yields n = 84, and the delivered corpus of 174 will be 2.1× the statistical minimum. A sensitivity analysis is reported across paired correlations $\rho \in \{0.4, 0.5, 0.6\}$ and across $h \in [0.213, 0.339]$. Effect sizes for continuous outcomes are MTTR $\delta = 0.60$ (medium, on log-scale) and Cost $\delta = 0.80$ (large, consistent with FrugalGPT's reported 40–98 percent cost-reduction range; (L. Chen et al., 2023)). The corpus is robust to $h \geq 0.25$ but would be under-powered for $h < 0.20$; this bound is reported explicitly as a scope condition (Table 4).

A per-hypothesis power audit across the eight ablation hypotheses at $\alpha = 0.00625$, $\rho = 0.5$, and n = 174 will be reported in Table 7 Panel C. A-H1, A-H2, A-H3, A-H5, A-H6, and A-H7 all meet or exceed 80 percent power at the delivered corpus. A-H4 (ambiguous-incident consensus, $\Delta HR@3 \geq 0.05$, $h \approx 0.10$ against an adjusted baseline of 0.55) is under-powered at approximately 65 percent. A-H8 (structural-and-integration bundle, $\Delta HR@3 \geq 0.08$, $h \approx 0.18$ against the adjusted AIOpsLab baseline of 0.60) is under-powered at approximately 62 percent. Both A-H4 and A-H8

are retained in the confirmatory family with the pre-registered asymmetric inferential rule specified in Table 3 footnote: rejection at $\alpha = 0.00625$ supports the contribution; non-rejection is reported as "inconclusive at delivered power" rather than "no support."

Panel D reports the per-hypothesis power audit for the baseline-comparison family under McNemar's exact test (B-H1 through B-H5) and Wilcoxon signed-rank (B-H6 through B-H8). McNemar's power depends on the discordant-pair proportion p_disc = p□□ + p□□ and the discordance ratio r = p□□ / (p□□ + p□□), not on Cohen's h directly; the audit is therefore presented across (p_disc, r) cells. At one-sided $\alpha = 0.00625$ and 80 percent power, the delivered n = 174 supports confirmatory inference for B-H1–B-H5 at p_disc ≥ 0.30 with r ≥ 0.70 (HELIOS-favorable). Below this threshold the family is under-powered; the asymmetric inferential rule applies to any B-Hx whose realized discordance falls below the powered range.

Table 19
Sample-Size Sensitivity for Cohen's h under One-Sided α = 0.00625, ρ = 0.5, 80% Power

Cohen's h	Required n per pair (no attrition)	Required n per pair (15% attrition)	Adequacy of delivered n = 174
0.15	≈ 245	≈ 282	Under-powered
0.20	≈ 138	≈ 159	Marginal
0.276 (binding)	**73**	**84**	**2.1× minimum**
0.35	≈ 47	≈ 54	**3.2× minimum**

Note. Sample sizes are computed under the paired-difference Cohen's *h* approximation for arcsine-transformed proportion differences.

Table 20
Panel A — CHASE-to-AIOpsLab Adjustment Components

Adjustment Component	Lower Bound	Midpoint (Binding)	Upper Bound	Source
Service-count dilution (CHASE → AIOpsLab)	−0.04	−0.05	−0.06	(L. Wu et al., 2020; Pham et al., 2024a; Zhang et al., 2025)
Fault-taxonomy expansion (4 → 6 categories)	−0.02	−0.03	−0.04	AIOpsLab specification (Y. Chen et al., 2025)
Labeling-discipline strictness (multi-cause → single-cause)	−0.02	−0.03	−0.04	AIOpsLab specification (Y. Chen et al., 2025); CHASE specification (Z. Zhao et al., 2025)

Adjustment Component	Lower Bound	Midpoint (Binding)	Upper Bound	Source
Total adjustment	**−0.08**	**−0.12**	**−0.14**	—
Adjusted CHASE-equivalent baseline on AIOpsLab	**HR@3 = 0.64**	**HR@3 = 0.60**	**HR@3 = 0.58**	Derived

Table 21
Panel B — Operational-Significance Threshold Sensitivity

Threshold Position	Threshold Value	HELIOS Target HR@3	Cohen's h	Required n per Pair (15% Attrition)	Adequacy at Delivered n = 174
Lower bound (−25%)	10 pp	0.70	0.213	145	Marginal (1.2× minimum)
Midpoint (binding)	**13 pp**	**0.73**	**0.276**	**84**	**Adequate (2.1× minimum)**

Threshold Position	Threshold Value	HELIOS Target HR@3	Cohen's h	Required n per Pair (15% Attrition)	Adequacy at Delivered n = 174
Upper bound (+25%)	16 pp	0.76	0.339	56	Robust (3.1× minimum)

Table 22
Panel C — Per-Hypothesis Power Audit (Ablation Family, α = 0.00625, ρ = 0.5, n = 174)

Hypothesis	Effect-Size Target	Effect Size (h or δ)	Adjusted Baseline	Power at n = 174	Status
A-H1 (LLM necessity)	ΔHR@3 ≥ 0.10	$h \approx 0.22$	0.60	~85%	Adequate
A-H2 (LLM-only suffi-ciency)	ΔHR@3 ≥ 0.10	$h \approx 0.22$	0.60	~85%	Adequate

Hypothesis	Effect-Size Target	Effect Size (h or δ)	Adjusted Baseline	Power at n = 174	Status
A-H3 (D-pipe statistical floor)	ΔHR@3 ≥ 0.15	$h \approx 0.32$	0.60	~96%	Robust
A-H4 (ambiguous-incident consensus)	ΔHR@3 ≥ 0.05	$h \approx 0.10$	0.55	~65%	**Under-powered (disclosed)**
A-H5 (uniform routing CpR)	Δlog-CpR ≥ 0.60	$\delta = 0.60$	—	~90%	Adequate
A-H6 (graph-only on narrative-heavy)	ΔHR@3 ≥ 0.12	$h \approx 0.27$	0.60	~91%	Adequate

Hypothesis	Effect-Size Target	Effect Size (h or δ)	Adjusted Baseline	Power at n = 174	Status
A-H7 (hallucination zero w/o LLM)	H_fact ≈ 0 vs. > 0	Sign test on near-zero rate	—	~93% (sign test, p□ = 0.05, p□ = 0.30, n = 174)	Adequate
A-H8 (structural-and-integration bundle)	ΔHR@3 ≥ 0.08	*h* ≈ 0.18	0.60	~62%	Under-powered (disclosed)

Note: A-H7 power is computed under the exact sign test for a one-sided difference in proportions (H_fact rate) at α = 0.00625, n = 174; the h-based formulation is not applicable because the null is a near-zero rate.

Table 23
Panel D — Per-Hypothesis Power Audit, B-Family under McNemar's Exact (α = 0.00625, 80% power)

(p_disc, r)	n Required	Adequacy at Delivered n = 174
(0.20, 0.60)	~530	Severely under-powered
(0.20, 0.70)	~325	Under-powered
(0.20, 0.80)	~200	Marginal
(0.30, 0.60)	~360	Under-powered

(p_disc, r)	n Required	Adequacy at Delivered n = 174
(0.30, 0.70)	~218	Marginal
(0.30, 0.80)	~140	Adequate (1.2× minimum)
(0.40, 0.60)	~270	Under-powered
(0.40, 0.70)	~165	Adequate (1.05× minimum)
(0.40, 0.80)	~108	Robust (1.6× minimum)

Note. Computed via mid-p McNemar's exact under one-sided alternative. The realized (p_disc, r) for B-H1 through B-H5 will be reported in Chapter Four; if any falls below the powered range, the asymmetric inferential rule applies.

The relationship between the Wilcoxon signed-rank test and the rank-based mixed-effects analysis specified in Section 3.5 requires explicit binding to eliminate researcher-degrees-of-freedom risk. The Wilcoxon signed-rank test on paired (variant, HELIOS-Full) differences within each (benchmark, metric) cell will be the binding confirmatory inference for the ablation family; the rank-based mixed-effects analysis (with problem_id as a random intercept and benchmark_source as a fixed effect, supplemented by nparLD) will be reported only on Tier 1 as a pseudoreplication-controlled sensitivity analysis whose role is to verify that the Wilcoxon inference does not shift materially when the within-Tier-1 dependence structure is modeled explicitly. For the baseline-comparison family, McNemar's exact test will be the binding confirmatory inference for paired binary HR@3 outcomes (B-H1 through B-H5); Wilcoxon signed-rank will be the binding confirmatory inference for paired continuous outcomes (B-H6, B-H7, B-H8). Paired one-sided t-tests on log-transformed continuous outcomes will be reported as sensitivity analyses

for B-H6, B-H7, and B-H8 but will not constitute the binding inference for any hypothesis, removing the conditional-test design that would inflate Type I error under Shapiro–Wilk gating. The rank-based mixed-effects analysis is not used as the confirmatory test for any baseline-comparison hypothesis. The binding is locked at OSF freeze; any post-hoc substitution would be recorded as a deviation-log entry with its analytic consequence — typically the affected hypothesis moving from confirmatory to exploratory status.

Fourth, BCa bootstrap confidence intervals. Confidence intervals will be reported for every variant–benchmark–metric cell using the bias-corrected and accelerated (BCa) bootstrap with 10,000 resamples; resampling is at the paired-incident level, the BCa seed sequence is pre-registered, and the acceleration estimate uses jackknife acceleration. Cliff's δ accompanies every Wilcoxon result; Cohen's d accompanies every paired-t result; Hodges–Lehmann estimates of location shift accompany every paired-difference reporting.

Fifth, pre-registration discipline. All confirmatory tests will operate under the pre-registration discipline specified in Section 3.5 and Section 3.3.3: pre-registered seed register, pre-registered SHA-256-hashed fault corpus, pre-registered variant feature-flag matrix bound to the binary commit, and pre-registered price book. Deviations will be recorded in the deviation log with timestamp, the clause deviated from, the concrete change, the reason, and the analytic consequence (typically: the affected test moves from confirmatory to exploratory). The deviation log is an append-only, cryptographically-signed file deposited at OSF at every commit; write access is restricted to the candidate, read access is public via OSF, and entries are written before any analysis benefiting from the deviation is conducted.

3.9.4 Evaluation Metrics

The confirmatory evaluation employs five primary metrics. Hit Rate at k (HR@k) for k {1, 3} is the fraction of incidents for which the ground-truth root cause appears in the top-k of the ranked root-cause list published by the MAHC consensus; HR@3 is the primary outcome for the ablation family and the baseline-comparison family. Mean Time to Resolution (MTTR), for incidents that receive autonomous remediation, is the elapsed time from anomaly detection to post-remediation telemetry restoration within the incident window [t_injection, t_injection + 1800 s]; it will be log-transformed prior to parametric analysis to correct for right-skew. Cost per Resolution (CpR) is the five-component sum of token cost, infrastructure cost, storage cost, human-minute cost, and opportunity cost, computed per incident under the pre-registered price book. Total CpR is the binding metric for B-H7; per-component decomposition will be reported descriptively (without confirmatory inference on individual components) to disambiguate algorithmic cost contributions from downstream-MTTR cost contributions, since opportunity cost (revenue forgone during MTTR) typically dominates the dollar-magnitude scale and could otherwise obscure token- and infrastructure-cost effects from algorithmic mechanisms. Time to Triage (TTT) is the elapsed time from anomaly detection to publication of the MAHC PROVISIONAL verdict. Hallucination rate is decomposed into structural (H_struct) and factual (H_fact) components, with independent-rater labeling against pre-specified rubrics and Cohen's $\kappa \geq 0.7$ required for inter-rater agreement.

Additional descriptive metrics — HR@10, mean reciprocal rank (MRR), mean first rank (MFR), precision, recall, macro F1, HITL escalation rate, sampling-regime stratification, hardware-isolation-tier stratification, retraining overhead in CPU-hours and GPU-hours, ethics-compliance rate, and the Reward Reconciliation Error on the pre-registered 25 percent delayed-label subset —

will be reported alongside the confirmatory set. The FAE-Score for narrative explanation quality, computed as the arithmetic mean across three judge models (GPT-4o, Claude Sonnet, and Gemini Pro) with 10 percent of judgments human-anchored for calibration, will accompany the Research Question 4 user study. These descriptive metrics support interpretability and secondary exploratory analysis; they are not subject to pre-registered hypothesis tests and do not enter the Holm–Bonferroni correction family.

3.10 Validity and Reliability

3.10.1 Internal Validity

Eight internal-validity threats are identified, each addressed by an explicit architectural or procedural mitigation that substantially strengthens — but does not eliminate — the corresponding internal-validity exposure.

Threat 1 — Input drift between variants. Because each variant will be produced from the same HELIOS binary by feature-flag toggling, and the snapshot hash registry will enforce that every active pipeline consumes the same content-hashed UEG-C snapshot for the same (fault, seed) pair, inter-variant variance arises only from the pipelines themselves. The paired-observation design will satisfy its structural precondition under this architectural enforcement.

Threat 2 — Hidden coupling between feature flags. The original `p4_cognitive` flag silently removed cognitive nodes from G-pipe's input; the resolution scopes the flag to L-pipe visibility through Persona-Profile tool-permission restriction. A hidden-coupling audit conducted at design freeze confirms that the remaining eleven flags toggle disjoint code paths. The audit certifies

disjoint code paths under single-flag toggling; it does not certify non-interacting semantic effects under arbitrary flag combinations. The chapter's confirmatory protocol ablates each contribution against HELIOS-Full singly, not in combination, so the joint-inactivation interaction surface is not exercised in the confirmatory family. Interaction-effect probes across selected flag combinations are identified as a direction for follow-on exploratory analysis.

Threat 3 — Cross-incident learning dependency in ORAR. This would violate the within-pair independence assumption if LinUCB parameters were updated on-line during the confirmatory protocol. The mitigation is the static-policy commitment (Section 3.3.3 and Section 3.6.5): LinUCB and ACP parameters will be frozen at the pre-warmed state for the entire confirmatory protocol, on-line updates restricted to the disjoint exploratory delayed-label subset, and freezing enforced at the binary level by the Variant Control Layer. The static-policy commitment narrows confirmatory inference for C6 to the cost-aware static-routing mechanism; on-line learning dynamics are evaluated only as exploratory evidence.

Threat 4 — LLM hallucination biasing the L-pipe verdicts. Hallucination is addressed at three architectural layers: at the telemetry layer, the P4 cognitive pillar captures hallucination candidates through tool-call schema-validation failures and through semantic drift between chain-of-thought and structured output; at the reasoning layer, the Cognitive Sentinel fires on detected pathological patterns; and at the consensus layer, the MAHC entropy regulariser tests for overconfident unanimous agreement and escalates to HITL rather than auto-executing.

Threat 5 — Reward hacking in the ORAR learning loop. Relevant only to the exploratory subset where on-line learning is enabled. Three anti-hacking regularisers will be applied during exploratory analysis: a variance penalty across similar incidents, an action-diversity bonus, and a held-out audit in which 5 percent of incidents are double-diagnosed by an independent agent for reward validation. Reward hacking is not a confirmatory threat because the confirmatory protocol freezes the learning loop.

Threat 6 — Control-plane failure during evaluation. When the Internal Cognitive Sentinel detects degradation in HELIOS's own LLM agents, the Deterministic Fallback Agent will activate using hand-coded SOPs for approximately 30 percent of common fault classes; remaining fault classes will degrade to alerting-only mode pending HITL intervention. The system cannot silently fail during evaluation, and control-plane degradation events are themselves logged as first-class covariates through the IOL mechanism.

Threat 7 — Adversarial input during telemetry ingestion. Adversarially-crafted input is contained by the Privacy Gateway and the Adversarial Robustness Monitor. Adversarial events are recorded as first-class CoE entries; incidents involving adversarial events will be stratified in analysis and will not contribute to confirmatory inference on the unperturbed cells.

Threat 8 — Opacity of the pre-warmed state. Discussed extensively in Section 3.3.3, where the disclosure regime — pre-registration of construction parameters, hash of the construction script, hash-verified disjointness at the (fault-type, service-family, injection-parameter) granularity, content-addressable parameter artifacts, runtime hash verification by the Variant Control

Layer — is specified. The residual exposure has two named components: structural similarity at coarser conceptual levels (shared HTTP semantics, anomaly-class taxonomy, OTEL conventions) that hash-disjointness at the instance level cannot exclude; and the construction-validity threat that the synthetic testbed and construction script were designed by the same researcher who designed HELIOS. The residual structural-similarity exposure is bounded by the small dimensionality of the LinUCB feature space (approximately 12 to 18 features, comprising incident features and the ACP stateful-action feature block) relative to the diversity of the confirmatory corpus, but neither this bound nor the disclosure regime fully dissolves the threat. External replication on a structurally distinct microservice paradigm — for example, an event-driven serverless mesh in place of a request-response Kubernetes deployment — would substantially strengthen the warrant beyond what the present disclosure regime achieves, and is identified in Section 3.11.5 as a priority direction for follow-on research.

3.10.2 Construct Validity

The primary construct-validity reliability assurance mechanism is architectural traceability: every construct C1 through C6 corresponds to a named architectural component, every independent variable is instantiated through a feature flag at the Variant Control Layer (or, for C1, through architectural commitment to evaluation-rigor infrastructure), and every dependent variable is measured at a specific architectural location with specified instrumentation (Section 3.3.6). This traceability substantially strengthens the reliability of construct measurement relative to comparator AIOps evaluations, without claiming to strengthen the validity of the operationalization in the stronger theoretical sense; the choice of architectural binding is itself a theoretical choice that determines which construct the measurement reflects.

Three secondary construct-validity threats remain. First, FAE-Score judge-model instability is mitigated by the three-judge ensemble plus periodic human-anchored calibration at 10 percent sample. Second, the SRE Likert trust scale in the RQ4 user study is a self-reported measure potentially subject to social-desirability bias; it will be triangulated with the objective HITL rejection counter, which is architecturally logged and cannot be modified by participants. Third, the Tier 4 cognitive-agentic faults used in the C2 exploratory empirical validation are constructed by the researcher specifically through NIKA-style instrumentation to exercise cognitive-node reasoning; they may not reflect the population of cognitive-agentic faults that would arise independently in deployed agentic systems. This threat motivates the reclassification of the C2 cognitive-agentic test as exploratory under E-H1 (Section 3.6.8 and Section 3.8.2).

3.10.3 External Validity

External validity is addressed at three levels. First, the evaluation will span five public benchmarks with heterogeneous microservice architectures and fault classes (AIOpsLab, DeathStarBench, Train-Ticket, PetShop, OpsEval), providing breadth of coverage to the extent that the corpus distribution permits. The 174-fault corpus is concentrated in the AIOpsLab benchmark family (approximately 76 percent from Tier 1 and Tier 2; Section 3.5). Confirmatory external-validity claims are therefore bounded to AIOpsLab and its parametric extension. Cross-benchmark results from Tier 3 (Train-Ticket and DeathStarBench under ChaosBlade injection) and Tier 4 (synthetic cognitive-agentic faults layered on AIOpsLab) are positioned as external-validity probes rather than as confirmatory cross-benchmark generalization: they will suggest that observed effects extend beyond the AIOpsLab family, but the per-benchmark sample sizes are insufficient for adequately powered between-benchmark inference. Generalization to environments outside AIOpsLab and its

parametric extension — for example, environments instrumented with non-OTEL telemetry, with substantially larger service counts, with proprietary observability stacks, or operating under fault regimes absent from the AIOpsLab fault taxonomy — would require an industrial replication study beyond the scope of this dissertation.

Second, the four-tier corpus augmentation provides triangulation across injection tools (Chaos Mesh, ChaosBlade, NIKA-style instrumentation) and fault taxonomies, supporting the secondary claim that observed effects will not be artifacts of a single fault-injection tool's failure mode; it does not support a claim of population-level generalization. Third, HELIOS's five core assumptions A1 through A5 (Section 3.11.1) bound the population to which findings generalize; assumption violations are disclosed rather than hidden. The external-validity claim of the study is therefore that confirmatory external validity holds for AIOpsLab and its parametric extension under the five core assumptions, with cross-benchmark results in Tiers 3 and 4 reported as external-validity probes that motivate, but do not constitute, broader generalization.

3.10.4 Reliability

Reliability will be assured through four commitments. First, all primary hypotheses will be pre-registered at OSF with a time-stamped freeze that binds the analysis plan; the freeze date and OSF deposit DOI will be recorded in the deviation log alongside this chapter. Second, all benchmark datasets, baseline-system configurations, and fault-instance hashes will be published at pre-registration, supporting independent replication. Third, the artifact specification in this chapter is at a level of detail sufficient for independent replication, and the feature-flag matrix, the algorithm specifications, and the metric-integrity-gate specification are binding contracts. Fourth, the CoE ledger's append-only, cryptographically-signed, Conflict-free Replicated Data Type (CRDT)-

replicated design will ensure that decision records produced during evaluation are auditable and tamper-evident.

Reproducibility will be pipeline-scoped, protocol-scoped, and tiered, in accordance with the bounded-reproducibility principle specified in Section 3.6.2. D-pipe and G-pipe will be reproducible to the level of bitwise output equality under the binding hardware and software configuration documented in Section 3.6.2. L-pipe will be reproducible at the level of greedy-decoded token-sequence determinism under Protocol A (Llama-3.1-70B-Instruct served via vLLM 0.6.x with `enforce_eager=True`, `disable_custom_all_reduce=True`, single-replica serving, temperature = 0, top-p = 1, top-k = 1); container digest, weights SHA-256, and tensor-parallel-degree configuration are deposited at OSF. Logit-level variance ε under Protocol A will be empirically measured on a 200-prompt probe set during the Chapter Four pilot pass and reported in Chapter Four with its standard error and measurement date; the measurement is conducted prior to chapter submission so that ε is a delivered empirical result at viva. Protocol B (commercial API access) will be reported as a sensitivity analysis. Any variant replicate that fails token-sequence equality under Protocol A will be logged as a deviation-log event and will not contribute to confirmatory inference.

A model-selection sensitivity analysis will defend against the threat that Protocol A results would not replicate under an alternative open-weight model family. A subset of the confirmatory ablation protocol will be repeated on Llama-3.1-70B-Instruct (binding) and Qwen-2.5-72B-Instruct (binding contrast family). Qwen-2.5-72B-Instruct is selected as the contrast family rather than Mixtral-8x22B-Instruct because its dense decoder-only architecture and 72-billion parameter count are closer to Llama-3.1-70B-Instruct's architecture and parameter count than Mixtral-8x22B's mixture-of-experts design, providing a more comparable contrast for the pattern-of-effects

invariance question. Both model identifiers will be content-hashed and deposited at OSF. Absolute scores are not expected to be invariant across families; the sensitivity analysis is designed to demonstrate that the ablation conclusions will not be artifacts of a single model choice.

Every artifact-identifier reference in this chapter — the binding hardware configuration, the binding model identifiers, the container digests, the frozen LinUCB and ACP parameter artifacts, the synthetic pre-training corpus and its construction script, the deviation-log file, and the GLMM specification — will be deposited at OSF as a content-addressable artifact at chapter submission. The Variant Control Layer will verify the hash of each loaded artifact against the hash recorded at OSF freeze at every confirmatory run start; hash divergence will block execution.

3.11 Ethical Considerations

The research will be conducted under the ethical standards of the researcher's institutional review board, applicable data-protection regulations, and the responsible-AI-research practices appropriate to a study whose artifact has potential autonomous-action capability.

Telemetry data handling is governed by the Privacy Gateway at the Data-Plane-to-Control-Plane boundary. Pattern-based PII detection and named-entity recognition for free-text log content will redact sensitive tokens before any signal reaches the Control Plane; redacted tokens will be replaced with deterministic placeholders that preserve grammatical structure for downstream LLM parsing. Per-tenant differential-privacy budgets will cap aggregated-query information leakage, and multi-tenant isolation within the UEG-C is enforced at the graph-partition level. Raw telemetry collected in any industrial-replication extension will remain within the industrial partner's infrastructure; only anonymized aggregate metrics will cross institutional boundaries.

The ethics of autonomous AI decision-making are addressed architecturally. The Monitor Agent will enforce ethics constraints across every automated action and record ethics-audit results as first-class CoE entries. The dynamic autonomic threshold α classifies all actions into risk tiers; high-risk actions (database modifications, network-policy changes, rollbacks) will unconditionally require human approval via the HITL SRE Gate. No autonomous remediation will be executed against production telemetry without prior validation in the FGSV shadow twin. These mechanisms are architectural first-class components, not post-hoc compliance additions.

Human participants in the RQ4 user study and any industrial-replication case study will participate voluntarily, with full disclosure of the study's purpose, the nature of the AI system under evaluation, and the data collected. Informed consent will be obtained in writing prior to any data collection, and participants may withdraw at any time without penalty. Participant data (Likert responses, override counts, time-to-decision measurements) will be anonymized at collection; only aggregate statistics will enter the dissertation. The user-study protocol, recruitment materials, consent forms, and debriefing script (Section 3.7) will be submitted to the researcher's institutional ethics review board for approval prior to data collection; IRB approval status will be recorded in the deviation log at OSF freeze. Participant data will be stored in accordance with applicable regulations including GDPR where relevant.

Responsible AI research practice is operationalized at the level of the artifact design itself: the Monitor Agent, HITL gate, Adversarial Robustness Monitor, Privacy Gateway, and FGSV shadow validator are architectural first-class components rather than review-stage compliance additions. Who sets the risk threshold, who may approve a high-risk change, and who reviews overrides and spend is specified in the business evaluation layer. Legal liability for a bad automatic

change remains outside the study. Power asymmetries that would follow from running the artefact at scale, beyond those decision rights, remain a direction for follow-on work.

3.12 Limitations of the Methodology

3.12.1 Core Assumptions and Their Scope Conditions

Five core assumptions bound the scope of findings; where they are violated, conclusions should be applied with caution.

A1 — OTEL instrumentation. Target microservices and agent platforms are instrumented (or instrumentable via sidecars) to emit OTEL-compliant signals. Performance in partially-instrumented environments is not evaluated.

A2 — Dedicated control-plane compute. An enterprise cluster overhead budget of 5 to 10 percent is available for the HELIOS Control Plane. HELIOS can operate in reduced-capability mode for resource-constrained environments, but the full multi-pipeline architecture requires enterprise-scale resources and reduced-capability mode is not the primary subject of evaluation.

A3 — Initial accuracy-calibration data. Bootstrap accuracy priors for the MAHC hierarchical-prior mechanism and the ORAR LinUCB arm parameters are seeded from public benchmark performance and pre-production fault-injection campaigns. Findings should not be extrapolated to organizations without a 30-day HITL calibration phase.

A4 — LLM access tier. Full P4 cognitive telemetry — a primary component of Contribution C2 — requires at minimum Tier 2 (logprob-API) LLM access. Tier 3 deployments receive reduced cognitive signal quality and P4-dependent findings may not replicate under Tier 3 conditions.

A5 — Historical incident data. The Knowledge Cortex warm tier requires seed incident records for retrieval-augmented-generation-based SOP matching. Cold-start organizations face reduced SOP-matching accuracy for approximately 90 days post-deployment.

3.12.2 Corpus and Evidential-Status Limitations

L1 — Benchmark concentration. Approximately 76 percent of the 174-fault corpus is drawn from the AIOpsLab benchmark family (Section 3.5). Confirmatory inferences are bounded to AIOpsLab and its parametric extension (Section 3.9.3); cross-benchmark results in Tiers 3 and 4 will be reported as external-validity probes rather than as confirmatory cross-benchmark generalizations. Generalization to non-AIOpsLab environments requires an industrial replication study identified as a priority direction for follow-on research.

L2 — Cognitive-agentic fault evaluation. The empirical evaluation of Contribution C2 on agentic-AI fault classes is constrained by the synthetic construction of the Tier 4 corpus and the limited sample size (n = 12), which is below the 73-fault statistical minimum required for adequately powered confirmatory inference. Tier 4 instances are constructed by the researcher through NIKA-style instrumentation specifically to exercise cognitive-node reasoning, raising a construct-validity concern that Tier 4 may not sample a population of cognitive-agentic faults arising independently

outside the construction protocol. The Tier 4 substrate is layered onto AIOpsLab rather than constructed independently, so the E-H1 evidence base is bounded to "AIOpsLab plus cognitive-agentic instrumentation"; an externally constructed cognitive-agentic fault corpus on a non-AIOpsLab substrate is the priority direction for upgrading E-H1 to confirmatory status.

L3 — Architectural contributions with exploratory empirical validation. Contributions C3′ (FGSV) and C5 (IOL) are positioned as architectural contributions with exploratory empirical validation rather than as confirmatorily evidenced contributions on the present corpus. The empirical validation of FGSV requires synthetic fidelity-break perturbations and the empirical validation of IOL requires injected control-plane degradation, neither of which arises spontaneously from the public benchmarks. Future confirmatory pre-registrations with pre-specified effect sizes for these synthetic perturbation protocols are identified as the appropriate path to upgrading C3′ and C5 from exploratory to confirmatory empirical evidence.

3.12.3 Technical Limitations

Five technical limitations further constrain the scope of claims. HELIOS will treat each incident as independent; cascade failures manifesting as multiple simultaneous alerts will spawn parallel diagnostic episodes rather than a single correlated investigation, with multi-incident correlation scoped as future work. The stratified fault-injection curriculum covers known fault classes but does not provide a formal coverage measure over the unknown fault surface; few-shot adaptation claims are conditional on curriculum coverage of the target fault class. The FGSV ASRE WAL-replay mode introduces 5 to 30 seconds of additional latency for stateful remediations depending on WAL size; reported MTTR for stateful fault classes will reflect this overhead. Hot-reloadable

Persona Profiles create a governance race condition during active incident episodes; profile refresh is restricted to episode boundaries. The FAE-Score multi-judge ensemble mitigates single-judge bias but remains subject to judge-model drift; periodic human-anchored calibration at 10 percent sample detects drift but cannot prevent it.

3.12.4 Methodological Trade-offs

The research design embeds seven principal methodological trade-offs, each with a documented implication for the scope of findings. Latency versus accuracy is resolved by Fast-Path PROVISIONAL at 30 seconds and CONFIRMED on full quorum, with MTTR including PROVISIONAL time. Cost versus quality is resolved by ORAR cascade plus ACP plus Non-LLM-First Gate; CpR results are configuration-dependent. Twin fidelity versus speed is resolved by the three-mode ASRE; MTTR for stateful incidents exceeds MTTR for routing incidents. Coverage versus overhead is resolved by Tiered Adaptive Sampling with backpressure-aware downgrade; under extreme resource pressure rare brief faults may be missed. Autonomy versus safety is resolved by the dynamic α guardrail and mandatory HITL for high-risk actions, with HITL latency not experimentally controlled. Generalization versus depth is resolved by five benchmarks with eight baselines and a priori power analysis; class-specific performance variance may exceed aggregate effect-size estimates. *Confirmatory rigor versus on-line learning* is resolved by the static-policy commitment for ORAR (Section 3.3.3, Section 3.6.5): the confirmatory protocol freezes LinUCB and ACP parameters, narrowing confirmatory inference for C6 to the cost-aware static-routing mechanism, while on-line learning dynamics are reserved for exploratory analysis.

3.12.5 Scalability and External-Replication Boundaries

The 150-millisecond Plane 2 latency SLO and the 10,000-edge per-incident subgraph bound are design commitments validated on microservice estates of at most approximately 100 services within the benchmark regime. Scaling to 500, 1,000, 2,500, and 5,000 synthetic services is a pre-registered separate experiment; the scalability claim is to be earned on measured results, not asserted as an architectural property. Multi-region deployments require CRDT CoE replication and may require hierarchical Knowledge Cortex caching to maintain query latency.

The external validity of qualitative and longitudinal findings would be constrained by the use of a single industrial replication site, if such a site is engaged. Findings specific to organizational dynamics of adoption (SRE trust trajectories, override-rate decline patterns, integration effort) may not generalize to organizations with substantially different operational cultures, service portfolios, or AI maturity levels. Replication studies across multiple industrial sites — and external replication of the static-policy pre-warming pass on a structurally distinct microservice paradigm (for example, an event-driven serverless mesh in place of a request-response Kubernetes deployment) — are identified as priority directions for follow-on research. These are the principal interventions that would substantially strengthen the warrant for the construction-validity threat (Section 3.3.3, Threat 8 of Section 3.9.1) beyond what the present disclosure regime achieves.

3.13 Summary

This chapter has specified the methodology of a Design Science Research study whose central methodological contribution is the runtime enforcement of pre-registration discipline through architectural mechanisms. The Variant Control Layer, snapshot hash registry, metric integrity gate,

exclusion ledger, deviation log, and reconciliation ledger collectively constitute Contribution C1 — a research-method-as-system-architecture contribution that transforms DSR evaluation rigor from a reporting convention into a runtime invariant of the artifact, and that is generalizable beyond HELIOS to AIOps research-method practice and arguably to DSR research-method practice more broadly.

Within this methodological frame, six named contributions are advanced: C1 (runtime-enforced evaluation rigor), C2 (UEG-C with cognitive nodes), C3′ (FGSV evaluation-infrastructure discipline), C4 (MAHC uncertainty-aware consensus), C5 (IOL meta-observability), and C6 (ORAR cost-aware adaptive routing with cross-field-transfer profile from delayed-feedback bandits). The five design principles — pipeline independence, ablation-first design, bounded reproducibility, separation of diagnosis from action, and failure-as-covariate discipline — are intended as transferable design knowledge contributions in the (Gregor & Hevner, 2013) sense.

The research philosophy grounds the work in critical realism with a pragmatist orientation toward artifact evaluation. HELIOS is specified as an experimental artifact at a level of detail sufficient to justify the construct-to-variable mapping, the data-collection instruments, and the planned hypothesis tests. The data-analysis plan pre-registers two confirmatory hypothesis families corrected separately under Holm–Bonferroni with pre-registered rank ordering — an eight-hypothesis ablation family testing the internal contribution of each architectural mechanism, and an eight-hypothesis baseline-comparison family testing external comparative advantage — together with a ten-hypothesis exploratory family. Effect sizes are framed under minimum-detectable-effect logic at the delivered corpus: the protocol detects Cohen's $h \geq 0.276$ at $n = 174$, one-sided $\alpha = 0.00625$, $\rho = 0.5$, 80 percent power, corresponding to a 13 percentage-point HR@3 improvement against the AIOpsLab-equivalent CHASE baseline of 0.60.

The central methodological commitment is that the architecture and the evaluation will be coupled by design under documented constraints rather than by-construction guarantees. The Class 1 evaluation-rigor apparatus — the components that constitute Contribution C1 — substantially strengthens the warrant for confirmatory inference under the bounded scope conditions of the study without claiming to dissolve every threat to validity. The chapter is honest about residual exposures: the construction-validity threat from researcher-designed pre-warming corpus is bounded but not dissolved; the AIOpsLab corpus concentration bounds external validity to AIOpsLab and its parametric extension; the n = 12 Tier 4 cognitive-agentic corpus supports an existence-proof for C2 cognitive-agentic claims rather than a population-efficacy claim; and A-H4 and A-H8 are confirmatorily retained under an asymmetric inferential rule that treats non-rejection as 'inconclusive at delivered power' rather than 'no support.' Each residual exposure is mapped to a priority follow-on study identified in Section 3.11. Chapter Four will present the evaluation results obtained under this methodology.

A Appendix

A.1 Appendix A — Acronyms used in this chapter

Acronym	Expansion
ACP	Action-Class Predictor
ARM	Adversarial Robustness Monitor
ASRE	ASRE mode (FGSV execution mode taxonomy)
BCa	Bias-corrected and accelerated (bootstrap)
CBR	Contextual Bandit Router (LinUCB-based ORAR component)
CoE	Chain-of-Evidence (ledger)
CpR	Cost per Resolution
CRDT	Conflict-free Replicated Data Type
DSR	Design Science Research
FAE-Score	FAE-Score (multi-judge narrative-quality ensemble)
FGSV	Fidelity-Gated Shadow Validation
GLMM	Generalised Linear Mixed Model
HIS	Hardware-isolation tier
HITL	Human-in-the-Loop
HR@k	Hit Rate at k
IOL	Internal Observability Loop
KS	Kolmogorov–Smirnov (test)
MAHC	Multi-Agent Hybrid Consensus

Acronym	Expansion
MAML-Quick	MAML-Quick (lightweight retraining variant)
MCP	Model Context Protocol
MTTR	Mean Time to Resolution
NIKA	NIKA (cognitive-fault instrumentation protocol)
ORAR	Operational-Reality-Aware Adaptive Routing
OSF	Open Science Framework
OTEL	OpenTelemetry
RCA	Root Cause Analysis
RRE	Reward Reconciliation Error
SEAN	SEAN (versioned remediation-template library)
SOP	Standard Operating Procedure
SRE	Site-Reliability Engineer
TTD	Time to Detect
TTT	Time to Triage
UEG-C	Unified Evidence Graph with Cognitive Nodes
VCL	Variant Control Layer

A.2 Appendix B — Reproducibility and Pre-Warming Artefacts

A.2.1 Reproducibility and Pre-Warming Artefacts

This appendix specifies the binding artefact identifiers, version locks, configuration parameters, and content hashes that support the reproducibility commitments of Chapter 3. All hashes are computed at OSF freeze and recorded in the OSF deposit alongside the chapter; any deviation between a recorded hash and the hash of a loaded artefact at runtime blocks execution of the confirmatory protocol via the Variant Control Layer hash check (Section 3.3.3).

Binding Hardware Configuration. The confirmatory protocol executes on the following hardware configuration. Any deviation is recorded in the deviation log.

Component	Specification
GPU	NVIDIA A100 80GB SXM4
GPU count per HELIOS-Full run	4 (tensor-parallel degree = 4)
CUDA driver	535.x (specific minor version locked at OSF freeze)
CUDA toolkit	12.1
cuDNN	8.9 (deterministic mode enabled)
Host CPU	AMD EPYC 7763 (64-core) or Intel Xeon Platinum 8480 (equivalent)
Host RAM	512 GiB
Network fabric	InfiniBand HDR 200 Gb/s

PyTorch and cuDNN Determinism Flags. The following flags are set at process start for D-pipe and G-pipe execution:

```
torch.use_deterministic_algorithms(True)
torch.backends.cudnn.deterministic = True
torch.backends.cudnn.benchmark = False
CUBLAS_WORKSPACE_CONFIG=:4096:8
```

For L-pipe execution under Protocol A, the additional vLLM flags are:

```
enforce_eager = True
disable_custom_all_reduce = True
tensor_parallel_size = 4
gpu_memory_utilization = 0.90
```

Binding Model Identifiers.

Role	Model Version	Container	Digest	Weights SHA-256
L-pipe primary (Protocol A)	Llama-3.1-70B-Instruct	meta-llama/Llama-3.1-70B-Instruct (HuggingFace)	[vllm/vllm-openai:0.6.x@sha256:...]	[recorded at freeze]

Role	Model Version	Container	Digest	Weights SHA-256
L-pipe contrast family (sensitivity analysis)	Qwen-2.5-72B-Instruct	Qwen/Qwen2.5-72B-Instruct (HuggingFace)	[vllm/vllm-openai:0.6.x@sha256:...]	[recorded at freeze]
Inference engine	vLLM	0.6.x (specific patch version locked)	[as above]	—

The contrast family Qwen-2.5-72B-Instruct is selected over Mixtral-8x22B-Instruct because its dense decoder-only architecture and 72-billion parameter count are closer to Llama-3.1-70B-Instruct than Mixtral-8x22B's mixture-of-experts design (Section 3.9.4). Bracketed values are populated at OSF freeze.

L-Pipe Decoding Configuration (Protocol A).

Parameter	Value
Temperature	0
Top-*p*	1
Top-*k*	1
Max output tokens	4096

Parameter	Value
Random seed	Fixed per (incident, seed) pair from the pre-registered seed register

Logit-Level Variance ε Probe-Set Methodology and Result. A 200-prompt probe set is constructed by sampling 200 fault narratives uniformly without replacement from the 174-fault corpus (with 26 prompts drawn from the synthetic delayed-label exploratory subset to fill the probe-set count). Each prompt is submitted to the Protocol A serving stack twice under identical configuration; per-token logit vectors are recorded at each position. The logit-level variance ε is reported as the mean across the 200 prompts of the per-prompt L2 distance between the two logit-vector sequences, normalised by sequence length.

Metric	Value	Standard Error	Date Measured
ε (mean per-token logit L2 distance)	[to be measured during Chapter Four pilot pass]	[to be measured]	[to be measured]

Metric	Value	Standard Error	Date Measured
Token-sequence equality rate (proportion of prompts with bit-identical token output across both runs)	[to be measured]	—	[to be measured]
Probe-set construction script SHA-256	[recorded at freeze]	—	—

The probe-set run is conducted on the binding hardware configuration (B.1) under the binding L-pipe decoding configuration (B.4) prior to chapter submission. If the pilot pass has not completed at submission for any reason, the affected reliability claim is downgraded per the deviation-log discipline specified in Section 3.6.2.

Pre-Warming Algorithm Pseudocode. Algorithm B.1: ORAR LinUCB and ACP Pre-Warming

```
Input: Synthetic pre-training corpus C_pre = 1,200 incidents on the
       three-service synthetic testbed (Section 3.3.3)
       Pre-registered seed sequence S_pre = (s_1, ..., s_1200)
       LinUCB exploration coefficient  _LinUCB
```

```
       ACP optimiser configuration (Adam, lr = 1e-4, batch_size = 32,
       max_epochs = 50, early_stop_patience = 5)
Output: Frozen LinUCB per-arm parameters {(A_a, b_a)} for
        a  {D, G, L, D+G, D+L, G+L, Full}
        Frozen ACP classifier weights W_ACP


# Algorithm: Training and Freezing the Cost-Aware Router with ACP

1.  Initialise A_a ← I_d for each arm a
   # d = feature dimension

2.  Initialise b_a ← 0_d for each arm a

3.  Initialise W_ACP ← Xavier uniform

4.  Split C_pre into:
       C_train = 80% (n = 960)
       C_val   = 20% (n = 240)
   using stratification on fault class

5.  for k = 1 to 960 do
```

```
6.      incident_k ← C_train[k] under seed s_k

7.      Construct the UEG-C subgraph for incident_k
        using the K-hop pruner

8.      Extract incident features x_k
        augmented with ACP output

9.      Select arm a_k via LinUCB on x_k
        under thawed updates

10.     Execute pipelines associated with arm a_k
        and observe reward r_k

11.     Update A_{a_k}:
            A_{a_k} ← A_{a_k} + x_k x_k^T

12.     Update b_{a_k}:
            b_{a_k} ← b_{a_k} + r_k · x_k

13.     Append (incident_k, ground-truth remediation class)
        to the ACP buffer
```

```
14. end for

15. Train W_ACP on the ACP buffer
    using early stopping on C_val

16. Compute SHA-256 hashes of:
        - Serialised {(A_a, b_a)}
        - W_ACP

17. Record the hashes in the variant manifest
    at OSF freeze

18. return frozen state
```

The pass is bit-deterministic given C_pre, S_pre, α_LinUCB, the ACP optimiser configuration, and the binding hardware (B.1). The hash recorded at line 16 is verified by the Variant Control Layer at every confirmatory run start.

Frozen LinUCB Parameter Summary.

Arm a	**Diagonal of** A_a **(Mean)**	**Diagonal of** A_a **(Max)**	$\|\mathbf{b}_a\|$ □	**Selection Rate on** C_val
D	[recorded at freeze]	[recorded]	[recorded]	[recorded]

Arm a	Diagonal of A_a (Mean)	Diagonal of A_a (Max)	$\|b_a\|$	Selection Rate on *C_val*
G	[recorded]	[recorded]	[recorded]	[recorded]
L	[recorded]	[recorded]	[recorded]	[recorded]
D+G	[recorded]	[recorded]	[recorded]	[recorded]
D+L	[recorded]	[recorded]	[recorded]	[recorded]
G+L	[recorded]	[recorded]	[recorded]	[recorded]
Full	[recorded]	[recorded]	[recorded]	[recorded]

The full per-arm matrices and vectors are deposited at OSF as content-addressable artefacts; this table reports summary statistics for examiner-level inspection.

Frozen ACP Classifier Weight Summary.

Output Class	Softmax Marginal on *C_val*	Top-3 Feature Coefficient Norms
Schema change	[recorded at freeze]	[recorded]
Data mutation	[recorded]	[recorded]
Config change	[recorded]	[recorded]

The full weight tensor is deposited at OSF; this table reports summary statistics.

Pre-Training Corpus Construction Protocol Hash.

Artefact	SHA-256
Synthetic testbed Helm chart	[recorded at freeze]
Fault-injection construction script	[recorded]
Pre-registered fault-class proportion specification	[recorded]
Pre-registered severity-distribution specification	[recorded]
Pre-registered injection-parameter range specification	[recorded]
Random seed sequence *S_pre*	[recorded]
Resulting pre-training corpus *C_pre*	[recorded]

The construction protocol hashes are computed and recorded prior to corpus generation, per the construction-validity disclosure in Section 3.3.3. The corpus hash at line 7 is computed after generation and verifies the deterministic relationship between the construction protocol and the resulting corpus.

Disjointness Verification Hash.

Verification	SHA-256 of Result
Intersection of (fault-type, service-family, injection-parameter) tuples between *C_pre* and the 174-fault confirmatory corpus	[recorded; expected null]

Verification	SHA-256 of Result
Intersection of (fault-type, service-family, injection-parameter) tuples between *C_pre* and the 14,000 paired (variant, HELIOS-Full) observations comprising the seven non-Full ablation variants × 5 benchmarks × 40 faults × 10 seeds (drawn from the 16,000-run total per Section 3.5)	[recorded; expected null]

A non-null intersection at either row vacates the static-policy commitment per Section 3.3.3 and triggers the deviation-log entry that reclassifies the affected confirmatory inferences as exploratory.

GLMM Specification (Tier 1 Sensitivity Analysis). The generalised linear mixed-effects model reported on Tier 1 (Section 3.5; Section 3.8.3) is specified as:

```
y_ij =  _0 +  _variant * variant_ij +  _benchmark * benchmark_ij + u_problem_id[i] +  _ij
```

where:

- y_ij = outcome metric for incident i under condition j
- variant_ij □ {HELIOS-Full, HELIOS-noX} (paired)
- benchmark_ij □ {AIOpsLab replica 1, AIOpsLab replica 2}
- u_problem_id[i] ~ N(0, σ²_problem) (random intercept)

- $\varepsilon_{ij} \sim N(0, \sigma^2_\varepsilon)$ (residual; unstructured covariance for seed-paired observations within problem_id)
- Estimation: REML (restricted maximum likelihood)
- Degrees of freedom: Kenward–Roger
- Software: lme4 1.1-x with lmerTest, R 4.x (specific versions locked at OSF freeze)

The GLMM is a sensitivity analysis for the ablation family; the binding confirmatory test is Wilcoxon signed-rank per Section 3.8.3.

Deviation Log Schema. The deviation log is an append-only, cryptographically-signed JSON-Lines file mirroring the CoE ledger design (Section 3.6.4). Each entry has the schema:

```
{
  "timestamp_utc": ISO-8601 string,
  "commit_sha": string (HELIOS binary commit at deviation),
  "clause": string (Chapter 3 section reference, e.g., "§3.6.5"),
  "change": string (concrete description of deviation),
  "reason": string (justification),
  "analytic_consequence": string (typically: "hypothesis X moves
                                  from confirmatory to exploratory"),
  "signature": string (Ed25519 signature over the preceding fields)
}
```

Write access is restricted to the candidate; read access is public via OSF. The log is deposited at OSF at every commit and is timestamped at deposit. Entries are written before any analysis benefiting from the deviation is conducted.

References

Adeba, J. L., Kim Deuk-Hun, & Kwak, J. (2025). Temporal Decay Loss for Adaptive Log Anomaly Detection in Cloud Environments. *Sensors*, *25*(9), 2649. Publicly Available Content Database (3203224519). https://doi.org/10.3390/s25092649

Ahmad, A., Li, P., Piechocki, R., & Inacio, R. (2025). Anomaly Detection in Offshore Open Radio Access Network Using Long Short-Term Memory Models on a Novel Artificial Intelligence-Driven Cloud-Native Data Platform. *Engineering Applications of Artificial Intelligence*, *161*, 112274. https://doi.org/10.1016/j.engappai.2025.112274

Aktaş, K., & Kilinc, H. H. (2024). Interaction Prediction and Anomaly Detection in a Microservices-based Telecommunication Platform. *Proceedings of the 2024 International Conference on Software and Systems Processes*, 56–65. https://doi.org/10.1145/3666015.3666017

Almajed, H., Alsaqer, A., & Albuali, A. (2025). Towards Effective Anomaly Detection: Machine Learning Solutions in Cloud Computing. *International Journal of Advanced Computer Science and Applications*, *16*(2). https://doi.org/10.14569/IJACSA.2025.01602132

An, K., Yang, F., Lu, J., Li, L., Ren, Z., Huang, H., Wang, L., Zhao, P., Kang, Y., Ding, H., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). *Nissist: An Incident Mitigation Copilot based on Troubleshooting Guides* (arXiv:2402.17531). arXiv. https://doi.org/10.48550/arXiv.2402.17531

Araujo, I., Antunes, N., & Vieira, M. (2023). Intrusion Detection and Tolerance for Microservice Applications. *12th Latin-American Symposium on Dependable and Secure Computing*, 176–181. https://doi.org/10.1145/3615366.3622794

Bansal, C., Renganathan, S., Asudani, A., Midy, O., & Janakiraman, M. (2020). DeCaf: diagnosing and triaging performance issues in large-scale cloud services. *Proceedings of the ACM/IEEE 42nd International Conference on Software Engineering: Software Engineering in Practice*, 201–210. https://doi.org/10.1145/3377813.3381353

Boesel, F., Schläpfer, L., Pozidis, H., & Gusat, M. (2023). Multivariate Anomaly Detection with Domain Clustering. *Proceedings of the 2023 ACM Symposium on Cloud Computing*, 193–199. https://doi.org/10.1145/3620678.3624655

C. Gao, L. Shi, H. Sun, T. Ma, Y. Chen, Q. Pei, & Y. Chen. (1 Aug.1, 2025). An Efficient Anomaly Detection Model Based on Tensor Decomposition and VARIMA for High-Dimensional Multivariate Time Series. *IEEE Internet of Things Journal*, *12*(15), 29443–29459. https://doi.org/10.1109/JIOT.2025.3570876

Chakraborty, S., Agarwal, S., Garg, S., Sethia, A., Pandey, U. N., Aggarwal, V., & Saini, S. (2023). *ESRO: Experience Assisted Service Reliability against Outages* (arXiv:2309.07230). arXiv. https://doi.org/10.48550/arXiv.2309.07230

Chakraborty, S., Agarwal, S., Garg, S., Sethia, A., Pandey, U. N., Aggarwal, V., & Saini, S. (2024). ESRO: Experience Assisted Service Reliability against Outages. *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering, ASE '23*, 255–267. https://doi.org/10.1109/ASE56229.2023.00131

Chang, B. R., Tsai, H.-F., & Chen, G.-R. (2024). Self-Adaptive Server Anomaly Detection Using Ensemble Meta-Reinforcement Learning. *Electronics*, *13*(12), 2348. https://doi.org/10.3390/electronics13122348

Chen, C., Yu, H., Lei, Z., Li, J., Ren, S., Zhang, T., Hu, S., Wang, J., & Shi, W. (2023). BALANCE: Bayesian Linear Attribution for Root Cause Localization. *Proceedings of the ACM on Management of Data*, *1*(1), 1–26. https://doi.org/10.1145/3588949

Chen, F., Zhang, Y., Fan, L., Liang, Y., Pang, G., Wen, Q., & Deng, S. (2024). Cluster-Wide Task Slowdown Detection in Cloud System. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 266–277. https://doi.org/10.1145/3637528.3671936

Chen, F., zhang, Y., Qin, Z., Fan, L., Jiang, R., Liang, Y., Wen, Q., & Deng, S. (2024). *Learning Multi-Pattern Normalities in the Frequency Domain for Efficient Time Series Anomaly Detection* (arXiv:2311.16191). arXiv. https://doi.org/10.48550/arXiv.2311.16191

Chen, J., Zhang, R., Chen, P., Ren, J., Wu, Z., Wang, Y., Li, X., & Xiong, L. (2024). MTG_CD: Multi-scale learnable transformation graph for fault classification and diagnosis in microservices. *Journal of Cloud Computing*, *13*(1), 103. https://doi.org/10.1186/s13677-024-00666-0

Chen, L., Zaharia, M., & Zou, J. (2023). *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance* (arXiv:2305.05176). arXiv. https://doi.org/10.48550/arXiv.2305.05176

Chen, N., Tu, H., Duan, X., Hu, L., & Guo, C. (2023). Semisupervised anomaly detection of multivariate time series based on a variational autoencoder. *Applied Intelligence*, *53*(5), 6074–6098. ABI/INFORM Collection (2778132167). https://doi.org/10.1007/s10489-022-03829-1

Chen, Y., Shetty, M., Somashekar, G., Ma, M., Simmhan, Y., Mace, J., Bansal, C., Wang, R., & Rajmohan, S. (2025). *AIOpsLab: A Holistic Framework to Evaluate AI Agents for*

Enabling Autonomous Clouds (arXiv:2501.06706). arXiv. https://doi.org/10.48550/arXiv.2501.06706

Chen, Y., Xie, H., Ma, M., Kang, Y., Gao, X., Shi, L., Cao, Y., Gao, X., Fan, H., Wen, M., Zeng, J., Ghosh, S., Zhang, X., Zhang, C., Lin, Q., Rajmohan, S., Zhang, D., & Xu, T. (2023). *Automatic Root Cause Analysis via Large Language Models for Cloud Incidents* (arXiv:2305.15778). arXiv. https://doi.org/10.48550/arXiv.2305.15778

Chen, Z., Kang, Y., Li, L., Zhang, X., Zhang, H., Xu, H., Zhou, Y., Yang, L., Sun, J., Xu, Z., Dang, Y., Gao, F., Zhao, P., Qiao, B., Lin, Q., Zhang, D., & Lyu, M. R. (2020). Towards intelligent incident management: why we need it and how we make it. *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1487–1497. https://doi.org/10.1145/3368089.3417055

Chen, Z., Liu, J., Gu, W., Su, Y., & Lyu, M. R. (2022). *Experience Report: Deep Learning-based System Log Analysis for Anomaly Detection* (arXiv:2107.05908). arXiv. https://doi.org/10.48550/arXiv.2107.05908

Cheng, Q., Sahoo, D., Saha, A., Yang, W., Liu, C., Woo, G., Singh, M., Saverese, S., & Hoi, S. C. H. (2023). *AI for IT Operations (AIOps) on Cloud Platforms: Reviews, Opportunities and Challenges* (arXiv:2304.04661). arXiv. https://doi.org/10.48550/arXiv.2304.04661

Cornacchia, A., Alabdulaal, I., Saghier, I., Mirdad, A., Fayoumi, O., & Canini, M. (2025). Between Promise and Pain: The Reality of Automating Failure Analysis in Microservices with LLMs. *Proceedings of the 16th ACM SIGOPS Asia-Pacific Workshop on Systems*, 155–167. https://doi.org/10.1145/3725783.3764388

D'Angelo, A., & d'Aloisio, G. (2024). Grammar-Based Anomaly Detection of Microservice Systems Execution Traces. *Companion of the 15th ACM/SPEC International Conference on Performance Engineering*, 77–81. https://doi.org/10.1145/3629527.3651844

Ding, R., Zhang, C., Wang, L., Xu, Y., Ma, M., Wu, X., Zhang, M., Chen, Q., Gao, X., Gao, X., Fan, H., Rajmohan, S., Lin, Q., & Zhang, D. (2023). TraceDiag: Adaptive, Interpretable, and Efficient Root Cause Analysis on Large-Scale Microservice Systems. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1762–1773. https://doi.org/10.1145/3611643.3613864

Duan, Y., Bao, H., Bai, G., Wei, Y., Xue, K., You, Z., Zhang, Y., Liu, B., Chen, J., Wang, S., & Ou, Z. (2024). Learning to Diagnose: Meta-Learning for Efficient Adaptation in Few-Shot AIOps Scenarios. *Electronics*, *13*(11), 2102. https://doi.org/10.3390/electronics13112102

Duan, Y., Xue, K., & Sun, H. (n.d.). *LogEDL: Log Anomaly Detection via Evidential Deep Learning*.

Duan, Y., Xue, K., Sun, H., Bao, H., Wei, Y., You, Z., Zhang, Y., Jiang, X., Yang, S., Chen, J., Duan, B., & Ou, Z. (2024). LogEDL: Log Anomaly Detection via Evidential Deep Learning. *Applied Sciences*, *14*(16), 7055. Publicly Available Content Database (3097822159). https://doi.org/10.3390/app14167055

Esposito, M., Bakhtin, A., Ahmad, N., Robredo, M., Su, R., Lenarduzzi, V., & Taibi, D. (2025). *Autonomic Microservice Management via Agentic AI and MAPE-K Integration* (arXiv:2506.22185). arXiv. https://doi.org/10.48550/arXiv.2506.22185

Fernando, D., Rodriguez, M. A., Arroba, P., Ismail, L., & Buyya, R. (2025). Efficient Training Approaches for Performance Anomaly Detection Models in Edge Computing Envi-

ronments. *ACM Transactions on Autonomous and Adaptive Systems*, *20*(2), 1–27. https://doi.org/10.1145/3725736

Fu, N., Cheng, G., Teng, Y., Dai, G., Yu, S., & Chen, Z. (2025). Intelligent Root Cause Localization in MicroService Systems: A Survey and New Perspectives. *ACM Computing Surveys*, *57*(12), 1–37. https://doi.org/10.1145/3736755

Gan, Y. (2021). Designing and Managing Large-Scale Interactive Microservices in Datacenters [Ph.D., Cornell University]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (2622279536). https://www.proquest.com/dissertations-theses/designing-managing-large-scale-interactive/docview/2622279536/se-2?accountid=25283

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2021). *Sage: Using Unsupervised Learning for Scalable Performance Debugging in Microservices* (arXiv:2101.00267). arXiv. https://doi.org/10.48550/arXiv.2101.00267

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2022a). Enabling Practical Cloud Performance Debugging with Unsupervised Learning. *ACM SIGOPS Operating Systems Review*, *56*(1), 34–41. https://doi.org/10.1145/3544497.3544503

Gan, Y., Liang, M., Dev, S., Lo, D., & Delimitrou, C. (2022b). Practical and Scalable ML-Driven Cloud Performance Debugging With Sage. *IEEE Micro*, *42*(4), 27–36. https://doi.org/10.1109/MM.2022.3169445

Gan, Y., Liu, G., Zhang, X., Zhou, Q., Wu, J., & Jiang, J. (2023). Sleuth: A Trace-Based Root Cause Analysis System for Large-Scale Microservices with Graph Neural Networks. *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 4*, 324–337. https://doi.org/10.1145/3623278.3624758

Gao, H., Xin, R., Chen, P., Li, X., Lu, N., & You, P. (2025). Memory-augment graph transformer based unsupervised detection model for identifying performance anomalies in highly-dynamic cloud environments. *Journal of Cloud Computing*, *14*(1), 40. https://doi.org/10.1186/s13677-025-00766-5

Ge, H., Ji, X., Peng, F., Chen, R., Xiang, N., Zhang, K., & Wu, W. (2024). SRdetector: Sequence Reconstruction Method for Microservice Anomaly Detection. *Electronics*, *14*(1), 65. https://doi.org/10.3390/electronics14010065

Girish, L., & Rao, S. K. N. (2023). Anomaly detection in cloud environment using artificial intelligence techniques. *Computing*, *105*(3), 675–688. https://doi.org/10.1007/s00607-021-00941-x

Gregor, S., & Hevner, A. R. (2013, June 1). *Positioning and Presenting Design Science Research for Maximum Impact1 | Management Information Systems Quarterly | MIS Quarterly*. https://misq.umn.edu/misq/article-abstract/37/2/337/103/Positioning-and-Presenting-Design-Science-Research

Gu, W., Zhong, R., Yu, G., Sun, X., Liu, J., Huo, Y., Chen, Z., Zhang, J., Gu, J., Yang, Y., & Lyu, M. R. (2025). *KPIRoot+: An Efficient Integrated Framework for Anomaly Detection and Root Cause Analysis in Large-Scale Cloud Systems* (arXiv:2506.04569). arXiv. https://doi.org/10.48550/arXiv.2506.04569

Gu, Y., Xiong, Y., Mace, J., Jiang, Y., Hu, Y., Kasikci, B., & Cheng, P. (2025). *Argos: Agentic Time-Series Anomaly Detection with Autonomous Rule Generation via Large Language Models* (arXiv:2501.14170). arXiv. https://doi.org/10.48550/arXiv.2501.14170

Gulenko, A. (2020). *Autonomic self-healing in cloud computing platforms*.

Guntupalli, R. (2025). *AI-driven anomaly detection and root cause analysis: Using machine learning on logs, metrics, and traces to detect subtle performance anomalies, security threats, or failures in complex cloud environments*. https://doi.org/https://doi.org/10.30574/wjarr.2025.26.2.1521

Gupta, L. (2019). Management and Security of Multi-Cloud Applications [Ph.D., Washington University in St. Louis]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (2212217278). https://www.proquest.com/dissertations-theses/management-security-multi-cloud-applications/docview/2212217278/se-2?accountid=25283

Hagemann, T., & Katsarou, K. (2020a). A Systematic Review on Anomaly Detection for Cloud Computing Environments. *2020 3rd Artificial Intelligence and Cloud Computing Conference*, 83–96. https://doi.org/10.1145/3442536.3442550

Hagemann, T., & Katsarou, K. (2020b). Reconstruction-based anomaly detection for the cloud: A comparison on the Yahoo! Webscope S5 dataset. *Proceedings of the 2020 4th International Conference on Cloud and Big Data Computing*, 68–75. https://doi.org/10.1145/3416921.3416934

Han, Y., Du, Q., Huang, Y., Wu, J., Tian, F., & He, C. (2024). The Potential of One-Shot Failure Root Cause Analysis: Collaboration of the Large Language Model and Small Classifier. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 931–943. https://doi.org/10.1145/3691620.3695475

Hardt, M., Orchard, W. R., Blöbaum, P., Kasiviswanathan, S., & Kirschbaum, E. (2024). *The PetShop Dataset -- Finding Causes of Performance Issues across Microservices* (arXiv:2311.04806). arXiv. https://doi.org/10.48550/arXiv.2311.04806

Harutyunyan, A., Poghosyan, A., Harutyunyan, L., Aghajanyan, N., Bunarjyan, T., & Vinck, A. J. H. (2023). Challenges and Experiences in Designing Interpretable KPI-diagnostics for Cloud Applications. *JUCS - Journal of Universal Computer Science*, *29*(11), 1298–1318. https://doi.org/10.3897/jucs.112570

He, H., Li, X., Chen, P., Chen, J., Liu, M., & Wu, L. (2024). Efficiently localizing system anomalies for cloud infrastructures: a novel Dynamic Graph Transformer based Parallel Framework. *Journal of Cloud Computing*, *13*(1), 115. https://doi.org/10.1186/s13677-024-00677-x

He, Z., Hu, G., & Lee, R. B. (2023). CloudShield: Real-time Anomaly Detection in the Cloud. *Proceedings of the Thirteenth ACM Conference on Data and Application Security and Privacy*, 91–102. https://doi.org/10.1145/3577923.3583639

Heeb, Z., Kalinagac, O., Soussi, W., & Gür, G. (2023). IoMiRCA: Root cause analysis in IoT-extended 5G microservice environments. *Proceedings of the 38th ACM/SIGAPP Symposium on Applied Computing*, 106–108. https://doi.org/10.1145/3555776.3577840

Heinrich, R., Van Hoorn, A., Knoche, H., Li, F., Lwakatare, L. E., Pahl, C., Schulte, S., & Wettinger, J. (2017). Performance Engineering for Microservices: Research Challenges and Directions. *Proceedings of the 8th ACM/SPEC on International Conference on Performance Engineering Companion*, 223–226. https://doi.org/10.1145/3053600.3053653

Hevner, A. R. (2007). *A Three Cycle View of Design Science Research*. *19*.

Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). *Design Science in Information Systems Research1*. https://dx.doi.org/10.2307/25148625

Hou, J. (2025). *Research on fault diagnosis and root cause analysis based on full stack observability* (arXiv:2509.12231). arXiv. https://doi.org/10.48550/arXiv.2509.12231

Hrusto, A., Engström, E., & Runeson, P. (2022). Optimization of anomaly detection in a microservice system through continuous feedback from development. *Proceedings of the 10th IEEE/ACM International Workshop on Software Engineering for Systems-of-Systems and Software Ecosystems*, 13–20. https://doi.org/10.1145/3528229.3529382

Hu, J., Long, L., Sui, H., Gu, Z., & Zheng, G. (2025). CFTL: System Log Parsing Method Driven from Clustering According to First Token and Length for Anomaly Detection. *Applied Sciences*, *15*(4), 1740. https://doi.org/10.3390/app15041740

Huang, H., Zhang, X., Chen, P., He, Z., Chen, Z., Yu, G., Chen, H., & Sun, C. (2024). TraStrainer: Adaptive Sampling for Distributed Traces with System Runtime State. *Proceedings of the ACM on Software Engineering*, *1*(FSE), 473–493. https://doi.org/10.1145/3643748

Huang, J., Chen, P., Yu, G., Wang, Y., Huang, H., & He, Z. (2024). *FaaSRCA: Full Lifecycle Root Cause Analysis for Serverless Applications* (arXiv:2412.02239). arXiv. https://doi.org/10.48550/arXiv.2412.02239

Huang, J., Yang, Y., Yu, H., Li, J., & Zheng, X. (2023). *Twin Graph-based Anomaly Detection via Attentive Multi-Modal Learning for Microservice System* (arXiv:2310.04701). arXiv. https://doi.org/10.48550/arXiv.2310.04701

Huang, T., Chen, P., & Li, R. (2022). A Semi-Supervised VAE Based Active Anomaly Detection Framework in Multivariate Time Series for Online Systems. *Proceedings of the ACM Web Conference 2022*, 1797–1806. https://doi.org/10.1145/3485447.3511984

Hwang, J., Shwartz, L., Wang, Q., Batta, R., Kumar, H., & Nidd, M. (2021). *FIXME: Enhance Software Reliability with Hybrid Approaches in Cloud* (arXiv:2102.09336). arXiv. https://doi.org/10.48550/arXiv.2102.09336

Islam, M. S., Rakha, M. S., Pourmajidi, W., Sivaloganathan, J., Steinbacher, J., & Miranskyy, A. (2025). Anomaly Detection in Large-Scale Cloud Systems: An Industry Case and Dataset. *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 377–388. https://doi.org/10.1109/ICSE-SEIP66354.2025.00039

Jha, N., Lin, S., Jayaraman, S., Frohling, K., Constantinides, C., & Patel, D. (2025). *LLM Assisted Anomaly Detection Service for Site Reliability Engineers: Enhancing Cloud Infrastructure Resilience* (arXiv:2501.16744). arXiv. https://doi.org/10.48550/arXiv.2501.16744

Ji, C., & Luo, H. (2025a). *Cloud-Based AI Systems: Leveraging Large Language Models for Intelligent Fault Detection and Autonomous Self-Healing* (arXiv:2505.11743). arXiv. https://doi.org/10.48550/arXiv.2505.11743

Ji, C., & Luo, H. (2025b). *Leveraging Large Language Model for Intelligent Log Processing and Autonomous Debugging in Cloud AI Platforms* (arXiv:2506.17900). arXiv. https://doi.org/10.48550/arXiv.2506.17900

Ji, X., Zhang, L., Zhang, W., Peng, F., Mao, Y., Liao, X., & Zhang, K. (2025). LEMAD: LLM-Empowered Multi-Agent System for Anomaly Detection in Power Grid Services. *Electronics*, *14*(15), 3008. https://doi.org/10.3390/electronics14153008

Jian, H. (n.d.). *Research on fault diagnosis and root cause analysis based on full stack observability*.

Jiang, G. (2024). Artificial intelligence-based adaptive anomaly detection technology for IaaS cloud virtual machines. *Journal of Engineering and Applied Science*, *71*(1), 102. https://doi.org/10.1186/s44147-024-00439-1

Jiang, X., Pan, Y., Ma, M., & Wang, P. (2023). Look Deep into the Microservice System Anomaly through Very Sparse Logs. *Proceedings of the ACM Web Conference 2023*, 2970–2978. https://doi.org/10.1145/3543507.3583338

Jin, S., & Bei, Z. (n.d.). *Breaking the Cycle of Recurring Failures: Applying Generative AI to Root Cause Analysis in Legacy Banking Systems*.

Jin, S., Bei, Z., Chen, B., & Xia, Y. (2024). *Breaking the Cycle of Recurring Failures: Applying Generative AI to Root Cause Analysis in Legacy Banking Systems* (arXiv:2411.13017). arXiv. https://doi.org/10.48550/arXiv.2411.13017

Jin, Y., Yang, Z., Liu, J., & Xu, X. (2025). *Anomaly Detection and Early Warning Mechanism for Intelligent Monitoring Systems in Multi-Cloud Environments Based on LLM* (arXiv:2506.07407). arXiv. https://doi.org/10.48550/arXiv.2506.07407

Jonnaikode, S. (n.d.). *Detecting Software Anomalies in Microservices Using Spectrograms, Scalograms, and Convolutional Neural Networks*.

Jonnaikode, S. (2025). Detecting Software Anomalies in Microservices using Spectrograms, Scalograms, and Convolutional Neural Networks [M.S., California Polytechnic State University]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (3235008860). https://www.proquest.com/dissertations-theses/detecting-software-anomalies-microservices-using/docview/3235008860/se-2?accountid=25283

Joulani, P., Gyorgy, A., & Szepesvari, C. (2013). Online Learning under Delayed Feedback. *Proceedings of the 30th International Conference on Machine Learning*, 1453–1461. https://proceedings.mlr.press/v28/joulani13.html

Khodabandeh, G., Ezaz, A., Babaei, M., & Ezzati-Jivan, N. (2025). *Utilizing Graph Neural Networks for Effective Link Prediction in Microservice Architectures* (arXiv:2501.15019). arXiv. https://doi.org/10.48550/arXiv.2501.15019

Kohli, R., & Grover, V. (2008). Business value of IT: An agenda for research. *Information Systems Research*, *19*(1), 26–40. https://doi.org/10.1287/isre.1070.0150

L. Wu, J. Tordsson, E. Elmroth, & O. Kao. (2020). MicroRCA: Root Cause Localization of Performance Issues in Microservices. *NOMS 2020 - 2020 IEEE/IFIP Network Operations and Management Symposium*, 1–9. https://doi.org/10.1109/NOMS47738.2020.9110353

Lee, C., Yang, T., Chen, Z., Su, Y., & Lyu, M. R. (2023a). *Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-source Data* (arXiv:2302.05092). arXiv. https://doi.org/10.48550/arXiv.2302.05092

Lee, C., Yang, T., Chen, Z., Su, Y., & Lyu, M. R. (2023b). Eadro: An End-to-End Troubleshooting Framework for Microservices on Multi-Source Data. *Proceedings of the 45th International Conference on Software Engineering, ICSE '23*, 1750–1762. https://doi.org/10.1109/ICSE48619.2023.00150

Li, L., Chu, W., Langford, J., & Schapire, R. E. (2010). A contextual-bandit approach to personalized news article recommendation. *Proceedings of the 19th International Conference on World Wide Web, WWW '10*, 661–670. https://doi.org/10.1145/1772690.1772758

Lin, C.-M., Chang, C., Wang, W.-Y., Wang, K.-D., & Peng, W.-C. (2024). *Root Cause Analysis In Microservice Using Neural Granger Causal Discovery* (arXiv:2402.01140). arXiv. https://doi.org/10.48550/arXiv.2402.01140

Lin, Y. (n.d.). *Self-Supervised Distributed Machine Learning for Robust Containerized Systems*.

Liu, D., He, C., Peng, X., Lin, F., Zhang, C., Gong, S., Li, Z., Ou, J., & Wu, Z. (2021). MicroHECL: High-Efficient Root Cause Localization in Large-Scale Microservice Systems. *2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 338–347. https://doi.org/10.1109/ICSE-SEIP52600.2021.00043

Liu, Y. (2021). Enable the Landing of Internet of Things: A Holistic Approach [Ph.D., Linkopings Universitet (Sweden)]. In *PQDT - Global*. ProQuest Dissertations & Theses Global (3070086618). https://doi.org/10.3384/diss.diva-175383

Liu, Y., Pei, C., Xu, L., Chen, B., Sun, M., Zhang, Z., Sun, Y., Zhang, S., Wang, K., Zhang, H., Li, J., Xie, G., Wen, X., Nie, X., Ma, M., & Pei, D. (2025). OpsEval: A Comprehensive Benchmark Suite for Evaluating Large Language Models' Capability in IT Operations Domain. *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 503–513. https://doi.org/10.1145/3696630.3728572

Ma, M., Xu, J., Wang, Y., Chen, P., Zhang, Z., & Wang, P. (2020). AutoMAP: Diagnose Your Microservice-based Web Applications Automatically. *Proceedings of The Web Conference 2020*, 246–258. https://doi.org/10.1145/3366423.3380111

Melville, N., Kraemer, K., & Gurbaxani, V. (2004). Review: Information technology and organizational performance: An integrative model of IT business value. *MIS Quarterly*, *28*(2), 283–322. https://doi.org/10.2307/25148636

Mingers, J. (2004). Real-izing information systems: critical realism as an underpinning philosophy for information systems. *Information and Organization, Critical Realism*, *14*(2), 87–103. https://doi.org/10.1016/j.infoandorg.2003.06.001

Nikiforov, R. (2018). *Clustering-based Anomaly Detection for microservices* (arXiv:1810.02762). arXiv. https://doi.org/10.48550/arXiv.1810.02762

Nobre, J., Pires, E. J. S., & Reis, A. (2023a). Anomaly Detection in Microservice-Based Systems. *Applied Sciences*, *13*(13), 7891. https://doi.org/10.3390/app13137891

Nobre, J., Pires, E. J. S., & Reis, A. (2023b). Anomaly Detection in Microservice-Based Systems. *Applied Sciences*, *13*(13), 7891. https://doi.org/10.3390/app13137891

Noguchi, K., Gel, Y. R., Brunner, E., & Konietschke, F. (2012). nparLD: An R Software Package for the Nonparametric Analysis of Longitudinal Data in Factorial Experiments. *Journal of Statistical Software*, *50*, 1–23. https://doi.org/10.18637/jss.v050.i12

Noor, A. I. (2021). Real-Time QoS Monitoring and Anomaly Detection on Microservice-Based Applications in Cloud-Edge Infrastructure [Ph.D., University of Newcastle Upon Tyne (United Kingdom)]. In *PQDT - UK & Ireland*. ProQuest Dissertations & Theses Global (2796835651). https://www.proquest.com/dissertations-theses/real-time-qos-monitoring-anomaly-detection-on/docview/2796835651/se-2?accountid=25283

Noor, A. I. (n.d.). *Real-Time QoS Monitoring and Anomaly Detection on Microservice-based Applications in Cloud-Edge Infrastructure*.

Panwar, A., Pal, H., Chen, J., Cho, K., Jiang, R., Zhao, M., & Krishnamurthy, R. (2025). *Reasoning-based Anomaly Detection Framework: A Real-time, Scalable, and Automated Approach to Anomaly Detection Across Domains* (arXiv:2510.03486). arXiv. https://doi.org/10.48550/arXiv.2510.03486

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, *24*(3), 45–77. https://doi.org/10.2753/MIS0742-1222240302

Pei, C., Wang, Z., Liu, F., Li, Z., Liu, Y., He, X., Kang, R., Zhang, T., Chen, J., Li, J., Xie, G., & Pei, D. (2025). *Flow-of-Action: SOP Enhanced LLM-Based Multi-Agent System for Root Cause Analysis* (arXiv:2502.08224). arXiv. https://doi.org/10.48550/arXiv.2502.08224

Petrov, V., Gennadinik, A., & Avksentieva, E. (2022). Metrics for machine learning evaluation methods in cloud monitoring systems. *2022 8th International Conference on Computer Technology Applications*, 168–175. https://doi.org/10.1145/3543712.3543715

Pham, L., Ha, H., & Zhang, H. (2024a). BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection. *Proceedings of the ACM on Software Engineering*, *1*(FSE), 2214–2237. https://doi.org/10.1145/3660805

Pham, L., Ha, H., & Zhang, H. (2024b). *BARO: Robust Root Cause Analysis for Microservices via Multivariate Bayesian Online Change Point Detection*. *1*(FSE), 2214–2237. https://doi.org/10.1145/3660805

Pham, L., Ha, H., & Zhang, H. (2024c). Root Cause Analysis for Microservice System based on Causal Inference: How Far Are We? *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 706–715. https://doi.org/10.1145/3691620.3695065

Pham, L., Ha, H., & Zhang, H. (2024d). *Root Cause Analysis for Microservice System based on Causal Inference: How Far Are We?* (arXiv:2408.13729). arXiv. https://doi.org/10.48550/arXiv.2408.13729

Poghosyan, A., Harutyunyan, A., Grigoryan, N., & Pang, C. (2023). Distributed Tracing for Troubleshooting of Native Cloud Applications via Rule-Induction Systems. *JUCS - Journal of Universal Computer Science*, *29*(11), 1274–1297. https://doi.org/10.3897/jucs.112513

Qi, Z., Xie, H., Li, Y., Tan, J., Li, F., & Lui, J. C. S. (2022). *LPC-AD: Fast and Accurate Multivariate Time Series Anomaly Detection via Latent Predictive Coding* (arXiv:2205.08362). arXiv. https://doi.org/10.48550/arXiv.2205.08362

Qiu, H., Banerjee, S. S., Jha, S., Kalbarczyk, Z. T., & Iyer, R. K. (2020). *FIRM: An Intelligent Fine-Grained Resource Management Framework for SLO-Oriented Microservices* (arXiv:2008.08509). arXiv. https://doi.org/10.48550/arXiv.2008.08509

Qiu, J., Du, Q., & Qian, C. (2019). KPI-TSAD: A Time-Series Anomaly Detector for KPI Monitoring in Cloud Applications. *Symmetry*, *11*(11), 1350. https://doi.org/10.3390/sym11111350

Rogers, E. M. (2003). *Diffusion of innovations* (5th ed.). Free Press.

Rouf, R., Rasolroveicy, M., Litoiu, M., Nagar, S., Mohapatra, P., Gupta, P., & Watts, I. (2024). InstantOps: A Joint Approach to System Failure Prediction and Root Cause Identification in Microserivces Cloud-Native Applications. *Proceedings of the 15th ACM/SPEC International Conference on Performance Engineering*, 119–129. https://doi.org/10.1145/3629526.3645047

Sarda, K., Namrud, Z., & Rouf, R. (2023). *ADARMA: Auto-Detection and Auto-Remediation of Microservice Anomalies by Leveraging Large Language Models*.

Sarika, P. K., Badampudi, D., Josyula, S. P., & Usman, M. (2023). Automating Microservices Test Failure Analysis using Kubernetes Cluster Logs. *Proceedings of the 27th International Conference on Evaluation and Assessment in Software Engineering*, 192–195. https://doi.org/10.1145/3593434.3593472

Shan, A., Kaur, J., Singh, R., Banka, T., Yavatkar, R., & Sridhar, T. (2025). *RCA Copilot: Transforming Network Data into Actionable Insights via Large Language Models* (arXiv:2507.03224). arXiv. https://doi.org/10.48550/arXiv.2507.03224

Shetty, M., Bansal, C., Kumar, S., Rao, N., & Nagappan, N. (2021). *SoftNER: Mining Knowledge Graphs From Cloud Incidents* (arXiv:2101.05961). arXiv. https://doi.org/10.48550/arXiv.2101.05961

Shetty, M., Bansal, C., Kumar, S., Rao, N., Nagappan, N., & Zimmermann, T. (2021). *Neural Knowledge Extraction From Cloud Service Incidents* (arXiv:2007.05505). arXiv. https://doi.org/10.48550/arXiv.2007.05505

Shetty, M., Chen, Y., Somashekar, G., Ma, M., Simmhan, Y., Zhang, X., Mace, J., Vandevoorde, D., Las-Casas, P., Gupta, S. M., Nath, S., Bansal, C., & Rajmohan, S. (2024). *Building AI Agents for Autonomous Clouds: Challenges and Design Principles* (arXiv:2407.12165). arXiv. https://doi.org/10.48550/arXiv.2407.12165

Shi, B., Luo, Y., Wang, J., Zhao, Y., Zhang, S., Hao, B., Zhao, C., Sun, Y., Zhang, Z., Sun, R., Li, H., Song, W., Chen, X., Miao, J., & Pei, D. (2025). FlowXpert: Expertizing Troubleshooting Workflow Orchestration with Knowledge Base and Multi-Agent Coevolution. *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V.2*, 4839–4850. https://doi.org/10.1145/3711896.3737221

Shivakumar, M. (n.d.). *AIOps–Driven Adaptive Anomaly Detection in Evolving Cloud Environments Using Transfer Learning*.

Shivakumar, M. (2025). Aiops–driven Adaptive Anomaly Detection in Evolving Cloud Environments using Transfer Learning [M.S., California Polytechnic State University]. In *ProQuest Dissertations and Theses*. ProQuest Dissertations & Theses Global (3235005709). https://www.proquest.com/dissertations-theses/aiops-driven-adaptive-anomaly-detection-evolving/docview/3235005709/se-2?accountid=25283

Soldani, J., & Brogi, A. (2023). Anomaly Detection and Failure Root Cause Analysis in (Micro) Service-Based Cloud Applications: A Survey. *ACM Computing Surveys*, *55*(3), 1–39. https://doi.org/10.1145/3501297

Somashekar, G., Dutt, A., Adak, M., Lorido Botran, T., & Gandhi, A. (2024). GAMMA: Graph Neural Network-Based Multi-Bottleneck Localization for Microservices Applications. *Proceedings of the ACM Web Conference 2024*, 3085–3095. https://doi.org/10.1145/3589334.3645665

Sun, Y., Lin, Z., Shi, B., Zhang, S., Ma, S., Jin, P., Zhong, Z., Pan, L., Guo, Y., & Pei, D. (2025). Interpretable Failure Localization for Microservice Systems Based on Graph Autoencoder. *ACM Transactions on Software Engineering and Methodology*, *34*(2), 1–28. https://doi.org/10.1145/3695999

Sun, Y., Luo, Y., Wen, X., Yuan, Y., Nie, X., Zhang, S., Liu, T., & Luo, X. (2025). *TrioXpert: An Automated Incident Management Framework for Microservice System* (arXiv:2506.10043). arXiv. https://doi.org/10.48550/arXiv.2506.10043

Sun, Y., Shi, B., Mao, M., Ma, M., Xia, S., Zhang, S., & Pei, D. (2024). ART: A Unified Unsupervised Framework for Incident Management in Microservice Systems. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1183–1194. https://doi.org/10.1145/3691620.3695495

Sun, Y., Zheng, T., Wen, X., Kuang, W., Liu, H., Zhang, S., Shen, C., Wu, B., & Pei, D. (2025). A Multimodal Intelligent Change Assessment Framework for Microservice Systems Based on Large Language Models. *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 378–388. https://doi.org/10.1145/3696630.3728561

Tang, P., Tang, S., Pu, H., Miao, Z., & Wang, Z. (2025). *MicroRCA-Agent: Microservice Root Cause Analysis Method Based on Large Language Model Agents* (arXiv:2509.15635). arXiv. https://doi.org/10.48550/arXiv.2509.15635

Tao, L., Zhang, S., Jia, Z., Sun, J., Ma, M., Li, Z., Sun, Y., Yang, C., Zhang, Y., & Pei, D. (2024). Giving Every Modality a Voice in Microservice Failure Diagnosis via Multimodal Adaptive Optimization. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1107–1119. https://doi.org/10.1145/3691620.3695489

Tien, C., Huang, T., Tien, C., Huang, T., & Kuo, S. (2019). KubAnomaly: Anomaly detection for the Docker orchestration platform with neural network approaches. *Engineering Reports*, *1*(5), e12080. https://doi.org/10.1002/eng2.12080

Vervaet, A. (2023). *MoniLog: An Automated Log-Based Anomaly Detection System for Cloud Computing Infrastructures* (arXiv:2304.11940). arXiv. https://doi.org/10.48550/arXiv.2304.11940

Wang, C., Huang, T., Li, M., Chen, P., & Chen, Z. (2024). A Bayesian LSTM Based Active Anomaly Detection Service for Large Online Systems. *Proceedings of the 15th Asia-Pacific Symposium on Internetware*, 407–416. https://doi.org/10.1145/3671016.3674818

Wang, H., Wu, Z., Jiang, H., Huang, Y., Wang, J., Kopru, S., & Xie, T. (2021). *Groot: An Event-graph-based Approach for Root Cause Analysis in Industrial Settings* (arXiv:2108.00344). arXiv. https://doi.org/10.48550/arXiv.2108.00344

Wang, H., Wu, Z., Jiang, H., Huang, Y., Wang, J., Kopru, S., & Xie, T. (2022). Groot: an event-graph-based approach for root cause analysis in industrial settings. *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering, ASE '21*, 419–429. https://doi.org/10.1109/ASE51524.2021.9678708

Wang, L., Zhang, C., Ding, R., Xu, Y., Chen, Q., Zou, W., Chen, Q., Zhang, M., Gao, X., Fan, H., Rajmohan, S., Lin, Q., & Zhang, D. (2023). Root Cause Analysis for Microservice Systems via Hierarchical Reinforcement Learning from Human Feedback. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5116–5125. https://doi.org/10.1145/3580305.3599934

Wang, S., Sandeepa, C., Senevirathna, T., Siniarski, B., Nguyen, M.-D., Marchal, S., & Liyanage, M. (2024). *Towards Accountable and Resilient AI-Assisted Networks: Case Studies and Future Challenges*. https://doi.org/10.1109/eucnc/6gsummit60053.2024.10597060

Wang, T., & Qi, G. (2024). *A Comprehensive Survey on Root Cause Analysis in (Micro) Services: Methodologies, Challenges, and Trends* (arXiv:2408.00803). arXiv. https://doi.org/10.48550/arXiv.2408.00803

Wang, T., Qi, G., & Wu, T. (2024). *KGroot: Enhancing Root Cause Analysis through Knowledge Graphs and Graph Convolutional Neural Networks* (arXiv:2402.13264). arXiv. https://doi.org/10.48550/arXiv.2402.13264

Wang, Y., Li, G., Wang, Z., Kang, Y., Zhou, Y., Zhang, H., Gao, F., Sun, J., Yang, L., Lee, P., Xu, Z., Zhao, P., Qiao, B., Li, L., Zhang, X., & Lin, Q. (2021). *Fast Outage Analysis of Large-scale Production Clouds with Service Correlation Mining* (arXiv:2103.03649). arXiv. https://doi.org/10.48550/arXiv.2103.03649

Wang, Y., Zhu, Z., Fu, Q., Ma, Y., & He, P. (2024). MRCA: Metric-level Root Cause Analysis for Microservices via Multi-Modal Data. *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 1057–1068. https://doi.org/10.1145/3691620.3695485

Wang, Y., Mäntylä, M. V., Demeyer, S., Beyazit, M., Kisaakye, J., & Nyyssölä, J. (2025). Cross-System Categorization of Abnormal Traces in Microservice-Based Systems via Meta-Learning. *Proceedings of the ACM on Software Engineering*, *2*(FSE), 576–598. https://doi.org/10.1145/3715742

Wang, Z., Cornacchia, A., Sacco, A., Galante, F., Canini, M., & Jiang, D. (2025). *A Network Arena for Benchmarking AI Agents on Network Troubleshooting* (arXiv:2512.16381). arXiv. https://doi.org/10.48550/arXiv.2512.16381

Xiang, Y., Chen, C. P., Zeng, L., Yin, W., Liu, X., Li, H., & Xu, W. (2025). *Simplifying Root Cause Analysis in Kubernetes with StateGraph and LLM* (arXiv:2506.02490). arXiv. https://doi.org/10.48550/arXiv.2506.02490

Xie, S., Wang, J., He, H., Wang, Z., Zhao, Y., Zhang, N., & Li, B. (2026). TVDiag: A Task-oriented and View-invariant Failure Diagnosis Framework for Microservice-based Systems with Multimodal Data. *ACM Transactions on Software Engineering and Methodology*, *35*(2), 1–39. https://doi.org/10.1145/3734868

Xie, Z., Zhang, S., Geng, Y., Zhang, Y., Ma, M., Nie, X., Yao, Z., Xu, L., Sun, Y., Li, W., & Pei, D. (2024). Microservice Root Cause Analysis With Limited Observability Through Intervention Recognition in the Latent Space. *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 6049–6060. https://doi.org/10.1145/3637528.3671530

Xin, R., Liu, H., Chen, P., Grosso, P., & Zhao, Z. (2024). FIRED: a fine-grained robust performance diagnosis framework for cloud applications. *Future Generation Computer Systems*, *155*, 300–311. https://doi.org/10.1016/j.future.2024.02.014

Xing, S., Wang, Y., & Liu, W. (2025). Multi-Dimensional Anomaly Detection and Fault Localization in Microservice Architectures: A Dual-Channel Deep Learning Approach with Causal Inference for Intelligent Sensing. *Sensors*, *25*(11), 3396. https://doi.org/10.3390/s25113396

Yao, Z., Pei, C., Chen, W., Wang, H., Su, L., Jiang, H., Xie, Z., Nie, X., & Pei, D. (2024). Chain-of-Event: Interpretable Root Cause Analysis for Microservices through Automatically Learning Weighted Event Causal Graph. *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 50–61. https://doi.org/10.1145/3663529.3663827

Yu, G., Chen, P., Chen, H., Guan, Z., Huang, Z., Jing, L., Weng, T., Sun, X., & Li, X. (2021). MicroRank: End-to-End Latency Issue Localization with Extended Spectrum Analysis in Microservice Environments. *Proceedings of the Web Conference 2021*, 3087–3098. https://doi.org/10.1145/3442381.3449905

Yu, G., Chen, P., Li, Y., Chen, H., Li, X., & Zheng, Z. (2023). Nezha: Interpretable Fine-Grained Root Causes Analysis for Microservices on Multi-modal Observability Data. *Proceedings of the 31st ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 553–565. https://doi.org/10.1145/3611643.3616249

Zhang, C., Dong, Z., Peng, X., Zhang, B., & Chen, M. (2024). Trace-based Multi-Dimensional Root Cause Localization of Performance Issues in Microservice Systems. *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, 1–12. https://doi.org/10.1145/3597503.3639088

Zhang, C., Peng, X., Sha, C., Zhang, K., Fu, Z., Wu, X., Lin, Q., & Zhang, D. (2022). DeepTraLog: trace-log combined microservice anomaly detection through graph-based deep learn-

ing. *Proceedings of the 44th International Conference on Software Engineering*, 623–634. https://doi.org/10.1145/3510003.3510180

Zhang, C., Peng, X., Zhou, T., Sha, C., Yan, Z., Chen, Y., & Yang, H. (2022). TraceCRL: contrastive representation learning for microservice trace analysis. *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 1221–1232. https://doi.org/10.1145/3540250.3549146

Zhang, S., Jin, P., Lin, Z., Sun, Y., Zhang, B., Xia, S., Li, Z., Zhong, Z., Ma, M., Jin, W., Zhang, D., Zhu, Z., & Pei, D. (2023). *Robust Failure Diagnosis of Microservice System through Multimodal Data* (arXiv:2302.10512). arXiv. https://doi.org/10.48550/arXiv.2302.10512

Zhang, S., Xia, S., Fan, W., Shi, B., Xiong, X., Zhong, Z., Ma, M., Sun, Y., & Pei, D. (2026). Failure Diagnosis in Microservice Systems: A Comprehensive Survey and Analysis. *ACM Transactions on Software Engineering and Methodology*, *35*(1), 1–55. https://doi.org/10.1145/3715005

Zhang, S., Zhu, J., Hao, B., Sun, Y., Nie, X., Zhu, J., Liu, X., Li, X., Ma, Y., & Pei, D. (2024). Fault Diagnosis for Test Alarms in Microservices through Multi-source Data. *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*, 115–125. https://doi.org/10.1145/3663529.3663833

Zhang, W., Guo, H., Yang, J., Tian, Z., Zhang, Y., Yan, C., Li, Z., Li, T., Shi, X., Zheng, L., & Zhang, B. (2024). *mABC: multi-Agent Blockchain-Inspired Collaboration for root cause analysis in micro-services architecture* (arXiv:2404.12135). arXiv. https://doi.org/10.48550/arXiv.2404.12135

Zhang, X., Wang, Q., Li, M., Yuan, Y., Xiao, M., Zhuang, F., & Yu, D. (2025). *TAMO: Fine-Grained Root Cause Analysis via Tool-Assisted LLM Agent with Multi-Modality Observa-*

tion Data in Cloud-Native Systems (arXiv:2504.20462). arXiv. https://doi.org/10.48550/arXiv.2504.20462

Zhang, X., Ghosh, S., Bansal, C., Wang, R., Ma, M., Kang, Y., & Rajmohan, S. (2024). *Automated Root Causing of Cloud Incidents using In-Context Learning with GPT-4* (arXiv:2401.13810). arXiv. https://doi.org/10.48550/arXiv.2401.13810

Zhang, Y., Guan, Z., Qian, H., Xu, L., Liu, H., Wen, Q., Sun, L., Jiang, J., Fan, L., & Ke, M. (2021). CloudRCA: A Root Cause Analysis Framework for Cloud Computing Platforms. *Proceedings of the 30th ACM International Conference on Information & Knowledge Management*, 4373–4382. https://doi.org/10.1145/3459637.3481903

Zhao, C., Ma, M., Zhong, Z., Zhang, S., Tan, Z., Xiong, X., Yu, L., Feng, J., Sun, Y., Zhang, Y., Pei, D., Lin, Q., & Zhang, D. (2023). Robust Multimodal Failure Detection for Microservice Systems. *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 5639–5649. https://doi.org/10.1145/3580305.3599902

Zhao, Z., Wang, Z., Zhang, T., Shen, Z., Dong, H., Lei, Z., Ma, X., Xu, G., Ding, Z., & Yang, Y. (2025). *CHASE: A Causal Hypergraph based Framework for Root Cause Analysis in Multimodal Microservice Systems* (arXiv:2406.19711). arXiv. https://doi.org/10.48550/arXiv.2406.19711

Zheng, L., Chen, Z., & Chen, H. (2025). *Online Multi-modal Root Cause Identification in Microservice Systems* (arXiv:2410.10021). arXiv. https://doi.org/10.48550/arXiv.2410.10021

Zheng, L., Chen, Z., He, J., & Chen, H. (2024). MULAN: Multi-modal Causal Structure Learning and Root Cause Analysis for Microservice Systems. *Proceedings of the ACM Web Conference 2024*, 4107–4116. https://doi.org/10.1145/3589334.3645442

Zhou, X., Peng, X., Xie, T., Sun, J., Ji, C., Liu, D., Xiang, Q., & He, C. (2019). Latent error prediction and fault localization for microservice applications by learning from system trace logs. *Proceedings of the 2019 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 683–694. https://doi.org/10.1145/3338906.3338961

Zhu, Y., Wang, J., Li, B., Tang, X., Li, H., Zhang, N., & Zhao, Y. (2024). *Root Cause Localization for Microservice Systems in Cloud-edge Collaborative Environments* (arXiv:2406.13604). arXiv. https://doi.org/10.48550/arXiv.2406.13604

Zhu, Z., Lee, C., Tang, X., & He, P. (2024). HeMiRCA: Fine-Grained Root Cause Analysis for Microservices with Heterogeneous Data Sources. *ACM Transactions on Software Engineering and Methodology*, *33*(8), 1–25. https://doi.org/10.1145/3674726